Today 12:10 PM

DaPlanStan_Part2_Complete_Submiss...
File

daplanstan-main(1).zip
File

casestudy(2).tex
File

DP_Csestudy_instructions(2).pdf
PDF

DP_Csestudy(2).pdf
PDF

I am now starting the Software Evolution and Refactoring part of our DaPlanStan case study.

The previous phase completed Design Pattern Discovery and Verification. Treat the uploaded Part 2 package as the frozen BEFORE baseline and the uploaded daplanstan-main.zip as the original source snapshot.

Do not redo baseline pattern discovery and do not modify the frozen baseline.

The frozen baseline contains:

P01 — Information Expert (GRASP) — Parsed Email to Trip Record Linking
P02 — Controller (GRASP) — Inbound Travel Email Intake Coordination
P03 — Controller (GRASP) — Trip Creation Coordination
P04 — Controller (GRASP) — Trip Membership Management Coordination
P05 — Controller (GRASP) — Bulk Itinerary Import Coordination
P06 — Controller (GRASP) — OAuth Sign-in and Identity Linking Coordination
P07 — Controller (GRASP) — Account Traveler Enrollment Coordination
P08 — Observer (GoF) — Outbound Mail Delivery Logging
P09 — Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

The frozen baseline counts are:

Controller (GRASP) = 6
Information Expert (GRASP) = 1
Observer (GoF) = 2
Total instances = 9
Distinct pattern types = 3

The required evolution scenarios are:

1. personalized itinerary generation
2. multiple booking providers
3. different cancellation policies

For this first step, do not change the source yet.

1. Read the Part 2 handoff and inspect the original source areas relevant to the three evolution requirements.

2. For each requirement, explain what functionality already exists in the baseline and what functionality is genuinely missing.

3. Identify the exact models, controllers, jobs, views, routes, integrations and other source areas likely to be affected.

4. Trace the current BEFORE behavior for each scenario using actual source files and methods.

5. Record reliable source paths and line ranges for the important BEFORE evidence.

6. Identify which frozen pattern IDs P01–P09 could potentially be affected by each scenario, but do not change their status or classification yet.

7. Keep the three scenarios separate. Do not combine them into one redesign.

8. Do not choose or introduce design patterns simply because they seem suitable. Any new pattern must later be justified by the actual evolved implementation.

9. Recommend the safest order in which to implement the three scenarios and explain briefly why.

10. Establish how these evolution statuses should later be applied:

preserved
modified
extended
replaced
removed
newly_introduced

11. Do not generate patterns_evolution.csv yet.

12. Do not implement code yet.

13. Do not perform the cross-language part.

At the end give me:

1. Baseline confirmation.
2. BEFORE trace for each evolution scenario.
3. Source-impact map.
4. Possible P01–P09 impact for each scenario.
5. Recommended implementation order.
6. Any uncertainties that must be resolved before coding.
7. Which single evolution scenario we should work on first.

Treat P01–P09 and their original counts as frozen BEFORE evidence.

Worked for 2m 17s

DaPlanStan — Software Evolution and Refactoring

Step 1: BEFORE-baseline analysis | 11 October 2026

1. Baseline confirmation

Frozen baseline verified

I inspected the uploaded Part 2 handoff, consistency-check record and original DaPlanStan source archive.

The frozen baseline is:

Pattern	Classification	Count
Controller	GRASP	6
Information Expert	GRASP	1
Observer	GoF	2
Total instances		**9**
Distinct pattern types		**3**

All nine original IDs and their classifications remain unchanged:

P01 Information Expert; P02–P07 Controller; P08–P09 Observer.

The handoff also establishes three important limitations:

- The baseline is source-supported, but independent human approval and complete runtime testing were not established.
- The original source archive's exact Git commit provenance is unverified.
- Evolution must be recorded against separate AFTER versions rather than retroactively editing BEFORE evidence.

Analysis boundary: The uploaded ZIP archives remain unchanged. No source modifications, new pattern classifications, evolution CSV, or cross-language implementations are part of this step.

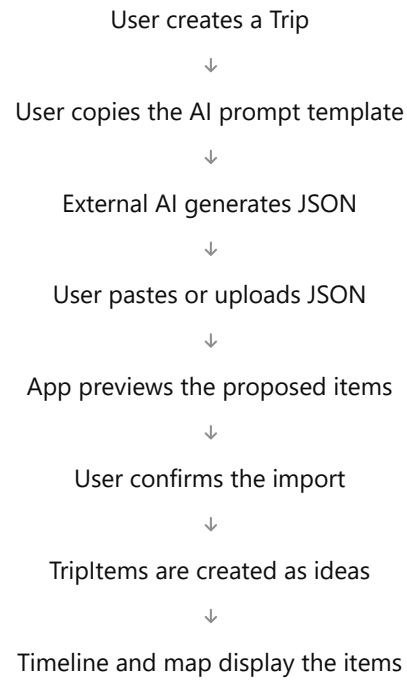
2. BEFORE trace — Scenario 1: Personalized itinerary generation

Existing functionality

DaPlanStan already supports trip creation, manual itinerary entries, importing externally generated AI ideas, and displaying results on a timeline and map.

However, **the application does not generate personalized itineraries itself**.

Current BEFORE workflow



Verified source evidence

Source location	Existing responsibility
`app/controllers/trips_controller.rb:35-48`	`create` creates the trip and initial owner membership
`app/models/trip.rb:4-19`	Trip associations, dates and validations

`app/models/trip_item.rb:9–28`	Item kinds and `idea` / `confirmed` statuses
`app/views/trip_items/import.html.erb:118–151`	Provides a prompt to use in an external AI assistant
`app/views/trip_items/import.html.erb:189–232`	Accepts pasted or uploaded JSON
`app/controllers/trip_items_controller.rb:31–58`	`import` parses, previews and confirms imported JSON
`app/controllers/trip_items_controller.rb:128–179`	`bulk_create_items` handles duplicate checking, persistence and geocoding requests
`app/jobs/geocode_trip_items_job.rb:4–21`	Geocodes missing locations and broadcasts refresh
`app/controllers/trips_controller.rb:11–29`	Loads the sorted itinerary for display
`app/views/trips/_timeline.html.erb:93–119`	Groups dated and undated items for rendering

Genuinely missing

The source has no internal personalized-itinerary generation workflow that collects preferences, creates a proposal and evaluates or schedules proposed activities.

Specifically, no implementation was found for preference-based selection, recommendation rules, automatic generation of a complete itinerary, travel-time or scheduling-conflict checks, or regeneration based on changed preferences.

A further distinction matters: `ParseInboxItemJob#perform` at `app/jobs/parse_inbox_item_job.rb:99–112` calls Anthropic, but its prompt at lines 6–18 explicitly requests conservative extraction of facts from emails. It is **not** an itinerary-generation service.

Preliminary conclusion: The existing bulk-import mechanism is a promising reuse point, but generation and personalization would be new functionality.

3. BEFORE trace — Scenario 2: Multiple booking providers

Existing functionality

DaPlanStan already accepts travel confirmation emails from different senders, extracts booking information and converts it into trip records.

It supports multiple provider *names in extracted data*, but not actual integration with multiple booking-provider APIs.

Current BEFORE workflow

Traveler forwards confirmation email

↓

TripMailbox resolves trip and verifies sender

↓

Approved email becomes an InboxItem

↓

ParseInboxItemJob extracts travel data with Anthropic

↓

User reviews parsed email and confirms

↓

InboxItem creates or links TripItems and Expenses

↓

Confirmed booking appears in itinerary

Verified source evidence

Source location	Existing responsibility
`app/mailboxes/trip_mailbox.rb:2–31`	`process` locates the trip, checks sender approval, creates `InboxItem` and queues parsing
`app/jobs/parse_inbox_item_job.rb:20–97`	Anthropic extraction schema for travel legs, accommodations, bookings and shortlist items
`app/jobs/parse_inbox_item_job.rb:63–79`	Booking data has `booking_type`, `provider`, reference, timing, price and currency
`app/jobs/parse_inbox_item_job.rb:99–139`	`perform` and `parse_with_anthropic` extract and store structured data

`app/controllers/inbox_items_controller.rb:16–35`	Confirmation, discarding, relinking and retry actions
`app/models/inbox_item.rb:67–99`	`fill_missing_links!` finds or creates associated itinerary entries
`app/models/inbox_item.rb:101–185`	Handles travel legs, accommodation and other booking categories
`app/models/inbox_item.rb:191–207`	`build_expense` links an extracted cost to a trip record
`app/controllers/trip_items_controller.rb:16–28`	Finds confirmation references and expenses shared by sibling entries
`app/views/trip_items/show.html.erb:127–157`	Displays booking reference and source email

A particularly important observation is that `InboxItem#fill_missing_links!` at lines 168–178 uses `booking["provider"]` as the itinerary item's name. This records descriptive information; it does not call the named provider.

Genuinely missing

The inspected source does not implement booking-provider adapters, provider-specific API clients, live availability or quote requests, cross-provider result comparison, booking execution, or provider booking status synchronization.

The database reinforces this distinction. `db/structure.sql:91–104` shows a `trip_items` table with confirmation reference and URL fields, but no dedicated booking-provider identity, provider booking ID or provider-specific integration state.

The existing Google/Apple provider configuration in `config/initializers/omniauth.rb:1–15` concerns **authentication providers**, not travel-booking providers. It should not be mistaken for evidence of the required functionality.

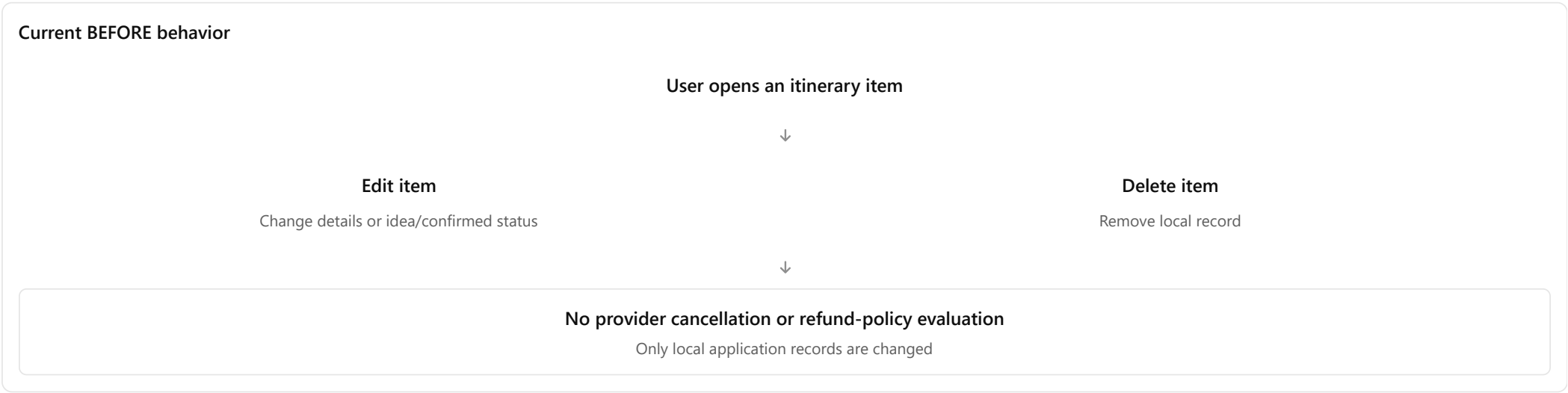
Preliminary conclusion: Email-based booking capture already exists. Direct multi-provider booking operations would be new, while any changes to confirmation-email interpretation could affect existing collaborations.

4. BEFORE trace — Scenario 3: Different cancellation policies

Existing functionality

DaPlanStan supports editing, confirming and deleting itinerary items, as well as managing recorded expenses.

It does **not** implement booking cancellation policies.



Verified source evidence

Source location	Existing behavior
`app/models/trip_item.rb:21–28`	`TripItem` has only `idea` and `confirmed` statuses
`app/controllers/trip_items_controller.rb:77–93`	Updates, destroys or toggles an item between idea and confirmed
`app/views/trip_items/_form.html.erb:119–136`	Shows the two existing item statuses
`app/views/trip_items/show.html.erb:238–249`	Provides a button to remove an item
`app/models/expense.rb:22–32`	Stores expense origin and payment confirmation state
`app/controllers/expenses_controller.rb:57–85`	Updates or deletes recorded expenses and toggles paid status
`app/controllers/trips_controller.rb:77–80`	Deletes a whole trip
`db/structure.sql:91–125`	Defines itinerary and expense records without cancellation-policy fields

The word *Cancel* in `app/views/trip_items/_form.html.erb:149–161` only means leaving the edit form. It does not cancel a reservation.

Genuinely missing

The inspected implementation contains no cancellation-policy model or rules, refund calculation, cancellation deadline, cancellation-fee evaluation, cancellation history, or booking cancellation operation.

There is also no `cancelled` value in the `TripItem` status enum.

Preliminary conclusion: Different cancellation policies require genuinely new domain behavior. Deleting a trip item must not be represented as canceling a provider reservation.

5. Consolidated source-impact map

The following identifies existing source areas to investigate or potentially modify. Any new components are described as missing capabilities rather than prescribed classes or design patterns.

All paths are relative to the `daplanstan-main/` directory in the uploaded source archive.

Scenario 1 — Personalized itinerary generation

Layer	Existing source area	Expected impact
Models	`app/models/trip.rb:4–19`	Trip context, dates and associations
Models	`app/models/trip_item.rb:9–28`	Suggested-item representation
Models, conditional	`app/models/traveler.rb:1–8` , `app/models/trip_member.rb:1–6`	Only if personalization uses individual traveler preferences
Controllers	`app/controllers/trip_items_controller.rb:31–58,128–179`	Existing preview, confirmation and import behavior
Controllers	`app/controllers/trips_controller.rb:11–29`	Itinerary loading and display
Jobs	`app/jobs/geocode_trip_items_job.rb:4–21`	Geocoding for generated locations

Views	`app/views/trip_items/import.html.erb:118–151,189–232`	Generation or import entry interface
Views	`app/views/trips/_timeline.html.erb:93–119`	Display of proposed items
Routes	`config/routes.rb:21–39`	Trip and itinerary endpoints
Integration	Anthropic currently appears in `ParseInboxItemJob`	Existing extraction capability must not be mistaken for generation

Missing source area: An in-app generation workflow and any preference representation it requires.

Scenario 2 — Multiple booking providers

Layer	Existing source area	Expected impact
Models	`app/models/inbox_item.rb:19–25,67–187`	Parsed booking data and record linking
Models	`app/models/trip_item.rb:1–28`	Booking-related itinerary data
Models	`app/models/expense.rb:1–28`	Associated monetary records
Controllers	`app/controllers/inbox_items_controller.rb:16–35`	Review and confirmation workflow
Controllers	`app/controllers/trip_items_controller.rb:16–28`	Booking reference and expense display context
Jobs	`app/jobs/parse_inbox_item_job.rb:20–139`	Existing structured booking extraction
Mailboxes	`app/mailboxes/trip_mailbox.rb:2–31`	Incoming confirmation handling
Views	`app/views/inbox_items/show.html.erb:50–61`	Human review of extracted messages
Views	`app/views/trip_items/show.html.erb:127–157,207–224`	References, source and expenses
Views	`app/views/trip_items/_form.html.erb:101–117`	Reference and URL editing
Routes	`config/routes.rb:21–46`	Existing trip, item and inbox endpoints
Database	`db/structure.sql:91–125`	Current booking-related fields

Integrations	Action Mailbox, Postmark and Anthropic	Existing email-intake path

Missing source area: Provider-specific booking operations, credentials, booking identities and external transaction handling.

An important structural detail is that `db/migrate/20260408155636_remove_legacy_travel_models.rb:12-16` removes the old `bookings` table. The current design represents bookings using `TripItem`; the evolution must be based on that actual model rather than assuming the old `Booking` model still exists.

Scenario 3 — Different cancellation policies

Layer	Existing source area	Expected impact
Models	`app/models/trip_item.rb:21-28`	Current booking-status limitations
Models	`app/models/expense.rb:22-32`	Paid amount and expense status
Models	`app/models/inbox_item.rb:12-14,67-99`	Relevant if cancellation updates arrive by email
Controllers	`app/controllers/trip_items_controller.rb:77-93`	Existing update, delete and status changes
Controllers	`app/controllers/expenses_controller.rb:57-85`	Existing expense modification
Controllers	`app/controllers/trip_members_controller.rb:26-35`	Permission context, only if membership-management responsibilities change
Views	`app/views/trip_items/_form.html.erb:119-161`	Status and edit interface
Views	`app/views/trip_items/show.html.erb:238-249`	Existing removal action
Routes	`config/routes.rb:30-46`	Existing itinerary, expense and inbox endpoints
Database	`db/structure.sql:91-125`	No existing policy or refund fields
Jobs and integrations	No cancellation-specific implementation found	New behavior would require independently verified implementation

Missing source area: Policy evaluation and cancellation-specific lifecycle handling. Provider communication would only be required if this scenario includes actual external cancellation.

6. Possible impact on frozen patterns P01–P09

The statuses below are not evolution classifications. They only indicate potential involvement.

Impact legend: Primary = existing responsibility is a likely touchpoint; Conditional = depends on the chosen implementation; — = no direct impact currently indicated.

Frozen pattern	Scenario 1 Personalized	Scenario 2 Providers	Scenario 3 Cancellation
P01 — Information Expert	Conditional	Primary	Conditional
P02 — Email Intake Controller	—	Conditional	Conditional
P03 — Trip Creation Controller	Conditional	—	—
P04 — Membership Controller	—	—	Conditional
P05 — Bulk Import Controller	Primary	Conditional	—
P06 — OAuth Controller	—	—	—
P07 — Traveler Enrollment Controller	—	—	—
P08 — Mail Logging Observer	—	Conditional	Conditional
P09 — Turbo Refresh Observer	Conditional	Conditional	Conditional

Interpretation

For **Scenario 1**, P05 is the strongest likely interaction because it already coordinates batch itinerary import. P03 becomes relevant if generation is triggered during trip creation. P01 is relevant only if email-derived data participates in generating suggestions.

For **Scenario 2**, P01 is the main existing data-linking touchpoint. P02 is relevant if the evolved solution extends incoming email handling. Direct provider API integrations could instead introduce separate functionality without materially changing either collaboration.

For **Scenario 3**, P01 and P02 are conditional touchpoints if cancellation emails are processed. P04 is conditional if its existing membership-management coordination actually changes, not merely because cancellation requires access checks. P08 or P09 may be involved if cancellation events cause emails or trip-scoped broadcasts.

There is no current reason to treat P06, the OAuth identity controller, as a booking-provider pattern. P07 should also remain outside the expected direct impact unless the evolved code demonstrates otherwise.

None of P01–P09 has been classified as preserved, modified, extended, replaced or removed yet.

7. Recommended implementation order

1. **Personalized itinerary generation**
   Lowest external-integration dependency, with reusable import and visualization behavior already present. Suitable for establishing the evolution workflow and regression-testing approach.

2. **Different cancellation policies**
   Policy evaluation can first be defined and verified against representative booking fixtures, without performing live provider cancellations. This helps establish the cancellation data requirements.

3. **Multiple booking providers**
   Highest integration uncertainty. It can build on clearly defined booking and cancellation data requirements, reducing the risk of repeatedly changing the domain model.

This order is a risk-management recommendation, not a requirement to combine the scenarios. Each should have an independently documented change, tests and source comparison.

8. Evolution status rules for later stages

The course instructions require baseline and AFTER records with evidence of actual implementations. The rules for applying statuses should be:

Status	Apply when

`baseline`	The unchanged original P01–P09 evidence
`preserved`	The original collaboration continues without material responsibility or participant changes
`modified`	An existing collaboration continues but its behavior or responsibility is materially changed
`extended`	An existing collaboration continues with additional behavior or participants
`replaced`	The former collaboration is superseded by a different collaboration
`removed`	The collaboration no longer exists in that source version
`newly_introduced`	A genuinely new, source-confirmed pattern collaboration appears

For example, if the evolved system still uses the original P05 import collaboration but adds participants, `extended` may be appropriate. If the import code remains unchanged, `preserved` may be appropriate. Those decisions must wait until the AFTER code has been inspected.

Retain the existing instance identity for a continuing collaboration. A new pattern should only be recorded when the evolved source demonstrates the relevant roles and interactions.

At every snapshot, record the actual surviving collaborations, exclude removed instances from current counts, and never alter the frozen nine-instance BEFORE count.

9. Uncertainties that must be resolved before coding

These are the main specification and verification gaps discovered during inspection.

Priority	Uncertainty	Why it matters
High	Does personalized itinerary generation need an in-app AI call, or deterministic suggestions using existing information?	Determines the scope and external dependency
High	Which preferences are required: interests, budget, dates, accessibility or individual travelers?	Determines the data and acceptance criteria
High	Does multiple booking providers mean live API booking or support for data from multiple providers?	The baseline already handles the latter partially

High	Are cancellation policies informational estimates or executable cancellations and refunds?	Changes security, integration and financial requirements
High	Which cancellation rules should be supported, and what happens after the deadline?	Needed for deterministic acceptance tests
Medium	What is the exact baseline source commit?	The uploaded archive has no verifiable Git history
Medium	How will baseline behavior be tested?	The source package does not contain a discovered application test suite, and the handoff reports no completed runtime verification

Two implementation risks found in the BEFORE source

1. Existing import callback handling

`app/controllers/trip_items_controller.rb:134–174` globally disables and later restores an Active Record save callback during bulk import. This deserves regression testing, particularly for failure handling and concurrent requests. No refactoring has been performed.

2. Booking matching by name

`app/models/inbox_item.rb:79–99` matches existing itinerary records using lowercase names, while lines 168–178 use the provider name for certain bookings. Separate reservations with the same provider name could therefore be treated as matching records. This is particularly relevant before adding multiple booking integrations.

These are source-derived risks, not evidence that a failure occurred in a runtime test.

10. Final Step 1 outcome

Requested outcome	Result
Frozen BEFORE baseline confirmed	✓
Three scenario BEFORE traces	✓
Existing versus missing functionality	✓
Models, controllers, jobs, views, routes and integrations mapped	✓

Source paths and line ranges recorded	✓
Potential P01–P09 impacts identified	✓
Implementation order recommended	✓
Evolution status rules established	✓
Original source modified	No
Frozen baseline modified	No
`patterns_evolution.csv` generated	No
Cross-language work performed	No

11. Which single scenario should we work on first?

Scenario 1 — Personalized itinerary generation.

It is the most contained starting point because DaPlanStan already supports JSON-based idea import, batch record creation, geocoding and itinerary display.

The next step should be to finalize Scenario 1's acceptance criteria, intended source changes, regression checks and implementation boundaries before editing code. Do not select a new design pattern in advance.

Step 1 is complete as a static BEFORE analysis. Runtime validation, exact Git provenance, detailed feature specifications and AFTER pattern verification remain outstanding. The original nine-instance baseline remains frozen.

We have completed the BEFORE analysis for the three software-evolution scenarios.

For now, work only on Scenario 1 — Personalized Itinerary Generation.

The frozen P01–P09 baseline must remain unchanged as BEFORE evidence.

From the previous analysis, DaPlanStan already supports:

1. Trip creation.

2. Manual itinerary items.
3. External AI-generated JSON import.
4. Preview and confirmation of imported itinerary ideas.
5. Bulk TripItem creation.
6. Geocoding.
7. Timeline and map display.

But it does not currently generate a personalized itinerary inside the application.

Before changing any source, design the implementation for Scenario 1.

I want the implementation to be realistic for this existing Rails application and small enough to complete and verify for the coursework.

1. Reinspect the relevant source before proposing changes. Do not rely only on the previous summary.

2. Define a precise feature scope for “personalized itinerary generation”.

The feature should allow a user to provide a small set of trip preferences and generate proposed itinerary items for an existing trip.

Keep the first version focused. Consider preferences such as:

- interests or activity categories
- budget level
- preferred pace
- optional free-text preference

Do not add unnecessary preference fields unless the existing source clearly requires them.

3. Decide whether the safest implementation should:

a. use an in-app Anthropic request,
b. use deterministic application logic,
or
c. use another approach already supported by the repository.

Compare the alternatives briefly and select one implementation for this case study.

The decision should consider:

- the existing Anthropic integration
- testability
- external credentials
- implementation size
- how easily the result can be demonstrated to the faculty
- reuse of the existing TripItem import/creation flow

4. Trace the proposed AFTER workflow from the user action through generation, preview, confirmation, TripItem creation, geocoding and display.

5. Identify exactly which existing files should be modified.

6. Identify any new files that genuinely need to be introduced.

For every proposed new class, job, module, controller action or model responsibility, explain why it is needed.

Do not create abstractions only to make a design pattern appear.

7. Decide where the personalization preferences should live.

Determine whether they should:

- be transient form input,
- be persisted on Trip,
- use a new model,
- or use another existing representation.

Prefer the smallest defensible change.

8. Explain how generated suggestions should reuse the existing TripItem representation.

Clarify:

- whether generated items begin as `idea`
- how dates/times are represented
- how duplicate suggestions should be handled
- when geocoding occurs
- how the user confirms or rejects generated suggestions

9. Examine whether the existing `TripItemsController#import` and `bulk_create_items` behavior should be reused directly, extracted, or left separate.

Do not refactor it merely for neatness. Only recommend extraction if the personalized-generation workflow actually needs the same application responsibility.

10. Define clear acceptance criteria for Scenario 1.

Include normal cases and important failure cases.

At minimum consider:

- generation for a valid trip

- missing preferences
- empty generation result
- malformed generated result
- duplicate itinerary suggestions
- generated places without coordinates
- user cancellation before confirmation
- authorization for the trip

11. Define the regression checks needed to prove that existing manual item creation and JSON import still work.

Pay particular attention to the previously identified callback handling in `bulk_create_items`.

12. Identify which frozen patterns P01–P09 could actually be affected by this proposed implementation.

Do not assign an evolution status yet.

13. Only after designing the actual collaboration, identify any design-pattern candidate that might emerge from the proposed implementation.

Treat it only as a candidate.

For each candidate explain:

- what actual problem it would solve
- what participants would exist in our proposed code
- why the collaboration might fit
- why it might still be ordinary Rails/application code instead

Do not confirm a new pattern before the code exists.

14. Do not modify the repository yet.

15. Do not generate `patterns_evolution.csv` yet.
16. Do not work on booking providers or cancellation policies yet.

At the end give me:

1. Final Scenario 1 scope.
2. Chosen generation approach and why.
3. Proposed AFTER workflow.
4. Exact files to modify.
5. Exact new files to introduce, if any.
6. Preference/data design.
7. Reuse/refactoring decision for the existing import flow.
8. Acceptance criteria.
9. Regression-test checklist.
10. Possible impact on P01–P09.
11. Any tentative pattern candidates.
12. A file-by-file implementation sequence that we can follow in the next step.

Do not write the code yet.

Worked for 4m 9s

DaPlanStan — Scenario 1 Implementation Design

Software Evolution and Refactoring | Design-only stage | 11 October 2026

No source changes

I re-inspected the original `daplanstan-main(1).zip` and the Part 2 handoff. The proposed design below is based on the actual Rails source, not just the previous BEFORE summary.

1. Reinspection findings and final feature scope

The relevant baseline source confirms:

Existing capability	Verified BEFORE evidence
Trip dates and description	`app/models/trip.rb:4-19` ; `db/structure.sql:61-70`
Manual item creation	`app/controllers/trip_items_controller.rb:60-73`
External JSON preview	`app/controllers/trip_items_controller.rb:31-58`
Batch creation and deduplication	`app/controllers/trip_items_controller.rb:128-179`
Imported items begin as ideas	`app/controllers/trip_items_controller.rb:148-162`
Existing Anthropic integration	`app/jobs/parse_inbox_item_job.rb:127-139`
Existing geocoding and map refresh	`app/jobs/geocode_trip_items_job.rb:4-21`
Timeline and map rendering	`app/controllers/trips_controller.rb:11-29` and `app/views/trips/_timeline.html.erb:93-119`

There are two important implementation constraints: trips have no dedicated destination or time-zone field, and `TripItemsController` currently resolves trips through `current_user.trips.find(params[:trip_id])`, which verifies membership but not the member's editing role.

Final Scenario 1 scope

Build a **Generate itinerary ideas** feature for an existing trip.

Proposed user interface

Illustrative form design, not implemented

Interests (choose at least one)

◯ Culture & history ◯ Food & cafes

◯ Nature & outdoors ◯ Shopping

Budget level

◯ Budget ◯ Moderate ◯ Premium

Preferred pace

Relaxed Balanced Active

Additional preference (optional)

For example: Prefer indoor activities and local food

Copy example preference input

This illustrative form shows the four proposed inputs. It does not call the application or generate a real itinerary.

The first version will:

- Take interests, budget, pace and optional notes.
- Reuse the existing trip's title, description and dates.
- Generate a small batch of **3–6 proposed activities or places**.
- Show a preview before saving anything.
- Allow the user to accept or reject the whole batch.
- Save accepted suggestions as `TripItem` records with status `idea`.
- Reuse existing geocoding, timeline and map functionality.

Generation will suggest activities rather than make reservations. It will not guarantee live opening hours, prices, geographic accuracy or conflict-free scheduling. Suggestions must be reviewed by the traveler.

For coursework, I recommend limiting generation to at most six suggestions per request. This bounds API output, review complexity and geocoding work.

2. Generation approach decision

Option	Advantages	Disadvantages	Decision
A. In-app Anthropic	Existing SDK and credentials configuration; realistic destination-specific suggestions; minimal new integration work	Requires a valid API key for live generation; variable output; network cost	**Selected**

B. Deterministic rules	No API credentials; repeatable; straightforward unit tests	No destination/place catalogue exists in the source; would largely generate generic activity templates	Not selected
C. Existing external-AI JSON import	Already implemented and easy to demonstrate	Does not satisfy in-app generation	Retain unchanged

Chosen approach: In-app Anthropic with controlled structured output

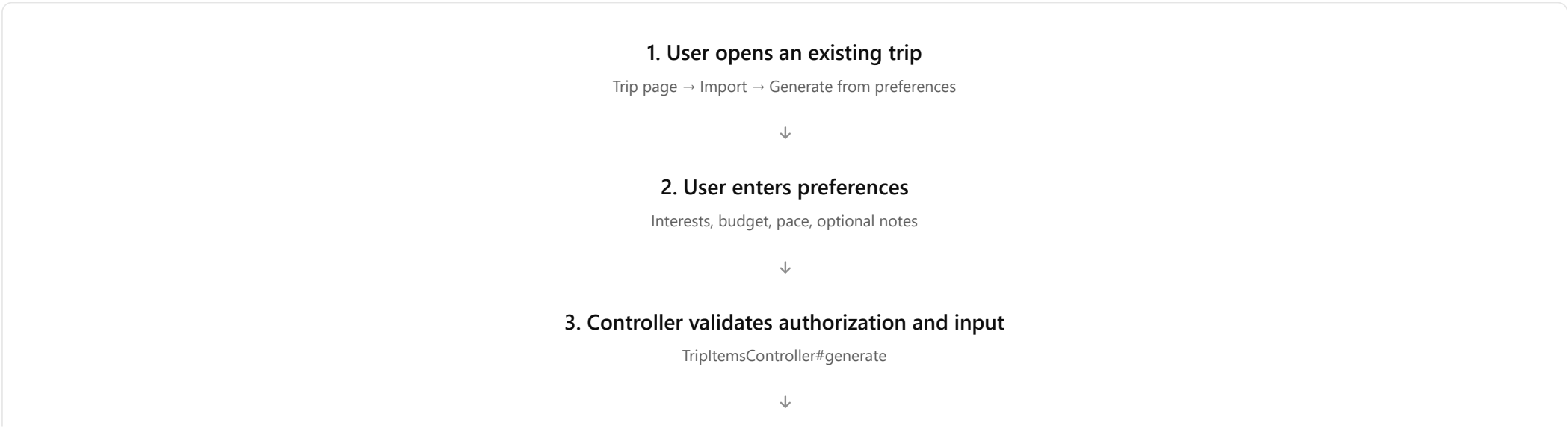
The source already includes the `anthropic` gem in `Gemfile:50-51`, an `ANTHROPIC_API_KEY` entry in `.env.example`, and a working application-level example of an Anthropic client request in `ParseInboxItemJob`.

Introduce one narrowly scoped generation service. It will build the prompt from the trip and preferences, request structured suggestions, validate the response and return data in the existing import format.

This avoids adding a new API vendor, database-backed recommendation catalogue or unnecessary pattern-oriented abstraction.

For verification, tests will substitute fixed responses for the Anthropic request. Live demonstration will require credentials; fixture-driven tests can demonstrate the generation workflow without paid API access. A missing key must produce a controlled error, not an invented itinerary.

3. Proposed AFTER workflow





Route and action design

Add `GET` and `POST` routes for `generate` to the existing nested `trip_items` collection in `config/routes.rb:30–34`.

Proposed route helper: `generate_trip_trip_items_path(@trip)`.

Use a single new `TripItemsController#generate` action with three behaviors:

Request	Purpose
`GET /trips/:trip_id/trip_items/generate`	Show preferences form
`POST` with preferences	Validate inputs, call generator, show preview
`POST` with explicit confirmation	Revalidate submitted suggestions, call existing `bulk_create_items`, redirect to itinerary

The implementation should use a synchronous Anthropic request for this small first version. A timeout or failed request should return an understandable error to the form. No background generation job or persistent generation status is necessary.

The import preview can be reused by rendering the existing `import` view with a small flag indicating that the suggestions came from the generator. Its confirmation form must then submit back to the guarded `generate` action.

Important trust boundary

Even after receiving valid Anthropic output, the browser's hidden confirmation JSON must be considered untrusted.

The confirmation action must validate the array again, restrict the allowed fields and enforce the maximum item count. It must never assume that hidden form values are safe merely because the service originally generated them.

4. Preference and TripItem data design

Where preferences live

Decision: Transient form input.

Storage approach	Decision
Transient request parameters	Selected
Additional columns on `Trip`	Not needed
New preference model/table	Not needed

New traveler-preference model	Not needed

For V1, the existing Trip record provides the persistent context: title, description, start date and end date. The four personalization inputs are submitted only when generating suggestions.

No migration is required.

The free-text preference should have a modest length limit (for example 300 characters), and the service should accept only enumerated budget, pace and interest values. Generated results do not store a new preference profile.

This makes the change easy to reverse and avoids committing to a persistent preference model before the use case requires one.

Output mapping

Use the existing `TripItem` fields and the same import-data keys already accepted by `bulk_create_items`.

Generated field	Existing destination	Rule
`name`	`TripItem.name`	Required, nonblank
`kind`	`TripItem.kind`	One of `eat`, `do`, `shop`, `other` for V1
`address`	`TripItem.address`	Place/city information when available
`notes`	`TripItem.notes`	Brief activity and preference relevance
`starts_at`	`TripItem.starts_at`	Optional proposed trip-local clock time
`ends_at`	`TripItem.ends_at`	Optional end time later than start
`status`	`TripItem.status`	Always assigned `idea` by existing persistence code
`added_by`	`TripItem.added_by`	Current authenticated user

Do not generate prices, expenses, booking references or confirmation statuses.

I recommend not accepting AI-invented coordinates for this feature. Even if the model provides latitude and longitude, the new service should discard them and rely on the application's geocoding workflow.

Dates and times

The generated item format should match the existing importer, for example `2026-11-16T10:00` for a proposed morning activity.

The service must validate dates against `Trip.start_date` and `Trip.end_date`. If a time is absent, store `starts_at = nil` and display that item in the existing *Date TBD* group.

The repository currently has no destination time-zone setting, and `parse_import_time` uses `Time.zone.parse` (`trip_items_controller.rb:176–179`). V1 should therefore label generated times as tentative and use the application's existing time-zone interpretation rather than claiming accurate destination-local scheduling.

Duplicate handling

Reuse the existing case-insensitive, name-based deduplication within the trip (`trip_items_controller.rb:133–165`).

- An existing item with the same name is skipped.
- Repeated names in the generated batch are skipped.
- Accepted records remain `idea`, even if the user clicked **Add to itinerary**.
- A duplicate does not create another record.

The existing method can enrich a duplicate's missing coordinates or URL. Because the selected generation approach supplies neither AI coordinates nor URLs, this new flow should not overwrite existing confirmed items through that enrichment behavior.

The current comparison does not fully normalize whitespace or distinguish same-named places at different addresses. That is a documented limitation, not a reason to redesign deduplication in this scenario.

Geocoding

Geocoding occurs **after acceptance and record creation**, not while generating or reviewing.

On successful creation, newly created items without complete coordinates are passed to `GeocodeTripItemsJob`. That job attempts location resolution and broadcasts an existing trip refresh. An unsuccessful geocode must not delete an otherwise valid suggestion; the item can remain visible in the timeline without a map pin.

Confirmation and rejection

For V1, approval is **whole-batch** rather than per-item.

- **Accept:** persist eligible suggestions as ideas.
- **Reject:** return to the preferences form without persisting anything.
- **After acceptance:** users can use the existing edit, confirm and remove controls on individual items.

Per-suggestion selection can be considered later, but it is not necessary for the first coursework implementation.

5. Exact source files to modify

Paths below are relative to the original Rails repository root. They are proposed AFTER changes, not edits already made.

Existing file	Planned change	Why
`config/routes.rb`	Add `GET` and `POST` collection routes for `generate`	Expose the new feature for a selected trip
`app/controllers/trip_items_controller.rb`	Add `generate`; validate preferences, authorization and confirmation; reuse `bulk_create_items`; make callback handling safe	Coordinate generation without duplicating persistence
`app/views/trip_items/import.html.erb`	Add a link to generation; reuse preview with conditional confirmation and rejection destinations; show proposed dates	Avoid duplicating the existing preview
`app/models/trip_item.rb`	Add a transient per-record geocoding-defer flag and guard	Eliminate the need for globally disabling callbacks during bulk creation
`config/application.rb`	Enable the currently commented Rails test-unit railtie	Allow automated Rails tests
`README.md`	Document the new feature, credentials, limitations and test commands	Reproducible faculty verification

The callback safety change is deliberate

`bulk_create_items` currently uses:

- `TripItem.skip_callback` at line 134.
- `TripItem.set_callback` in its `ensure` block at lines 172–174.

These change the model's callback configuration globally for the duration of the import. Other concurrent requests could therefore experience different geocoding behavior.

The targeted change is to stop disabling the global callback and instead mark only the specific batch-created `TripItem` instances for deferred geocoding. Existing `GeocodeTripItemsJob` will later load those records and perform geocoding normally.

That requires a small modification to `TripItem#geocode_from_address` at lines 34–37 and the existing bulk creation method. It is an actual reliability fix relevant to reuse by two workflows, not a cosmetic extraction.

No change is proposed to `Geocodable`, `GeocodeTripItemsJob`, `TripsController`, `Trip` or the database schema.

6. Exact new files

New file	Responsibility	Necessity
`app/services/itinerary_suggestion_generator.rb`	Build prompt, invoke Anthropic, normalize and validate response, return proposed items or a controlled error	Needed to keep external API logic out of the controller and independently test it
`app/views/trip_items/generate.html.erb`	Render the four preference inputs and validation/API errors	Needed for the new user interface
`test/test_helper.rb`	Minimal Rails/Minitest setup and common test configuration	Needed because the original archive has no discovered application test suite
`test/services/itinerary_suggestion_generator_test.rb`	Test output validation and API failure handling with stubbed responses	Needed for repeatable generator verification
`test/controllers/trip_items_generation_test.rb`	Test request-to-preview-to-confirm workflow and authorization	Needed for feature acceptance evidence

`test/controllers/trip_items_import_regression_test.rb`	Protect existing import, manual item creation and callback behavior	Needed to verify existing behavior remains intact

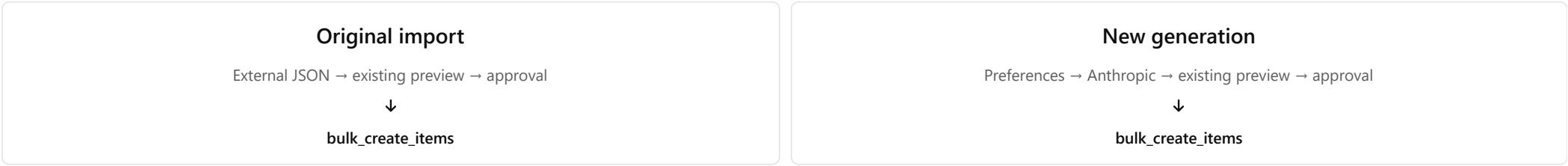
No new model, migration, generation job, JavaScript controller or provider-adapter hierarchy is justified.

The Rails test infrastructure should be added only once, using the standard existing Rails dependencies rather than introducing a second testing framework.

7. Import-flow reuse and refactoring decision

Decision: Reuse `bulk_create_items` directly. Do not extract it into a separate service for V1.

The generation action and original import action already have the same application responsibility at the final stage: persist a user-approved batch of itinerary ideas.



The existing method can be shared as a private controller method without extraction because both actions are in `TripItemsController`.

The new generator service is different: it performs the separate responsibility of creating and validating suggestions. That belongs outside the persistence method.

The targeted changes to `bulk_create_items` should be limited to making deferred geocoding instance-scoped and ensuring its existing counts and duplicate behavior remain consistent.

There is also an existing confirmation-path gap at `trip_items_controller.rb:32–33`: the original import confirmation parses JSON without the same array-shape validation as the preview branch. The new generation confirmation must validate independently. If the original import confirmation is also hardened, that change must receive separate regression evidence and must not silently change normal import behavior.

8. Scenario 1 acceptance criteria

The following criteria should be satisfied before declaring Scenario 1 implemented.

ID	Test situation	Expected result
AC01	Trip owner opens Generate from preferences	Form appears for the selected trip
AC02	Valid trip and preferences submitted	Anthropic returns a validated preview, targeting 3–6 suggestions
AC03	No interest selected	Validation message; no API request and no records created
AC04	Invalid budget, pace or oversized free text	Input rejected safely
AC05	Anthropic returns an empty list	Friendly message; no items created
AC06	Anthropic returns malformed output, invalid item types or invalid dates	Controlled error; no persistence
AC07	Generation request fails or API key is unavailable	Error displayed; no records created
AC08	Generated suggestion duplicates an existing itinerary name	Existing deduplication skips the duplicate
AC09	Generated batch contains repeated names	Only one corresponding item is created
AC10	Generated place lacks coordinates	Item can be saved; geocoding is queued
AC11	Geocoding fails	Item remains visible without a map pin
AC12	User rejects generated preview	No new TripItem or Expense records
AC13	User accepts preview	Eligible suggestions saved as `idea` with correct `trip_id` and `added_by`
AC14	Accepted items have proposed dates	Dated items appear in the existing day-grouped timeline
AC15	Confirmation submitted twice	No second copy is created for an exact case-insensitive name match
AC16	Unauthenticated or unrelated user requests the feature	Access denied according to existing authentication and trip-scoping rules

AC17	Authenticated trip member without the required role requests generation	Generation is denied
AC18	Generated confirmation JSON is tampered with	Unexpected fields and invalid values cannot bypass validation

For V1, I recommend **trip-owner-only generation**, matching the owner-only import action area in `app/views/trips/_trip_panel.html.erb:178-249`. This must be checked in both generation and confirmation requests, not only by hiding the link.

The existing `import` action has broader membership-based access. That existing behavior should remain unchanged during this scenario rather than silently introducing a global permission redesign.

For valid output, the generator should aim for 3–6 suggestions and accept a nonempty, validated result of at most six. An empty result must never be presented as successful generation.

9. Regression-test checklist

This is particularly important because Scenario 1 shares persistence code with an existing frozen collaboration.

Planned regression verification 0/23

Interactive planning checklist — not executed test results

Existing manual creation

- ○ Create an idea manually with a valid name and kind
- ○ Create a manual item with an address and verify normal geocoding remains active
- ○ Edit an existing manual item
- ○ Confirm and unconfirm an existing item
- ○ Reject an invalid manual item without creating a record

Existing JSON import

- ○ Preview a valid pasted JSON array
- ○ Preview a valid uploaded JSON file

- [ ] Confirm valid JSON and verify created items
- [ ] Skip duplicate names and report correct counts
- [ ] Reject malformed/non-array confirmation payloads without a server error
- [ ] Preserve missing-name skipping and existing URL/coordinate enrichment behavior

Callback and background job

- [ ] Verify batch imports never globally disable TripItem save callbacks
- [ ] Verify new batch items queue the intended GeocodeTripItemsJob
- [ ] Verify an unrelated manual item still geocodes after a batch import
- [ ] Verify geocoding behavior remains correct when an import item fails validation
- [ ] Verify callback behavior after successful and unsuccessful imports
- [ ] Verify a failed geocode does not delete the imported item
- [ ] Verify job refreshes the relevant trip

Isolation and consistency

- [ ] Verify generation creates no record before confirmation
- [ ] Verify TripItems created for one trip never enter another trip
- [ ] Verify generation does not create InboxItems or Expenses
- [ ] Verify dates, kinds and idea status in the existing timeline
- [ ] Verify existing email parsing and TripItem linking still operate

Reset checklist

The source currently has the Rails test-unit railtie commented out at `config/application.rb:15`, and the uploaded archive does not contain a discovered application test suite.

Once the test infrastructure is enabled, the planned command is `bin/rails test`. Anthropic responses, Geocoder lookups and queued jobs should be stubbed or inspected with Rails' test adapter. Exact results must be recorded after execution, not assumed now.

For coursework, preserve before/after verification logs and capture the source diff for the callback change. This is what will make the refactoring claim defensible.

10. Possible impact on frozen P01–P09

Pattern ID	Expected involvement	Reason
P01 — Parsed Email Information Expert	No direct impact	New suggestions do not originate from `InboxItem`
P02 — Email Intake Controller	No direct impact	Mail intake is not changed
P03 — Trip Creation Controller	No direct impact	Generation happens after trip creation
P04 — Membership Controller	No direct impact	Generation checks a member's role but does not change membership management
P05 — Bulk Import Controller	**Direct impact**	New generation action reuses `bulk_create_items`, with a targeted callback-safety change
P06 — OAuth Controller	No direct impact	No authentication-provider change
P07 — Traveler Enrollment Controller	No direct impact	No enrollment change
P08 — Mail Logging Observer	No direct impact	Generation does not send email
P09 — Turbo Refresh Observer	**Existing behavior exercised**	The reused geocoding job broadcasts a trip refresh

P05 is the principal frozen collaboration to examine in the AFTER snapshot. P09 should be regression-tested because the evolved workflow invokes its existing notification mechanism, but the design does not require changing that mechanism.

Reading a `Trip` or checking `TripMember.role` does not automatically mean that P03 or P04 has changed. Pattern status must be determined by changes to the actual recorded collaboration, not by incidental reuse of a participating model.

No evolution status is assigned at this stage.

11. Tentative design-pattern candidates

Only one candidate is worth carrying forward for later verification.

Candidate: Controller (GRASP) — Personalized Itinerary Generation Coordination

Aspect	Proposed interpretation
Problem	Coordinate a user request for generated itinerary suggestions without mixing AI integration, review and persistence
Potential coordinator	`TripItemsController#generate`
Potential collaborators	`Trip`, `ItinerarySuggestionGenerator`, existing import preview, existing `bulk_create_items`, `TripItem`, `GeocodeTripItemsJob`
Why it may fit	A controller receives a system event and delegates specialized work
Why it may not be a separate pattern instance	It could simply be another Rails action belonging to the same wider itinerary-import collaboration already represented by P05

`ItinerarySuggestionGenerator` by itself is not evidence of Strategy, Factory, Adapter or another GoF pattern. It is initially just a service object encapsulating one external API operation.

Therefore:

Candidate status: tentative and unconfirmed.

The next stage must first implement and inspect the actual participants before determining whether P05 was extended or whether a genuinely independent pattern collaboration exists.

12. File-by-file implementation sequence for the next step

This is the recommended coding order. It keeps test setup, source changes and verification manageable.

1 **Preserve and verify the original snapshot**

Original ZIP and Part 2 baseline

Keep the frozen files unchanged. Record the original archive checksum and establish a separate AFTER working copy.

2 **Prepare test infrastructure**

config/application.rb; new test/test_helper.rb

Enable Rails test support and create minimal test helpers for an authorized user, trip and itinerary items.

3 **Write the expected behavior tests**

New generation service, controller and regression test files

Define the acceptance cases with stubbed Anthropic responses before making production changes.

4 **Implement the suggestion generator**

New app/services/itinerary_suggestion_generator.rb

Read trip context and preferences, call Anthropic, validate the tool response, and normalize up to six suggestion hashes.

5 **Add the new routes**

config/routes.rb

Add GET and POST generate routes under the existing trip_items collection.

6 **Implement the controller workflow**

app/controllers/trip_items_controller.rb

Add the owner authorization check, form handling, generation, preview setup and guarded confirmation calling bulk_create_items.

7 **Make batch geocoding callback handling safe**

app/models/trip_item.rb; app/controllers/trip_items_controller.rb

Replace global callback toggling with per-record deferred geocoding, without changing manual creation or the background geocoding job.

8 **Implement the user interface**

New app/views/trip_items/generate.html.erb; existing app/views/trip_items/import.html.erb

Add preference inputs, an entry link, a generated-preview mode, accept/reject controls and proposed scheduling details.

9 **Run acceptance and regression tests**

New test files, execution logs and source diff

Run tests with stubbed services; check manual creation, existing JSON import, callback behavior, authorization, duplicates and map refresh.

10 **Document and verify the change**

README.md; Scenario 1 coursework evidence

Record implementation scope, exact changed paths, environment, test commands, actual results, limitations and BEFORE/AFTER comparison evidence.

When actual tests are run, retain passed/failed counts, command output and screenshots or logs as coursework evidence. Once this scenario is implemented and verified, its pattern effects can be assessed against the still-frozen BEFORE baseline.

Final design decision

Decision	Scenario 1 plan
Feature	Personalized itinerary idea generation for an existing trip
Generation method	In-app Anthropic request
Inputs	Interests, budget, pace, optional notes
Preferences storage	Transient form input
New production class	`ItinerarySuggestionGenerator` only
New background job	None
New database migration	None
Preview	Reuse existing import preview
Persistence	Reuse `bulk_create_items`
Initial item status	`idea`
Geocoding	Existing `GeocodeTripItemsJob`
Authorization	Trip owner for V1
Refactoring	Targeted global-callback safety fix
Primary pattern impact	P05; P09 behavior also exercised
New pattern status	Candidate only, not confirmed

Scenario 1 design is complete as a source-grounded implementation proposal. No repository files have been changed, no tests have been claimed as executed, and `patterns_evolution.csv` has not been generated. The frozen P01–P09 BEFORE inventory remains untouched.

The next step is to implement Scenario 1 in this sequence, verify the resulting behavior, and only then perform the AFTER pattern analysis.

We have finished the design for Scenario 1 — Personalized Itinerary Generation.

Now implement Scenario 1 in a separate AFTER working copy of the DaPlanStan source.

The original uploaded baseline source and frozen P01–P09 evidence must remain unchanged.

Use the implementation design from the previous step, but recheck the actual source before editing each file. If the real source differs from an assumption in the design, follow the source and document the difference rather than forcing the proposed design.

The agreed Scenario 1 scope is:

1. Generate itinerary ideas for an existing trip.
2. Inputs:
   - one or more interests
   - budget level
   - preferred pace
   - optional free-text preference
3. Use an in-app Anthropic request.
4. Generate no more than six suggestions.
5. Preview suggestions before persistence.
6. Save accepted suggestions as existing TripItem records with status `idea`.
7. Reuse existing geocoding, timeline and map behavior.
8. Do not create bookings, expenses or confirmed reservations.
9. Preferences remain transient.
10. No database migration is planned.
11. Generation is owner-only for this first version.

12. The existing external JSON import must continue working.

Implement this carefully.

1. First create or work from a separate evolved source copy.

Do not edit the frozen original ZIP.

Identify the AFTER source version clearly, for example:

`scenario1_personalized_itinerary`

Do not create `patterns_evolution.csv` yet.

2. Reinspect these files before changing them:

- `config/routes.rb`
- `app/controllers/trip_items_controller.rb`
- `app/models/trip_item.rb`
- `app/models/concerns/geocodable.rb`
- `app/jobs/geocode_trip_items_job.rb`
- `app/views/trip_items/import.html.erb`
- `app/views/trips/_trip_panel.html.erb`
- `app/jobs/parse_inbox_item_job.rb`
- `config/application.rb`

Record the relevant original line ranges before editing.

3. Implement `ItinerarySuggestionGenerator`.

Create:

`app/services/itinerary_suggestion_generator.rb`

Its responsibility should be limited to:

- receiving a Trip and validated preference values
- building the itinerary-generation prompt
- invoking Anthropic using the repository's existing client style
- requesting structured output
- validating and normalizing the returned suggestions
- returning no more than six safe suggestion hashes
- returning or raising a controlled application-level error for API failure or invalid output

Do not put persistence, controller authorization or geocoding inside this service.

Use only existing TripItem-compatible fields needed for Scenario 1:

- `name`
- `kind`
- `address`
- `notes`
- `starts_at`
- `ends_at`

Allowed generated kinds for V1 should be limited to the existing values needed for activities:

`eat`
`do`
`shop`

`other`

Do not accept AI-generated status, confirmation references, prices, expenses, URLs or coordinates.

4. Validate preferences before calling Anthropic.

Use explicit allowed values.

Interests should use a small fixed whitelist based on the approved design.

Budget should allow only the selected V1 values.

Pace should allow only the selected V1 values.

Limit the optional note to a reasonable maximum length.

At least one interest is required.

Invalid preferences must not cause an Anthropic request.

5. Add the generation routes.

Add GET and POST support under the existing nested trip-item routes.

Do not disturb existing import, create, edit, update or delete routes.

Show the exact route declarations added.

6. Add `TripItemsController#generate`.

The action must handle three states:

GET:
show the preferences form.

POST generation:
validate authorization and preferences;
call `ItinerarySuggestionGenerator`;
prepare the generated preview;
save nothing.

POST confirmation:
treat submitted suggestion JSON as untrusted;
parse and validate it again;
allow only approved fields and values;
enforce the six-item maximum;
then reuse the existing approved batch-persistence behavior.

Reject malformed or tampered confirmation input without persistence.

7. Enforce authorization in the controller.

Do not rely only on whether the Generate link is visible.

For this V1 implementation, generation and generated-result confirmation should require the authenticated user to have the trip owner role.

Do not change the existing authorization policy of the ordinary JSON import unless required for correctness.

8. Reuse existing batch persistence rather than duplicating it.

Use the existing `bulk_create_items` responsibility where possible.

Preserve:

- `idea` status
- current user as `added_by`
- duplicate skipping
- existing date parsing behavior
- existing geocoding queueing
- existing result-count behavior

Do not extract a new persistence service merely to create a pattern.

9. Carefully address the existing global callback handling.

Before modifying it, verify that the current source really uses `TripItem.skip_callback` and `TripItem.set_callback` around batch creation.

If confirmed, replace that global callback manipulation with the smallest safe instance-scoped mechanism necessary for batch-created records to defer automatic geocoding until `GeocodeTripItemsJob`.

The change must satisfy both conditions:

- a batch import/generation does not trigger synchronous geocoding for each created item
- unrelated TripItem saves do not have their callbacks globally disabled

Do not change the external behavior of ordinary manual TripItem saves.

Explain exactly how the new mechanism works.

10. Add the generation UI.

Create:

`app/views/trip_items/generate.html.erb`

Include:

- interests
- budget
- pace
- optional preference note
- validation/API errors
- generate button
- cancel/back option

Add an appropriate Generate itinerary ideas entry point in the existing trip UI.

Keep the styling consistent with the existing Rails views rather than redesigning the interface.

11. Reuse the existing preview where practical.

If the existing `import.html.erb` can safely display generated suggestions using a small explicit mode flag, reuse it.

The generated preview must clearly provide:

- Accept
- Reject

Reject must persist nothing.

Accept must return to the guarded generation confirmation path rather than bypassing its validation.

If reuse of the existing view would require fragile or extensive conditionals, create a small shared partial instead and explain why.

12. Handle failures safely.

Implement controlled behavior for:

- missing API key
- Anthropic request failure
- malformed response
- empty response
- more than six returned suggestions
- invalid kind
- invalid/missing name
- invalid dates
- dates outside the trip range
- malformed confirmation JSON
- tampered hidden fields

No failure should silently create partial itinerary records.

13. Add tests only after checking how this Rails application is configured.

Do not automatically modify `config/application.rb` merely because the previous design proposed it.

First determine whether the existing Rails configuration can run the standard test framework.

Only make the smallest configuration change actually required.

Add focused tests for:

- generator validation with stubbed Anthropic responses
- GET generation authorization
- valid generation preview
- no persistence before confirmation
- successful confirmation
- duplicate suggestions
- malformed/tampered confirmation
- API failure
- owner versus unauthorized member
- existing JSON import regression
- manual TripItem creation regression
- deferred geocoding behavior

External Anthropic and geocoding requests must be stubbed during automated tests.

14. Run whatever verification is actually possible in the available environment.

Attempt the appropriate Rails test command.

Record:

- exact command
- number of tests run
- passes
- failures
- errors
- skipped tests

If the application cannot run because of missing Ruby, dependencies, credentials, database setup or another environment problem, show the actual error and mark runtime verification as blocked.

Do not report unexecuted tests as passed.

15. After implementation, produce a precise change record.

For every changed or new file show:

- file path
- purpose
- important methods added or changed
- reliable AFTER line ranges
- whether it belongs to feature implementation, refactoring, UI or testing

Also show a concise BEFORE → AFTER workflow.

16. Do not classify the final pattern evolution yet.

At the end you may identify whether the implemented source now appears to:

- preserve P05
- modify P05
- extend P05
- or introduce a separate Controller candidate

But mark this only as a question for the next verification step.

Do not finalize an evolution status yet.

Do not add any new pattern to the frozen inventory yet.

17. Do not work on multiple booking providers or cancellation policies yet.

At the end give me:

1. Implementation summary.
2. Complete list of changed and new files.
3. Important code changes with AFTER line ranges.
4. Final user workflow.
5. Authorization behavior.
6. Anthropic integration behavior.
7. Validation and failure handling.
8. Explanation of the callback-safety change.
9. Tests added.
10. Actual test/runtime results.
11. Any implementation differences from the previous design and why.
12. Remaining limitations.
13. Source files that must be inspected in the next AFTER pattern-verification step.
14. A downloadable ZIP or equivalent complete source snapshot of the Scenario 1 AFTER implementation if the environment permits file creation.

Do not proceed to Scenario 2 yet.

Worked for 6m 45s

Scenario 1 — AFTER Implementation

Separate source copy created

The original `daplanstan-main(1).zip` and frozen P01–P09 pattern evidence remain unchanged. The implementation is being prepared under the source-version label:

`scenario1_personalized_itinerary`

Implemented feature

The evolved source now includes an in-app Anthropic itinerary generator, owner-only routes and controller actions, a preference form, a generated-ideas preview, signed confirmation, and reuse of the existing `TripItem` batch-persistence and geocoding workflow.

The implementation also replaces the original global callback manipulation with an instance-scoped geocoding deferral mechanism.

Runtime verification is currently blocked: this environment has Ruby 3.3.8, while the repository specifies Ruby 3.4.2, and the locked Rails 8.1.3 dependencies are not installed. The attempted `bin/rails test` command exited with `Bundler::GemNotFound`. That is an environment failure, not a passed or failed application test.

I'm completing the source-level checks, test evidence and downloadable AFTER archive separately from that runtime limitation.

Download the Scenario 1 implementation

Scenario 1 AFTER Source

Complete Rails source snapshot · ZIP · 23 MB

Download Scenario 1 AFTER Implementation ↗

BEFORE → AFTER Source Diff

Unified patch · 57 KB

Download Source Changes Patch ↗

The ZIP contains the evolved application, tests, verification logs, README updates and `SCENARIO1_CHANGE_RECORD.md`.

2–3. Complete file changes and AFTER evidence

Seven modified original files

File	AFTER lines	Modification
`config/routes.rb`	30–35	Added generation GET/POST
`app/controllers/trip_items_controller.rb`	60–70, 138–253	Generation, signed confirmation, batch persistence
`app/models/trip_item.rb`	32–41	Instance-scoped geocoding deferral

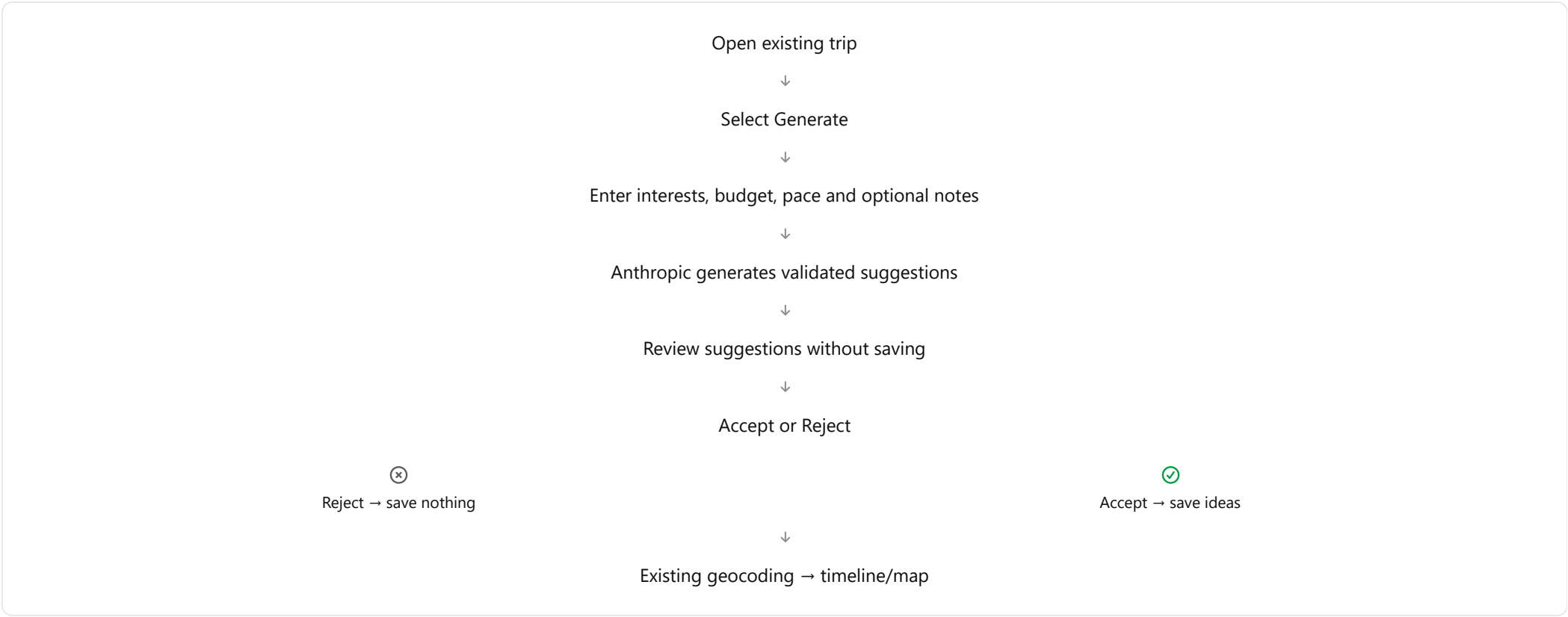
app/views/trip_items/import.html.erb	20–127	Reused preview with Accept/Reject
app/views/trips/_trip_panel.html.erb	236–247	Owner-only Generate link
config/application.rb	15	Enabled Rails test-unit railtie
README.md	112–148	Feature and verification instructions

Twelve newly introduced files

File	Responsibility
app/services/itinerary_suggestion_generator.rb	Anthropic integration and validation
app/views/trip_items/generate.html.erb	Preferences form
test/test_helper.rb	Rails testing setup
test/services/itinerary_suggestion_generator_test.rb	Generator tests
test/controllers/trip_items_generation_test.rb	Generation workflow and authorization tests
test/controllers/trip_items_import_regression_test.rb	Existing import and manual creation regression tests
evidence/isolated_service_checks.rb	Standalone validation checks
evidence/isolated_service_checks_output.txt	Executed checks
evidence/ruby_syntax_checks.txt	Syntax results
evidence/rails_test_attempt.log	Rails boot failure evidence
evidence/runtime_verification_summary.txt	Verification summary
SCENARIO1_CHANGE_RECORD.md	Detailed BEFORE/AFTER handoff

No original source files were removed.

4. Final user workflow



5–7. Authorization, Anthropic and validation

Authorization: Both generation and confirmation check the authenticated user's owner membership through `current_member_for(@trip)&.owner?`. The original trip-scoping logic prevents unrelated users accessing the trip. Ordinary JSON-import authorization remains unchanged.

Anthropic: The generator uses the existing Anthropic Ruby client style and requests structured tool output using `claude-haiku-4-5-20251001`. It requests no more than six ideas and accepts only approved activity fields.

Validation: Invalid preferences, empty output, malformed output, excessive suggestions, incorrect kinds, invalid names, invalid dates and unexpected fields are rejected. The confirmation also verifies a signed preview payload bound to the current trip and user, expiring after 20 minutes.

The generator does not create confirmed bookings, expense records or reservation references.

8. Callback-safety change

The original bulk importer globally disabled the `TripItem` geocoding callback using `skip_callback`, then restored it with `set_callback`.

The AFTER implementation instead constructs each batch item with:

```ruby
item.defer_geocoding = true
item.save!
```

`TripItem#geocode_from_address` returns early only for that particular object when the flag is set.

This means manual saves retain the registered callback, while batch-created items defer geocoding until the existing background job loads fresh records. Generated confirmations use a transaction and enqueue geocoding after persistence.

9–10. Tests and actual results

Standalone validation checks

6/6

20 assertions · 0 failures

Ruby syntax checks

9/9

All reported Syntax OK

⚠ Rails integration tests: blocked

Measurement	Actual result
Command	`bin/rails test`
Exit code	1
Tests executed	0
Passed	Not available
Failed	Not available

Errors	Not available
Skipped	Not available
Blocker	`Bundler::GemNotFound`

The six passing standalone tests exercise generator validation using lightweight substitutes for Rails utilities. They do not establish that the full Rails feature works end to end.

To complete runtime verification in an environment with the required Ruby and dependencies, run:

```bash
bundle install
RAILS_ENV=test bin/rails db:test:prepare
bin/rails test
```

11–12. Differences from the design and limitations

The implementation follows the agreed design, with three practical additions: signed preview confirmation to resist hidden-field tampering, transactional confirmation for all-or-nothing database writes, and an explicit message if geocoding enqueue fails after suggestions have already been saved.

The main outstanding limitation is runtime verification. The Rails routes, database persistence, rendered UI and real Anthropic request have not yet been exercised together.

Other limitations remain: generated suggestions may reference inaccurate or unavailable places; scheduling is tentative; trip-specific time zones are not modeled; and duplicate detection still relies primarily on case-insensitive names.

13. Next AFTER pattern-verification scope

The next teammate should inspect `TripItemsController#import`, `#generate`, `#bulk_create_items`, `ItinerarySuggestionGenerator`, `TripItem#geocode_from_address`, `Geocodable`, `GeocodeTripItemsJob`, the generation/import views, and the routes.

The main question is whether P05's existing collaboration has been preserved, modified or extended, or whether the new generation coordination deserves recognition as a separate Controller collaboration. P09's existing Turbo refresh behavior should also be checked.

No evolution statuses have been assigned and no pattern counts have been changed.

14. Final handoff status

- ✓ Separate AFTER source snapshot created and archived
- ✓ Original files and P01–P09 baseline preserved
- ✓ Implementation and focused tests written
- ✓ Syntax, isolated validation and archive checks completed
- ! Full Rails runtime verification still required
- ✓ Scenario 2 and cancellation policies untouched

Scenario 1 has an implemented, downloadable AFTER source snapshot, but it should not be called fully verified until the Rails integration tests can run. The next work should be runtime verification followed by source-based pattern-evolution assessment—not Scenario 2.

Scenario 1 — Personalized Itinerary Generation has now been implemented in the separate AFTER source version:

scenario1_personalized_itinerary

Do not start Scenario 2 yet.

The original baseline P01–P09 is frozen and must not be edited retroactively.

The implementation report claims:

1. An in-app Anthropic itinerary generator was added.
2. TripItemsController#generate now coordinates generation, preview and confirmation.
3. The existing bulk_create_items behavior is reused for persistence.

4. TripItem batch geocoding now uses instance-scoped deferral instead of globally disabling the callback.
5. Existing GeocodeTripItemsJob and Turbo refresh behavior are reused.
6. Full Rails tests could not run because the available Ruby/dependency environment does not match the repository.
7. Standalone validation and Ruby syntax checks passed, but these are not substitutes for Rails integration tests.

Now perform the Scenario 1 AFTER verification and pattern-evolution analysis.

Use the actual Scenario 1 AFTER source snapshot and the original baseline source. Do not rely only on the implementation summary.

1. Verify the implementation claims against the actual changed source.

Inspect at minimum:

- config/routes.rb
- app/controllers/trip_items_controller.rb
- app/services/itinerary_suggestion_generator.rb
- app/models/trip_item.rb
- app/models/concerns/geocodable.rb
- app/jobs/geocode_trip_items_job.rb
- app/views/trip_items/generate.html.erb
- app/views/trip_items/import.html.erb
- app/views/trips/_trip_panel.html.erb
- config/application.rb
- the new test files
- SCENARIO1_CHANGE_RECORD.md
- the source diff

For every important claim, record reliable AFTER file and line ranges.

2. Verify that the baseline behavior has not been incorrectly rewritten.

Check specifically that:

- ordinary JSON import still exists
- manual TripItem creation still exists
- imported/generated records are persisted as ideas
- duplicate handling is still present
- geocoding is still delegated to GeocodeTripItemsJob
- the Turbo trip refresh mechanism still exists
- P01–P04 and P06–P08 have not been accidentally changed by unrelated edits

Do not assume “unchanged” merely because the implementation report says so.

3. Examine the callback refactoring carefully.

Compare BEFORE and AFTER.

BEFORE:
bulk_create_items used TripItem.skip_callback and TripItem.set_callback.

AFTER:
batch-created TripItem objects are reported to use an instance-level defer_geocoding flag.

Determine:

- whether the global callback manipulation is actually gone
- whether the normal model callback remains registered
- whether the flag affects only the intended object
- whether the background job later loads a fresh object that can geocode normally
- whether any source-level path could leave geocoding permanently suppressed

Record this as a refactoring verification, not as a new design pattern unless a recognized pattern mechanism genuinely exists.

4. Reconstruct the actual AFTER Personalized Itinerary Generation collaboration.

Start from the user request and trace:

route
→ TripItemsController#generate
→ authorization
→ preference validation
→ ItinerarySuggestionGenerator
→ Anthropic
→ validated suggestions
→ preview
→ signed confirmation
→ bulk_create_items
→ TripItem persistence
→ GeocodeTripItemsJob
→ Turbo refresh
→ timeline/map

Clearly distinguish application participants from Rails, Anthropic and Turbo framework/dependency participants.

5. Now compare the AFTER collaboration with frozen P05:

P05 — Controller (GRASP) — Bulk Itinerary Import Coordination.

Do not decide from class names alone.

Compare:

- system event
- responsibility
- coordinator
- collaborators
- entry route
- input source
- preview responsibility
- persistence responsibility
- geocoding delegation
- outcome

Then decide whether P05 is:

preserved
modified
extended
replaced
removed

Use exactly one status.

6. Evaluate the tentative candidate:

Controller (GRASP) — Personalized Itinerary Generation Coordination.

The main question is whether this is a genuinely independent Controller collaboration or whether it is simply an additional entry path inside the already-counted P05 collaboration.

Apply the same counting discipline used in Part 2.

To confirm it as a new instance, show that it represents a distinct system event/use-case coordination with a sufficiently independent responsibility and participant collaboration.

To reject it as a separate instance, explain why counting it separately would double-count P05.

Give one final status:

CONFIRMED NEW INSTANCE
or
REJECTED AS SEPARATE INSTANCE

Do not leave it uncertain unless the actual source is genuinely insufficient.

7. Re-evaluate P09:

P09 — Observer (GoF) — Trip-Scoped Turbo Refresh Notifications.

Determine whether Scenario 1:

- changes P09
- extends P09
- or merely exercises the existing unchanged mechanism

Assign the appropriate evolution status with source evidence.

8. Check all remaining frozen instances P01–P04 and P06–P08.

For each, determine whether Scenario 1 left the actual collaboration unchanged.

Do not call something modified simply because one of its participant classes is referenced elsewhere.

9. Produce the complete Scenario 1 pattern-evolution result for P01–P09.

For each instance give:

- Pattern ID
- stable instance name
- pattern name/category
- Scenario 1 status
- AFTER participants
- AFTER source locations
- concise reason
- change description

If a new pattern instance is confirmed, add it separately with status:

newly_introduced

Do not modify the frozen baseline row.

10. Recalculate the Scenario 1 AFTER counts only after the classification is complete.

Show:

- Controller instances
- Information Expert instances
- Observer instances
- total pattern types
- total current instances

If the new Controller candidate is rejected, explain why the total remains 9.

If it is confirmed, explain why the total becomes 10.

Do not count a refactoring observation as another pattern instance.

11. Verify the reported test evidence.

Separate these clearly:

- Ruby syntax checks
- isolated service/validation checks
- Rails integration tests
- live Anthropic verification
- live geocoding verification

Do not merge them into one “tests passed” statement.

If the full Rails command is still blocked, preserve the exact blocker and do not estimate results.

12. Record the Scenario 1 refactoring evidence.

For the callback change state:

- original code smell/risk
- exact BEFORE mechanism
- exact AFTER mechanism
- refactoring technique in plain software-engineering terms
- intended benefit
- behavior that should remain unchanged
- what remains unverified at runtime

Do not invent a named refactoring technique if a standard name is not clearly applicable.

13. Do not modify the source in this step unless a factual verification error prevents analysis.

If you discover an implementation defect, stop and report it rather than silently fixing the code.

14. Do not begin multiple booking providers or cancellation policies.

15. Do not produce the final cross-scenario patterns_evolution.csv yet.

Instead, prepare the verified Scenario 1 rows/data that can later be appended to that CSV.

At the end give me:

1. Source-verification result.
2. Verified AFTER workflow.
3. Callback-refactoring verification.
4. P05 evolution decision.
5. New Controller candidate decision.
6. P09 evolution decision.
7. Status of all P01–P09.
8. Any newly introduced confirmed instance.
9. Scenario 1 AFTER pattern counts.
10. Verification/test evidence split by what was actually executed.
11. Refactoring evidence.
12. Remaining limitations.
13. A compact Scenario 1 evolution record ready to append later to patterns_evolution.csv.
14. Whether Scenario 1 is sufficiently documented to move to Scenario 2.

Do not start Scenario 2.

Worked for 5m 36s

DaPlanStan — Scenario 1 AFTER Verification

Source analysis complete Rails runtime verification blocked

I inspected the original baseline ZIP, Scenario 1 AFTER ZIP, frozen Part 2 pattern records, implementation diff, changed source files, tests and verification logs.

Final pattern-evolution decisions:

- **P05 — MODIFIED**
- **New personalized-generation Controller — CONFIRMED NEW INSTANCE**
- **P09 — PRESERVED**
- **P01–P04 and P06–P08 — PRESERVED**

BEFORE instances	AFTER instances
9	10
Frozen original baseline	Includes one new Controller

1. Source-verification result

The archive-level comparison was performed using file contents, not the previous implementation summary.

Finding	Verified result
Original source files	400
AFTER source files	412
Original files modified	7
New files	12
Original files deleted	0
Source archive integrity	Passed

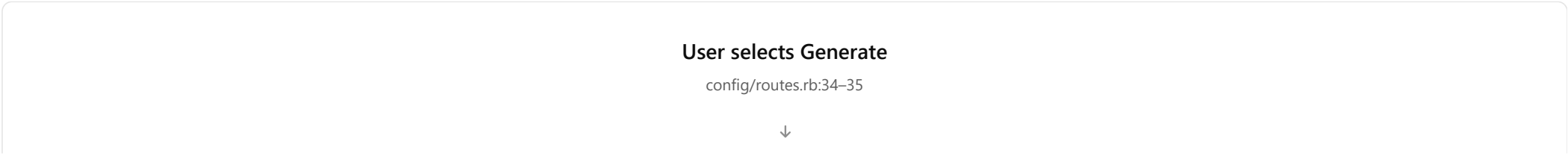
The source diff agrees with the seven modified and twelve newly added files. The original baseline ZIP checksum is unchanged.

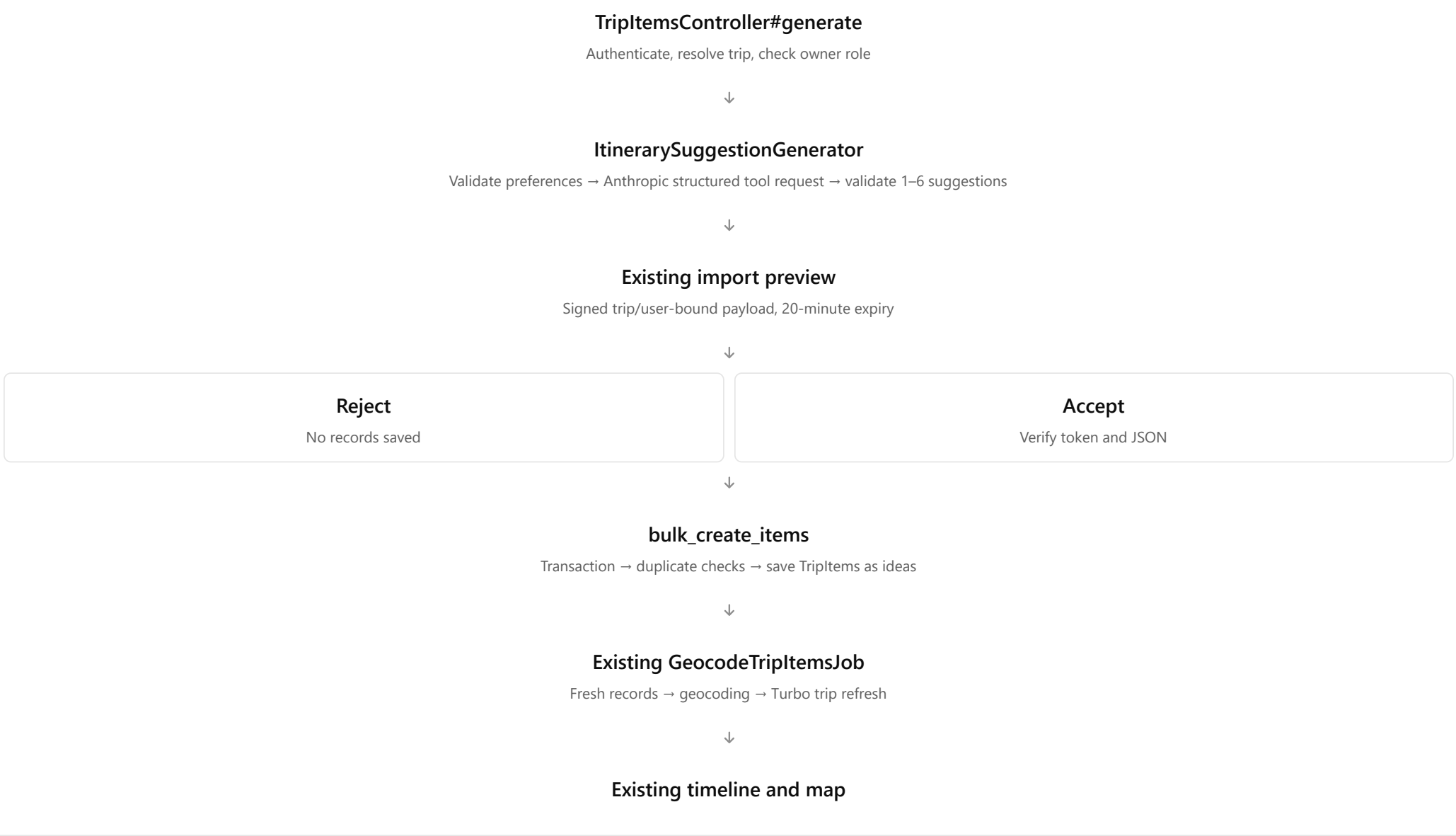
Verified implementation evidence

Claim	AFTER source evidence	Finding
In-app Anthropic generation	`app/services/itinerary_suggestion_generator.rb:111–138`	Present
Generation routes	`config/routes.rb:30–41`	Present
Owner-only generation	`app/controllers/trip_items_controller.rb:60–70`	Present
Preference validation	Generator `:42–54`	Present
Maximum six suggestions and field validation	Generator `:56–104`	Present
Preview without persistence	Controller `:138–156`	Present
Signed confirmation	Controller `:158–197`	Present
Reused batch persistence	Controller `:201–253`	Present
Instance-scoped callback deferral	`app/models/trip_item.rb:32–41`	Present
Existing geocoding and Turbo broadcast	`app/jobs/geocode_trip_items_job.rb:4–21`	Unchanged
Existing JSON import	Controller `:31–58`	Retained
Manual item creation	Controller `:72–85`	Retained

These are source-level findings. They do not establish that all behavior has passed integration tests.

2. Verified AFTER workflow





The application participants are the Rails controllers, generator service, Trip, TripMember, TripItem, Geocodable, geocoding job and views.

Rails routing, Active Record, Active Job, MessageVerifier and Turbo provide framework mechanisms. Anthropic is the external generation provider. These are not independently counted application pattern instances.

3. Callback-refactoring verification

BEFORE

`trip_items_controller.rb:134,172-174`

Ruby

```ruby
TripItem.skip_callback(
  :save, :after,
  :geocode_from_address
)
```

Restored globally with `set_callback`.

AFTER

`trip_items_controller.rb:236-238`

Ruby

```ruby
item.defer_geocoding = true
item.save!
```

The model checks the flag for that instance.

The global callback manipulation has been removed from the AFTER application source.

The normal callback remains registered in `app/models/concerns/geocodable.rb:4-6`. The new `TripItem#geocode_from_address` override returns early only when that instance's transient `defer_geocoding` flag is set (`trip_item.rb:32-41`).

`GeocodeTripItemsJob#perform` subsequently calls `TripItem.find_by(id: id)` at line 9, obtaining a fresh object without the transient flag, and calls `geocode_from_address` at line 12.

Refactoring conclusion: The implementation replaces global callback configuration changes with instance-scoped geocoding deferral. No permanent callback removal or suppression path was identified from the source.

This is a reliability refactoring, **not a new design pattern**.

The intended manual-save and background-job behavior is structurally preserved, but concurrent execution and actual Rails callback behavior still need runtime verification.

4. P05 evolution decision

MODIFIED

P05 — Controller (GRASP) — Bulk Itinerary Import Coordination

Comparison	BEFORE	AFTER
System event	Import external JSON	Same
Coordinator	`TripItemsController#import`	Same
Input	Pasted/uploaded JSON	Same
Preview	Import view	Same, now supports generated mode
Persistence	`bulk_create_items`	Same method, revised implementation
Duplicates	Case-insensitive name matching	Retained
Status	`idea`	Retained
Geocoding	Background job	Retained
Callback management	Global toggle	Per-instance deferral
Error/queue behavior	Best-effort importer	Original behavior retained; helper also supports strict mode for generation

Decision: `modified`.

P05 still exists, is responsible for the same external JSON-import event, and continues to use the same principal participants.

However, its batch-persistence implementation was materially changed to support safer callback handling and optional strict/queue behavior. Therefore, `preserved` would understate the implementation change.

It is not `replaced` or `removed`, and I would not classify P05 itself as `extended` into personalized generation because that is separately coordinated through a new system event.

5. New Controller candidate decision

CONFIRMED NEW INSTANCE

S1-N01 — Controller (GRASP) — Personalized Itinerary Generation Coordination

Criterion	Evidence
Distinct user event	User requests personalized ideas for an existing trip
Separate route	`config/routes.rb:34–35`
Dedicated coordinator	`TripItemsController#generate:60–70`
Distinct input	Interests, budget, pace and notes
Distinct collaborator	`ItinerarySuggestionGenerator`
Distinct external behavior	Anthropic structured generation
Separate preview security	Signed, user/trip-bound confirmation
Shared downstream operation	Existing P05 `bulk_create_items`
Distinct outcome	Personalized ideas produced and optionally committed

Why it counts separately

The original P05 does not interpret preferences, invoke an AI generator, manage a generation-specific preview token or coordinate the new personalization request.

Those responsibilities belong to the newly implemented generation event, whose coordinator delegates persistence to the already established batch helper.

Sharing that helper does not make the two system events identical, and the helper itself is not counted again.

This satisfies the existing Part 2 discipline of counting a distinct, source-supported Controller collaboration rather than merely counting controller classes or methods.

S1-N01 is a confirmed new source-level pattern instance, with `newly_introduced` as its evolution status. Runtime functionality remains separately unverified.

6. P09 decision

PRESERVED

P09 — Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

The following files and behaviors are byte-for-byte unchanged:

- `app/models/inbox_item.rb:1-5` — `broadcasts_refreshes_to :trip`
- `app/jobs/geocode_trip_items_job.rb:4-21` — explicit trip refresh broadcast
- `app/views/trips/show.html.erb:2-9` — trip-stream subscription
- `app/views/inbox_items/index.html.erb:1` — inbox subscription

Scenario 1 invokes the existing geocoding job; it does not introduce a different subject, observer, subscription or notification collaboration.

P09 status: `preserved` .

7. Complete P01–P09 evolution results

All stable instance names have been retained.

ID	Stable pattern instance	Type	AFTER status
P01	Parsed Email to Trip Record Linking	Information Expert (GRASP)	`preserved`
P02	Inbound Travel Email Intake Coordination	Controller (GRASP)	`preserved`
P03	Trip Creation Coordination	Controller (GRASP)	`preserved`
P04	Trip Membership Management Coordination	Controller (GRASP)	`preserved`
P05	Bulk Itinerary Import Coordination	Controller (GRASP)	`modified`
P06	OAuth Sign-in and Identity Linking Coordination	Controller (GRASP)	`preserved`
P07	Account Traveler Enrollment Coordination	Controller (GRASP)	`preserved`
P08	Outbound Mail Delivery Logging	Observer (GoF)	`preserved`
P09	Trip-Scoped Turbo Refresh Notifications	Observer (GoF)	`preserved`
S1-N01	Personalized Itinerary Generation Coordination	Controller (GRASP)	`newly_introduced`

AFTER participant and source-location audit

Pattern	Principal AFTER participants	Important AFTER source locations
P01	InboxItem, InboxItemsController, TripItem, Trip, Expense	`inbox_item.rb:19–207`; `inbox_items_controller.rb:16–35`; `trip_item.rb:1–28`
P02	ApplicationMailbox, TripMailbox, InboxItem, ParseInboxItemJob, TripMailer	`trip_mailbox.rb:2–60`; `parse_inbox_item_job.rb:99–139`
P03	TripsController, Trip, Account, TripMember, FetchTripCoverImageJob	`trips_controller.rb:35–48`; `trip.rb:4–19,31–37`; `fetch_trip_cover_image_job.rb:4–32`
P04	TripMembersController, TripMember, Traveler, InvitationMailer	`trip_members_controller.rb:6–65`; `trip_member.rb:1–6`; `routes.rb:42`
P05	TripItemsController, TripItem, Geocodable, GeocodeTripItemsJob, import view	`trip_items_controller.rb:31–58,199–259`; `trip_item.rb:1–41`; `geocode_trip_items_job.rb:4–21`
P06	SessionsController, UserIdentity, User, Account, Traveler, OmniAuth	`sessions_controller.rb:4–81`; `user_identity.rb:1–8`; `omniauth.rb:1–16`
P07	TravelersController, Account, Traveler, User, InvitationMailer	`travelers_controller.rb:5–56`; `traveler.rb:1–28`; `routes.rb:50–52`
P08	MailLogger, InvitationMailer, TripMailer, Action Mailer	`mail_logger.rb:1–7`; `invitation_mailer.rb:2–13`; `trip_mailer.rb:2–10`
P09	InboxItem, GeocodeTripItemsJob, Turbo Streams, trip/inbox views	`inbox_item.rb:1–5`; `geocode_trip_items_job.rb:4–21`; `trips/show.html.erb:2–9`
S1-N01	TripItemsController, ItinerarySuggestionGenerator, Trip, TripMember, TripItem, shared preview and batch helper	`trip_items_controller.rb:60–70,138–253`; `itinerary_suggestion_generator.rb:42–153`; `generate.html.erb:1–45`

The downloadable JSON record includes the additional collaborators, framework roles, full source-location strings, status reasons and change descriptions for every instance.

For P01, the `TripItem` file changed only in its geocoding override; the original email-linking Information Expert collaboration remains unchanged. For P04 and P07, nested route line positions shifted, but membership and enrollment coordination did not change.

8. Recalculated AFTER counts

Pattern instances by version

Metric	BEFORE	Scenario 1 AFTER
Controller (GRASP)	6	**7**
Information Expert (GRASP)	1	1
Observer (GoF)	2	2

Total current instances	**9**	**10**
Distinct pattern types	**3**	**3**

The total increases by exactly one because S1-N01 is a new Controller collaboration. The callback refactoring and reuse of P09 do not introduce additional instances.

9. Actual verification and test evidence

I reran the source-level checks in the available environment.

Verification tier	Actual result	Interpretation
Ruby syntax	9 of 9 files `Syntax OK`	Passed
Isolated validation	6 tests, 20 assertions, 0 failures, 0 errors, 0 skips	Passed
Rails integration	0 cases executed	Blocked before discovery
Live Anthropic	Not executed	Unverified
Live geocoding/Turbo	Not executed	Unverified

The Rails command was:

Bash

```
bin/rails test
```

It failed while loading dependencies with:

Plain text

```
Bundler::GemNotFound:
Could not find rails-8.1.3 ...
```

Ruby 3.3.8 is available, while the repository specifies Ruby 3.4.2. The locked Rails and related application gems are not installed.

Therefore there are **no meaningful Rails pass, fail, error or skip counts yet**.

The Rails test files exist, covering generator validation, owner authorization, preview, confirmation, duplicates, tampering, callback behavior, manual creation and import regression. Their assertions remain unexecuted in the full application environment.

10. Remaining limitations and implementation risks

The static inspection did not reveal a reason to reclassify or remove any of the other frozen pattern collaborations.

The principal limitations are runtime verification gaps and inherited or deliberately narrow feature behavior:

- Rails authorization, rendering and database transactions have not been exercised end to end.
- The Anthropic API model and actual structured response behavior have not been tested live.
- Geocoding and Turbo refresh delivery have not been exercised in a running application.
- Manual creation and ordinary JSON-import regressions are covered by written tests, but those Rails tests have not run.
- Name-based deduplication can treat two different places with the same name as duplicates.
- A background-job enqueue or execution failure can leave map pins missing until another geocoding attempt.
- The application has no destination-specific time-zone model.

No implementation fixes were made during this verification step.

11. Append-ready Scenario 1 records

Full AFTER Verification Report

Source comparison, exact line references, collaboration reasoning, refactoring and test evidence

Download Scenario 1 Verification Report ↗

Scenario 1 Evolution Records

JSON draft · 10 AFTER records · full participant roles, locations, statuses and change descriptions

Download Append-Ready Evolution Data ↗

The JSON contains the required evolution-data fields, along with additional audit metadata. It is deliberately **not** named or structured as the final `patterns_evolution.csv`.

When building that final CSV, retain the frozen nine BEFORE rows unchanged, then incorporate these AFTER observations using the prescribed columns and updated source locations.

12. Is Scenario 1 ready for Scenario 2?

Documentation and source-pattern classification: Yes. Full runtime verification: No.

The Scenario 1 source comparison, collaboration reconstruction, pattern-status decisions, counts and refactoring record are complete enough for a coursework handoff.

Before calling the implementation fully verified, the team should run the Rails test suite in a compatible environment and preserve its actual results. Any failures should be resolved before final coursework submission.

Scenario 1 final source-level pattern result: 10 instances across 3 types, including one newly introduced GRASP Controller.

The frozen P01–P09 BEFORE baseline remains unchanged. Scenario 2 and cancellation policies have not been started.

We have completed Scenario 1 — Personalized Itinerary Generation.

Use the Scenario 1 AFTER source version as the starting point for the next evolution:

scenario1_personalized_itinerary

Do not return to the original baseline for implementation.

Current verified pattern state after Scenario 1:

P01 Parsed Email to Trip Record Linking — Information Expert (GRASP) — preserved
P02 Inbound Travel Email Intake Coordination — Controller (GRASP) — preserved
P03 Trip Creation Coordination — Controller (GRASP) — preserved

P04 Trip Membership Management Coordination — Controller (GRASP) — preserved
P05 Bulk Itinerary Import Coordination — Controller (GRASP) — modified
P06 OAuth Sign-in and Identity Linking Coordination — Controller (GRASP) — preserved
P07 Account Traveler Enrollment Coordination — Controller (GRASP) — preserved
P08 Outbound Mail Delivery Logging — Observer (GoF) — preserved
P09 Trip-Scoped Turbo Refresh Notifications — Observer (GoF) — preserved
S1-N01 Personalized Itinerary Generation Coordination — Controller (GRASP) — newly introduced

Current counts:

Controller (GRASP) = 7
Information Expert (GRASP) = 1
Observer (GoF) = 2
Total instances = 10
Distinct pattern types = 3

Scenario 1 Rails integration tests remain blocked by the environment. Do not describe them as passed.

Now perform Scenario 2 DESIGN ONLY.

Scenario 2 requirement:

SUPPORT MULTIPLE BOOKING PROVIDERS.

Do not modify source code yet.

1. Reinspect the actual Scenario 1 AFTER source.

Focus especially on:

- InboxItem

- InboxItemsController
- TripItem
- TripItemsController
- TripMailbox
- ParseInboxItemJob
- Trip and TripMember
- current database schema
- routes
- views related to imported booking information
- any provider-like fields already present
- current booking/ref/confirmation data handling

Do not rely only on previous notes.

2. Establish the current BEFORE behavior for Scenario 2.

Explain exactly how booking information currently enters the application.

Trace:

booking confirmation email
→ mailbox
→ InboxItem
→ ParseInboxItemJob
→ Anthropic extraction
→ parsed booking/provider/reference data
→ InboxItem confirmation/linking
→ TripItem and/or Expense

Identify the exact current role of provider names.

Determine whether provider information is currently:

- only descriptive metadata
- used in any branching logic
- used for API calls
- represented by classes/modules
- persisted in the schema
- used for booking execution

3. Define a realistic scope for “multiple booking providers”.

Do not build real Expedia/Booking.com/airline integrations unless the source and environment make that reasonable.

Compare at least these implementation choices:

A. Provider-specific adapters for normalized booking data
B. Provider-specific strategy objects for provider behavior
C. Direct conditional logic in controller/model
D. Live external booking-provider APIs
E. A lightweight provider gateway abstraction with local/mock provider implementations

For each option explain:

- fit with the existing application
- implementation complexity
- external API requirements
- testability
- architectural effect
- whether it genuinely corresponds to a recognized pattern or would be forced

Then recommend one implementation approach for this coursework.

4. The Scenario 2 change must be demonstrable without depending on paid or inaccessible APIs.

Prefer deterministic or locally testable provider behavior.

If external provider APIs would prevent reliable testing, design provider implementations that simulate or normalize provider-specific responses while preserving a realistic booking-provider abstraction.

Clearly state what the feature does and does not claim.

5. Define the exact user-visible behavior.

For example, determine whether the evolved application should support:

- selecting a booking provider
- retrieving or constructing provider-specific booking options
- normalizing different provider response formats
- storing provider name/reference
- displaying provider information
- confirming a selected booking option
- maintaining current TripItem behavior

Do not silently invent payment processing.

6. Decide whether a database migration is required.

Inspect the existing schema first.

If existing TripItem or InboxItem fields already store enough provider/reference information, prefer reuse.

If a new field/model is genuinely required, justify it.

Do not create a migration merely to make a pattern easier to demonstrate.

7. Identify the likely production files that would need modification.

For each file give:

- current responsibility
- proposed change
- reason

Also identify proposed new files.

Do not implement them yet.

8. Design the provider abstraction.

If the source supports it, specify:

- common interface/responsibility
- provider implementations
- selection mechanism
- caller/context
- normalized return structure
- failure behavior

Avoid fake textbook participants.

Map every proposed role to a concrete Ruby class/module/method that would actually be created.

9. Examine whether this evolution would genuinely introduce a Strategy, Adapter, Factory or another recognized pattern.

Do not confirm any pattern yet.

For every candidate explain:

- why it might qualify
- what source structure would be needed
- what would make it a false positive

Remember that multiple provider classes participating in one interchangeable collaboration count as ONE pattern instance, not one instance per provider.

10. Protect the current Scenario 1 behavior.

The Scenario 2 design must preserve:

- personalized itinerary generation
- original JSON import
- batch persistence
- instance-scoped geocoding deferral
- existing geocoding job
- Turbo refresh
- current email ingestion
- current authentication and membership behavior

Identify regression risks.

11. Define acceptance criteria before implementation.

Include at minimum:

- at least two provider implementations
- provider selection
- normalized result shape
- invalid/unknown provider handling
- provider-specific failure handling
- no accidental cross-provider state leakage
- current import/email behavior preserved
- Scenario 1 generation preserved
- tests do not make live external requests

12. Define the test plan.

Separate:

- unit tests for provider abstraction
- controller/service tests
- persistence tests
- regression tests
- optional runtime/manual checks

All external provider behavior should be stubbed or local for automated tests.

13. Define the BEFORE → AFTER flow.

Show the exact expected Scenario 2 flow in concise arrow form.

14. Analyze expected effects on the current 10 confirmed pattern instances.

For each of P01–P09 and S1-N01 state only:

- likely directly affected
- possibly affected
- likely unchanged

Do not assign final evolution statuses yet.

15. Identify tentative new pattern candidates created by the proposed provider design.

Do not add them to counts yet.

16. Recommend a source-version name for the implemented AFTER snapshot, such as:

scenario2_multiple_booking_providers

17. Give an implementation sequence for the next coding prompt.

At the end provide:

1. Verified Scenario 2 BEFORE state.
2. Recommended feature scope.
3. Chosen provider-design approach and why.
4. Proposed classes/modules and responsibilities.
5. Whether a migration is needed.
6. Files to modify.
7. Files to add.
8. User workflow.
9. Acceptance criteria.
10. Test plan.
11. Regression risks.
12. Tentative pattern candidates only.

13. Expected impact on current P01–P09 and S1-N01.
14. Step-by-step coding sequence for the next prompt.

Do not modify the source.
Do not assign final pattern-evolution statuses.
Do not change pattern counts.
Do not start cancellation-policy work.

Worked for 3m 25s

DaPlanStan — Scenario 2: Multiple Booking Providers

Design-only stage | Starting source: `scenario1_personalized_itinerary`

Source reinspection complete

I re-inspected the actual Scenario 1 AFTER ZIP, including the Rails models, controllers, mailbox, parsing job, routes, schema and booking-related views. I verified that the inspected files match the supplied AFTER archive.

Recommended design: Add a small booking-provider comparison feature using **two locally simulated providers and a common normalized offer interface**. Users can browse provider-specific sample offers and add a selected offer to their itinerary as an `idea`.

This meets the coursework's multiple-provider requirement without requiring inaccessible booking APIs or pretending that a real reservation has been made.

The existing 10 confirmed pattern instances and 3 pattern types remain unchanged during this design step.

1. Verified Scenario 2 BEFORE state

How booking information currently enters DaPlanStan

1. Booking confirmation email

Traveler forwards a confirmation to the trip inbox

2. TripMailbox#process

Identifies trip, validates sender and creates InboxItem

↓

3. ParseInboxItemJob

Approved email is sent to Anthropic for structured extraction

↓

4. InboxItem.parsed_data

Extracted provider, booking type, reference, dates and costs stored as JSON

↓

5. InboxItemsController#update

User reviews and chooses Add to trip

↓

6. InboxItem#fill_missing_links!

Finds or creates TripItem; creates Expense when eligible

↓

7. Existing itinerary

Displays confirmed items, references and associated expenses

Exact BEFORE source evidence

All paths refer to the **Scenario 1 AFTER snapshot**, which is now Scenario 2's BEFORE version.

Source file and lines	Verified current responsibility
`app/mailboxes/trip_mailbox.rb:2–31`	Identifies the trip and sender, creates InboxItem and queues parsing
`app/jobs/parse_inbox_item_job.rb:6–18`	Instructs Anthropic to extract facts conservatively from booking confirmations
`app/jobs/parse_inbox_item_job.rb:20–97`	Defines structured extraction fields

`app/jobs/parse_inbox_item_job.rb:63-79`	Defines booking type, provider, confirmation reference, location, dates and price
`app/jobs/parse_inbox_item_job.rb:99-139`	Calls Anthropic and saves parsed data
`app/models/inbox_item.rb:19-25`	Stores/retrieves parsed JSON
`app/controllers/inbox_items_controller.rb:16-35`	Handles confirm, discard, relink and retry
`app/models/inbox_item.rb:67-99`	Finds or creates related TripItems
`app/models/inbox_item.rb:101-185`	Maps travel legs, accommodations and bookings
`app/models/inbox_item.rb:191-207`	Creates related booking expenses when eligible
`app/views/inbox_items/show.html.erb:50-135`	Displays email review actions and linked records
`app/views/trip_items/show.html.erb:127-157`	Displays confirmation reference and originating email
`app/views/trip_items/show.html.erb:207-227`	Displays linked expense

What does `provider` mean today?

At `app/jobs/parse_inbox_item_job.rb:68-78`, Anthropic extracts a booking `provider` value.

At `app/models/inbox_item.rb:168-178`, that value is passed as the name of a `TripItem`:

Ruby

```ruby
item = find_or_upsert.call(booking["provider"], kind, {
  starts_at: booking["datetime"].presence,
  address: booking["location"],
  confirmation_ref: booking["confirmation_reference"]
```

```
    # Other attributes omitted
})
```

The `booking_type` determines the itinerary kind. The provider name itself does not select any provider implementation.

Question	Verified BEFORE answer
Are provider names extracted from email?	Yes
Are they retained in parsed JSON?	Yes
Are they used as TripItem names?	Yes
Are they used to choose booking logic?	No
Are booking-provider API clients present?	None found
Are separate booking-provider classes present?	None found
Is there a dedicated TripItem provider column?	No
Can users search or compare provider offers?	No
Can the application make live bookings?	No

The `provider` field in `user_identities` and the Google/Apple configuration in `config/initializers/omniauth.rb` refer to authentication providers, not booking providers.

Also, the original legacy `bookings` table was dropped by `db/migrate/20260408155636_remove_legacy_travel_models.rb:12–16`. Current booking information lives in `InboxItem`, `TripItem` and `Expense`.

Existing schema relevant to Scenario 2

Entity	Existing fields	Design implication
Trip	`title`, `description`, `start_date`, `end_date`	Can supply trip context; no reliable dedicated destination

TripItem	`name`, `kind`, `status`, `notes`, `address`	Can represent a prospective provider option
TripItem	`amount`, `currency`	Can hold an explicitly labeled sample estimated cost
TripItem	`confirmation_ref`, `url`	Can represent genuine reference/link data, but should not be populated with fabricated bookings
InboxItem	`parsed_data_json`, `review_status`	Already handles real confirmation-email extraction
Expense	`source`, `amount`, `confirmed_at`	Existing expense tracking; should not be created by demo offer selection

Key limitation: `InboxItem#fill_missing_links!` matches records using case-insensitive item names (`inbox_item.rb:79–99`). Multiple reservations with the same provider name may be confused. Scenario 2 should avoid increasing that risk and regression-test interactions with provider-created ideas.

2. Recommended feature scope

Multiple Booking Providers — V1

Users can select one of two simulated providers, inspect normalized sample travel offers and save a selected option into an existing trip as an itinerary idea.

Included

- Two independent provider implementations.
- Provider and demo-destination selection.
- Distinct provider response formats.
- Normalized offer presentation.
- Save selected offer to TripItem as `idea`.
- Display provider identity and sample offer details.
- Reuse current geocoding, timeline and map mechanisms.

Excluded

- Real Expedia, Booking.com or airline APIs.
- Live inventory, availability or pricing.

- Payment processing.
- External booking execution.
- Fabricated confirmation references.
- Automatic expenses.
- Cancellation-policy functionality.

Because the Trip schema has no destination field, the demo should offer a small validated destination selector, such as Rome and Paris. This prevents the application from incorrectly treating an arbitrary trip title as a geographic location.

The proposed demonstration providers are fictional and clearly labeled:

- **Atlas Demo** — supplies mock provider offers in one JSON format.
- **Beacon Demo** — supplies different mock provider offers in another JSON format.

Both will represent sample travel activities or stays. The user can compare their descriptions and simulated costs, but no real-world price or availability claims will be made.

3. Comparison of implementation approaches

Option	Fit and complexity	External API	Testability	Pattern assessment
A. Provider-specific adapters for normalized data	Good fit; small–medium	Not required with fixtures	High	Adapter candidate if distinct interfaces are actually adapted
B. Provider-specific strategies	Medium; useful if providers implement differing sourcing algorithms	Optional	High	Strategy candidate, but separate classes alone are insufficient
C. Direct conditional logic	Smallest initial change, but provider rules accumulate in controller	None	Moderate	Generally ordinary conditional code
D. Live provider APIs	Most realistic operationally, but high complexity and uncertain access	Required	Low without working mocks/credentials	No pattern guaranteed

E. Lightweight gateway with local providers	**Best fit for coursework**; contained and demonstrable	None	High	Possible Adapter and Controller collaborations, subject to verification

Selected approach: E, with provider-specific normalization from A

Use a lightweight provider registry, two provider-specific normalizers and local JSON sample responses.

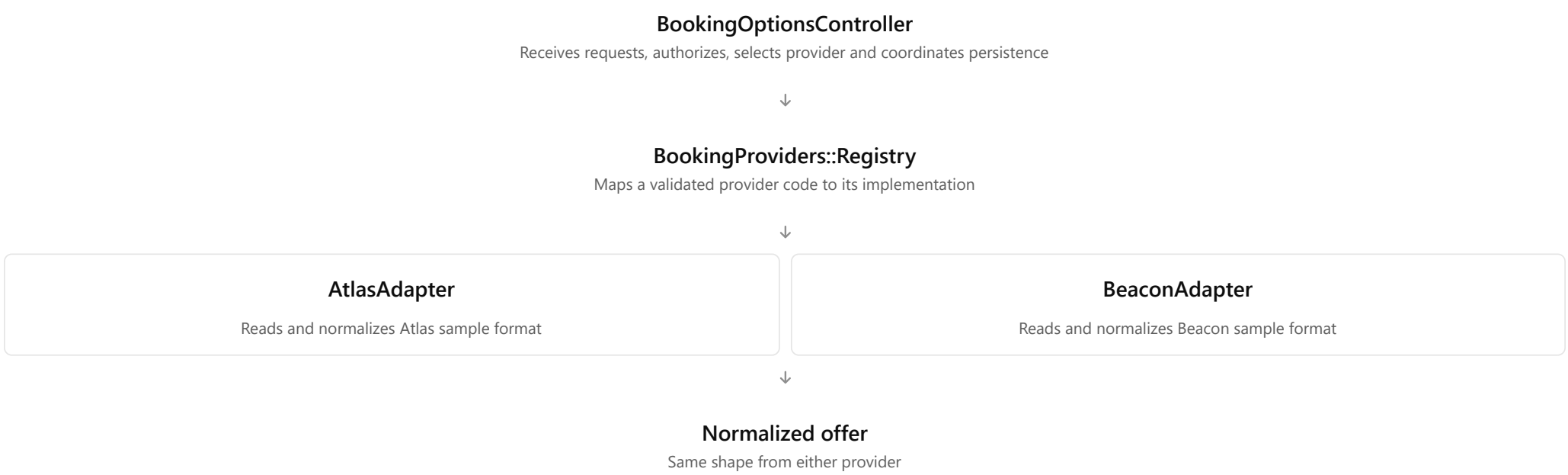
Do not add a generic booking engine, a hierarchy of abstract base classes, a real API SDK or a complex factory.

The two provider implementations will share a small calling contract while normalizing different data formats into one consistent offer structure.

This supplies a real application responsibility: insulating the controller and UI from provider-specific response shapes.

4. Proposed provider classes and responsibilities

Proposed application collaboration



↓

TripItem

Selected option saved as idea, then existing geocoding and display

Concrete planned Ruby responsibilities

Proposed class/module	Proposed methods	Purpose
`BookingOptionsController`	`index`, `create`	Show offers and add one selected offer
`BookingProviders::Registry`	`.find(code)`, `.available`	Resolve only approved providers
`BookingProviders::AtlasAdapter`	`#offers_for(destination_code:)`	Read Atlas mock data and normalize its response
`BookingProviders::BeaconAdapter`	`#offers_for(destination_code:)`	Read Beacon mock data and normalize its response

A separate abstract provider superclass is not needed.

Common normalized result

Both implementations should return hashes with the same shape:

‹/› Ruby

```ruby
{
  provider_code: "atlas_demo",
  provider_name: "Atlas Demo",
  offer_id: "atlas-rome-walk-01",
  name: "Historic City Walk",
  kind: "do",
  address: "Rome, Italy",
  description: "Sample guided walking activity",
  sample_amount: 25.00,
```

```
  currency: "EUR"
}
```

This is a proposed data contract, not output already produced by the repository.

The provider identity and offer ID should be unique within the demo catalogue. The `kind` must be an accepted `TripItem` enum value.

The two raw fixture formats should genuinely differ. For example, Atlas can use `offers`, `title` and nested `price`, while Beacon uses `results`, `label` and separate amount/currency fields.

The adapters normalize those differences rather than forcing the controller to understand each source schema.

Selection and failure behavior

The registry accepts only fixed codes such as `atlas_demo` and `beacon_demo`. Unknown codes fail cleanly.

A missing offer, malformed fixture, invalid amount, unsupported kind or provider failure returns a controlled error and creates no database records.

For adding an offer, the browser submits only the provider code, destination code and offer ID. The server must look up the chosen offer again from the selected provider.

Never trust client-submitted offer descriptions, prices, status or references. This also prevents an offer belonging to Atlas from being silently retrieved or saved through Beacon's adapter.

No mutable global "current provider" variable is needed.

5. Database migration decision

No migration recommended

The existing `trip_items` table at `db/structure.sql:91-104` can store the selected mock option adequately for V1.

TripItem field	Proposed saved value

`name`	Provider-prefixed offer name
`kind`	Normalized valid activity/stay type
`status`	`idea`
`address`	Sample location
`notes`	Provider display name, sample offer ID, description and explicit not-booked notice
`amount` , `currency`	Sample estimated amount, if provided
`starts_at` , `ends_at`	`nil` unless a user supplies real dates later
`confirmation_ref`	`nil`
`inbox_item_id`	`nil`
`added_by`	Current authenticated user

The name prefix will distinguish otherwise identical offer names from different demo providers.

No `Expense` should be created. Clicking **Add to itinerary** must not change the item to `confirmed` .

This is sufficient to demonstrate normalization, provider selection and itinerary integration without altering the database.

Limitation: Provider metadata in the name/notes is not a durable relational booking identity. Real booking synchronization or searching by provider IDs would justify dedicated `provider_code` and `provider_booking_id` fields in a future migration. That functionality is outside Scenario 2 V1.

6. Existing production files to modify

Only a small number of existing application files need modification under this design.

File	Current responsibility	Proposed change and reason
`config/routes.rb:21–48`	Trip and nested resource routes	Add nested `booking_options` index/create routes

`app/views/trips/_trip_panel.html.erb:195–255`	Trip management actions	Add owner-only **Browse sample offers** entry point
`README.md`	Setup, usage and test instructions	Document the two local providers, demo limitations and test commands

No change is planned to the following existing production files:

- `InboxItem`, `InboxItemsController`, `TripMailbox`, `ParseInboxItemJob`
- `Trip`, `TripMember`, `TripItem`, `TripItemsController`
- `ItinerarySuggestionGenerator`
- `Geocodable`, `GeocodeTripItemsJob`
- `Expense` or the database schema

The new controller should create a normal TripItem using the existing model, rather than forcing the single-offer flow through Scenario 1's private `bulk_create_items` method.

7. New files to introduce

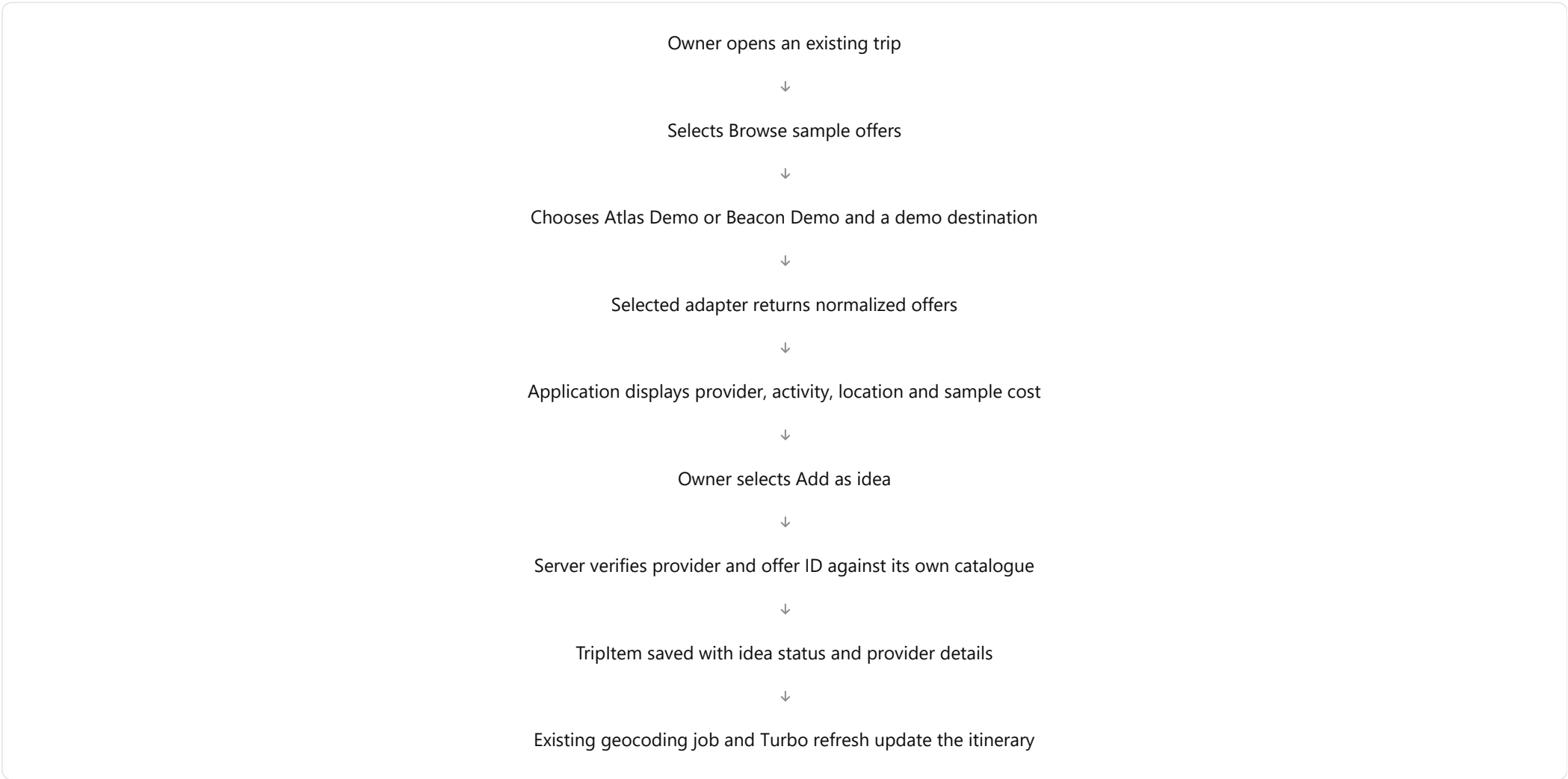
Proposed new path	Responsibility
`app/controllers/booking_options_controller.rb`	Provider selection, authorization and saving a selected offer
`app/views/booking_options/index.html.erb`	Provider selector, destination selector and offer cards
`app/services/booking_providers/registry.rb`	Fixed whitelist and provider lookup
`app/services/booking_providers/atlas_adapter.rb`	Normalize Atlas fixture data
`app/services/booking_providers/beacon_adapter.rb`	Normalize Beacon fixture data
`config/booking_provider_fixtures/atlas_demo.json`	Sample Atlas catalogue
`config/booking_provider_fixtures/beacon_demo.json`	Sample Beacon catalogue
`test/services/booking_providers_test.rb`	Provider interface, normalization and error cases

`test/controllers/booking_options_controller_test.rb`	Authorization, selection, persistence and tamper resistance

These names are planned filenames, not files already created.

No new job, model, migration or JavaScript controller is required.

8. Proposed user-visible workflow



The page should clearly say:

> Demonstration offers only. Adding an offer to your itinerary does not make a reservation or process a payment.

Use `GET` to browse and `POST` to save. Proposed route declaration:

```ruby
resources :booking_options, only: [:index, :create]
```

It will be nested inside the existing `resources :trips` block.

Authorization should be owner-only for V1, using the existing `current_member_for(@trip)&.owner?` approach. This preserves Scenario 1's owner-only generation policy without changing membership management.

Newly created provider ideas can use `defer_geocoding = true`, followed by `GeocodeTripItemsJob.perform_later`, just like Scenario 1's safe batch creation. The existing callback and job source do not need modification.

9. Acceptance criteria

ID	Acceptance criterion	Expected behavior
AC01	Both demo providers are available	Two selectable providers displayed
AC02	Atlas selected	Atlas offers returned using the normalized shape
AC03	Beacon selected	Beacon offers returned in the same normalized shape
AC04	Provider changed	Results switch to the selected provider without retaining the previous provider's offers
AC05	Unknown provider code	Controlled validation error; no save
AC06	Invalid or unknown offer ID	Rejected without creating TripItem
AC07	Wrong provider/offer combination	Rejected without cross-provider fallback

AC08	One provider returns malformed data	Controlled error; no malformed offer displayed or saved
AC09	Provider raises an application error	Other provider remains independently usable
AC10	Valid offer selected	One TripItem added with `status = idea`
AC11	Offer selection repeated	Avoid a duplicate matching provider-prefixed offer within the trip
AC12	Offer lacks coordinates	Item can be saved and geocoded through existing job
AC13	Provider offer added	No InboxItem or Expense created; no false confirmation reference
AC14	Unauthenticated or non-owner user attempts selection/persistence	Operation denied
AC15	Existing email confirmation processed	Original inbox/Anthropic linking behavior preserved
AC16	Scenario 1 generation and JSON import used	Existing workflow remains operational
AC17	Automated provider tests run	No live provider API requests
AC18	Malicious/tampered price, name, status or destination submitted	Server ignores client-supplied offer attributes and resolves approved offer data

A database uniqueness guarantee is not available without additional schema support. V1 can implement application-level duplicate checks, with concurrent duplicate submissions documented as a remaining limitation.

10. Test plan

Planned Scenario 2 test checklist

0/23

Planning tracker — no tests executed in this step

Provider unit tests

- [ ] Atlas fixture normalizes to the approved offer structure
- [ ] Beacon fixture normalizes to the same structure

- [ ] Different source fields are mapped correctly
- [ ] Unknown provider and invalid offer are rejected
- [ ] Malformed or empty provider data is handled safely
- [ ] Providers do not share mutable selection state

Controller and persistence tests

- [ ] Owner can view and choose a provider
- [ ] Non-owner and unrelated user cannot save offers
- [ ] Saved offer uses status idea and the correct user/trip
- [ ] Tampered offer fields and cross-provider IDs are rejected
- [ ] No InboxItem, Expense or confirmation reference is created
- [ ] Duplicate selection is handled and geocoding is queued

Regression tests

- [ ] Original confirmation-email pipeline still works
- [ ] InboxItem linking and confirmed booking handling remain unchanged
- [ ] Manual TripItem creation and JSON import work
- [ ] Scenario 1 generation, preview and confirmation work
- [ ] Instance-scoped geocoding deferral remains intact
- [ ] Existing trip-scoped Turbo refresh remains intact

Manual demonstration, if Rails can boot

- [ ] Switch between Atlas Demo and Beacon Demo
- [ ] Add a sample option as an idea
- [ ] View provider details on its TripItem
- [ ] Verify normal timeline/map behavior
- [ ] Inspect the database to confirm no fabricated booking or expense

Reset checklist

Unit tests should use the local JSON fixtures. Controller tests should stub provider selection and geocoding jobs where appropriate. They should not call Anthropic or live Geocoder services.

Scenario 1's Rails integration suite remains blocked by the previously identified Ruby/dependency mismatch. This Scenario 2 design does not change that status or claim any unexecuted tests as passed.

11. Regression risks and safeguards

Risk	Planned safeguard
Provider implementation leaks state into another selection	Stateless adapters; each request uses explicit provider/destination arguments
Provider code is tampered with	Fixed whitelist in registry
Offer attributes are tampered with	Server-side offer lookup on POST
Demo option is mistaken for a real booking	Explicit sample labeling, `idea` status, no confirmation reference
Duplicate records created	Provider-prefixed name matching and tests
Provider item accidentally linked to a later booking email	Avoid using a bare provider name as the demo item name; add matching regression test
Manual saves unexpectedly geocode synchronously	Use per-instance deferral for new provider-created items
Scenario 1 implementation disturbed	Leave generator and import code unchanged
Live geocoding causes unstable tests	Stub Geocoder and assert job enqueue
Provider fixtures are unavailable or malformed	Controlled provider error; no partial persistence

The most important separation is between **a prospective offer saved as an idea** and **a genuine booking confirmation extracted from email**. These are not interchangeable business states.

12. Tentative design-pattern candidates

No new pattern is confirmed in this step.

Candidate A — Adapter (GoF)

Role	Proposed participant
Client	`BookingOptionsController#index / #create`
Target interface	Shared `#offers_for(destination_code:)` contract
Concrete adapters	`BookingProviders::AtlasAdapter`, `BookingProviders::BeaconAdapter`
Adapted inputs	Atlas and Beacon provider-specific fixture schemas

Why it might qualify: The two implementations translate independently shaped provider data into one interface expected by the application.

Why it might be a false positive: Mapping two local JSON fixtures may be ordinary data normalization rather than adapting independently implemented services or APIs. The existence of files named `Adapter` is not proof of GoF Adapter.

If confirmed later, the two implementations would constitute **one Adapter collaboration**, not two instances.

Candidate B — Controller (GRASP)

Proposed instance: Multiple Booking Provider Option Coordination.

- System event: User requests offers from a provider and selects one.
- Coordinator: `BookingOptionsController`.
- Collaborators: Provider registry, selected provider adapter, Trip, TripMember, TripItem and geocoding job.
- Outcome: A validated, selected provider offer becomes an itinerary idea.

This is a different user operation from P05's external JSON bulk import or S1-N01's Anthropic personalized generation.

It could qualify as a new Controller collaboration if the implemented behavior remains independently coordinated and source-supported.

It should not be counted merely because a new Rails controller class was created.

Candidate C — Strategy (GoF), weaker

The providers share an interchangeable calling contract. However, that fact alone does not establish Strategy.

Strategy would become more defensible if the implementations encapsulated meaningfully different provider-selection or offer-retrieval algorithms selected by a common context.

If they mainly translate different fixture schemas, **Adapter is the stronger candidate**. Do not automatically count both Strategy and Adapter for the same responsibility.

Factory assessment

`BookingProviders::Registry.find(code)` is expected to be a simple whitelist-based lookup. There is no present justification to call it Factory Method or Abstract Factory.

13. Expected effects on current confirmed patterns

These are impact forecasts only, not evolution-status classifications.

Current ID	Pattern collaboration	Expected impact
P01	Parsed Email to Trip Record Linking	Possibly affected — provider-created ideas may interact with name-based email matching
P02	Inbound Travel Email Intake Coordination	Likely unchanged
P03	Trip Creation Coordination	Likely unchanged

P04	Trip Membership Management Coordination	Likely unchanged
P05	Bulk Itinerary Import Coordination	Likely unchanged
P06	OAuth Sign-in and Identity Linking Coordination	Likely unchanged
P07	Account Traveler Enrollment Coordination	Likely unchanged
P08	Outbound Mail Delivery Logging	Likely unchanged
P09	Trip-Scoped Turbo Refresh Notifications	Possibly affected — existing job/refresh mechanism reused
S1-N01	Personalized Itinerary Generation Coordination	Likely unchanged

There are no expected direct modifications to the implementation of any existing confirmed collaboration. Potential effects on P01 and P09 will need verification against actual interactions after coding.

Pattern counts stay frozen at the Scenario 1 state for now:

Pattern	Current confirmed count
Controller (GRASP)	7
Information Expert (GRASP)	1
Observer (GoF)	2
Total instances	**10**
Distinct types	**3**

14. Implementation sequence for the next coding prompt

Recommended new source-version name:

```
scenario2_multiple_booking_providers
```

The next implementation should start from a copy of `scenario1_personalized_itinerary`, not from the original DaPlanStan source.

1 **Freeze the Scenario 1 input**

Scenario 1 AFTER ZIP

Record its checksum and copy the complete source into the Scenario 2 working directory. Preserve Scenario 1 untouched.

2 **Establish provider fixtures**

config/booking_provider_fixtures/*.json

Create two fixed local provider response samples with different schemas and stable offer IDs.

3 **Implement normalization**

app/services/booking_providers/atlas_adapter.rb and beacon_adapter.rb

Convert both sources into the same validated normalized offer structure.

4 **Implement provider lookup**

app/services/booking_providers/registry.rb

Define a strict provider whitelist and controlled errors for missing providers or invalid offers.

5 **Add provider routes**

config/routes.rb

Add nested booking_options index/create routes without altering current trip_item, inbox or generation routes.

6 **Implement user coordination**

app/controllers/booking_options_controller.rb

Add trip-scoped owner authorization, provider selection, server-side offer validation, idea persistence and job enqueue.

7 **Create the provider UI**

app/views/booking_options/index.html.erb

Display provider and destination selectors, mock offer cards, sample pricing and Add as idea actions.

8 **Add the trip entry point**

app/views/trips/_trip_panel.html.erb

Add Browse sample offers alongside existing trip actions, using the current styling.

Implement verification tests

9 test/services/booking_providers_test.rb; test/controllers/booking_options_controller_test.rb

Test both provider formats, invalid selections, authorization, tampering, persistence and isolation without external calls.

10 **Run regressions and prepare evidence**

README.md; test logs; Scenario 2 change record

Attempt actual Rails tests, record environment blockers honestly, compare the source and preserve the AFTER diff for pattern verification.

Final Scenario 2 design decision

Design question	Decision
Starting codebase	`scenario1_personalized_itinerary`
AFTER version name	`scenario2_multiple_booking_providers`
Core feature	Browse locally simulated provider offers and add one as an itinerary idea
Providers	Atlas Demo and Beacon Demo
Technical approach	Lightweight registry with two provider-specific normalizers
External booking API dependency	None
Provider response format	Normalized hashes
Persistence	Existing `TripItem`
Status of selected option	`idea`
Real reservation created	No
Payment or Expense created	No
Database migration	Not required for V1
New background job	None
Existing email workflow	Retained unchanged
Scenario 1 generation	Retained unchanged

Tentative new patterns	Adapter (GoF) and Controller (GRASP)
Pattern classifications/counts	Unchanged until AFTER verification
Code modifications in this step	None

Scenario 2 design is complete. The recommended scope is small enough for a coursework demonstration while giving the next verification stage real provider-selection, normalization and coordination behavior to inspect.

The remaining decision is not which textbook pattern to introduce—it is whether the actual implemented provider classes demonstrate a recognized collaboration once the code and tests exist.

Pasted text(20261011-072120).txt
Document

Worked for 9m 22s

DaPlanStan — Scenario 2 Implementation Complete

AFTER source created and packaged

Scenario 2 — Multiple Booking Providers has been implemented in a separate Rails source copy.

The implementation uses **Atlas Demo** and **Beacon Demo**, two fictional providers with different local data formats. Users can compare sample offers and save a selected offer as a `TripItem` with status `idea`.

No real reservations, payments, expenses or confirmation references are created.

1. Download the completed files

Scenario 2 AFTER Source
Complete Rails project · ZIP · 23 MB

Download Scenario 2 Complete Source ↗

Scenario 1 → Scenario 2 Source Patch
Unified patch · 53 KB · Patch dry-run successful

Download Source Diff ↗

The ZIP passed archive integrity verification and contains `SCENARIO2_CHANGE_RECORD.md`, the implementation, test files and execution evidence.

2. Implementation summary

Requirement	Result
Start from Scenario 1 AFTER	✓
Separate Scenario 2 version	✓
Atlas and Beacon fixtures	✓
Provider registry and normalization	✓
Owner-only offer browsing	✓
Server-side offer validation	✓
Save selected offer as `idea`	✓
Provider-specific duplicate detection	✓
Reuse existing geocoding	✓
Automated test files added	✓
Full Rails integration tests	ⓘ Blocked by environment

3. Exact source changes

The file comparison against the Scenario 1 ZIP confirmed:

3	17	0
Modified files	New files	Deleted files

Modified existing files

File	AFTER lines	Change
`config/routes.rb`	42	Added nested booking-option routes
`app/views/trips/_trip_panel.html.erb`	246–253	Added owner-only Offers link
`README.md`	155–201	Added Scenario 2 usage, limitations and test instructions

The route added is:

```ruby
resources :booking_options, only: [ :index, :create ]
```

New production files

File	AFTER lines	Purpose
`app/controllers/booking_options_controller.rb`	1–77	Browse and save selected offers
`app/services/booking_providers/registry.rb`	1–68	Whitelist and normalized-offer validation
`app/services/booking_providers/atlas_adapter.rb`	1–37	Atlas format normalization
`app/services/booking_providers/beacon_adapter.rb`	1–35	Beacon format normalization
`app/views/booking_options/index.html.erb`	1–49	Selection and offer interface
`config/booking_provider_fixtures/atlas_demo.json`	Entire file	Fictional Atlas offers
`config/booking_provider_fixtures/beacon_demo.json`	Entire file	Fictional Beacon offers

The other ten new files are three Rails test files, five evidence files, a standalone validation test script and `SCENARIO2_CHANGE_RECORD.md`.

4. Final provider collaboration



5. Atlas and Beacon normalization

Both providers have different source representations but return the same nine-field offer contract.

Information	Atlas source	Beacon source
Collection	`destinations → offers`	`results`
Offer ID	`id`	`code`
Name	`title`	`label`
Category	`category`	`type`
Location	`location`	`address`
Description	`details`	`summary`
Sample amount	`price.value`	`amount`
Currency	`price.currency`	`currency`

The normalized structure contains `provider_code`, `provider_name`, `offer_id`, `name`, `kind`, `address`, `description`, `sample_amount` and `currency`.

Both fixtures support Rome and Paris, with five fictional offers per provider.

Invalid kinds, amounts, currencies, offer IDs and malformed fixture structures result in controlled errors.

6. Authorization, tampering and persistence

The controller requires an authenticated user with an owner role for the selected trip.

On POST, it accepts only the three identifying parameters: `provider_code`, `destination_code` and `offer_id`.

It retrieves the selected offer again from the server-owned fixture. Browser-supplied prices, names, kinds, status values and confirmation references are ignored.

The saved item has:

Field	Value

`status`	`idea`
`added_by`	Current authenticated user
`name`	Provider-prefixed offer name
`notes`	Provider identity, offer ID, description and demo disclaimer
`amount / currency`	Validated fictional estimate
`starts_at / ends_at`	`nil`
`confirmation_ref`	`nil`
`inbox_item_id`	`nil`

A controlled marker, such as `[demo_provider_offer:atlas_demo:atlas-rome-walk-01]`, is placed in the notes. It is used to identify an already-added idea from the same provider and offer ID.

No Expense or InboxItem is created.

7. Geocoding reuse

The new controller sets `defer_geocoding = true` on the specific new TripItem before saving it.

After saving, it calls the existing `GeocodeTripItemsJob.perform_later`.

The existing job then loads a fresh record and performs geocoding normally, followed by the existing Turbo refresh.

No global callback manipulation was introduced, and the existing `TripItem`, `Geocodable` and geocoding job source files remain unchanged.

8. Tests added and actual results

New Rails tests

File	Coverage

`test/services/booking_providers_test.rb`	Fixture formats, normalized output, invalid data, registry, isolation
`test/controllers/booking_options_controller_test.rb`	Authorization, offer selection, tampering, duplicates, persistence
`test/models/scenario2_email_and_refresh_regression_test.rb`	Genuine confirmation-email linking and existing geocoding/Turbo path

Verification executed

Standalone checks

10/10

67 assertions · 0 failures/errors/skips

Ruby syntax checks

9/9

All Syntax OK

Full Rails verification remains blocked.

Detail	Actual result
Command	`bin/rails test`
Exit code	1
Available Ruby	3.3.8
Required Ruby	3.4.2
Primary error	`Bundler::GemNotFound`
Missing dependency	Rails 8.1.3 and other locked gems
Rails tests executed	0
Rails passes/failures/errors/skips	Not available

The provider validation checks were executed using standalone Ruby, not Rails. They do not establish that database persistence, authorization or rendering has passed integration tests.

No live booking-provider requests were made. Live Anthropic and geocoding functionality were not retested.

9. Regression status

The source comparison confirms that the following original areas were not modified:

- Scenario 1 personalized generation and preview/confirmation.
- Existing external JSON import and `bulk_create_items`.
- Manual TripItem creation.
- Instance-scoped callback deferral.
- Existing geocoding and Turbo broadcasting.
- TripMailbox, ParseInboxItemJob, InboxItem and InboxItemsController.
- Authentication, membership and Expense models.
- Database schema and migrations.

The existing Scenario 1 tests remain unchanged, and additional email-linking and refresh regression tests were written.

Regression preservation is established at the source-diff level, not yet by a successful Rails test run.

10. Implementation differences and remaining limitations

The feature follows the approved design. The principal implementation detail added is a controlled provider-and-offer marker in `TripItem.notes` to support deduplication without a migration.

Remaining limitations include sequential rather than concurrency-safe duplicate prevention, provider metadata being stored as text rather than relational fields, fixed demo destinations and invented costs, and the unavailable Rails integration environment.

The Source ZIP contains the complete change record and logs needed to continue verification in a compatible Rails environment.

11. Tentative pattern candidates for the next step

No design pattern has been newly confirmed or counted.

Candidate	Source collaboration to inspect
Adapter (GoF)	Registry, AtlasAdapter, BeaconAdapter and their normalized interface

Controller (GRASP)	BookingOptionsController coordinating the provider browsing and offer-addition system events
Strategy (GoF)	Consider only if the implemented behaviors demonstrate genuine interchangeable algorithms, not merely different fixture mappings

There is no present justification to count the registry as a Factory simply because it selects a provider.

The previously confirmed pattern counts remain:

Pattern	Count
Controller (GRASP)	7
Information Expert (GRASP)	1
Observer (GoF)	2
Total	**10**
Distinct types	**3**

12. Final handoff

Scenario 2 now has a complete separate AFTER source snapshot and an applicable source patch. The patch also passed a dry-run against the Scenario 1 input source.

The next step is Scenario 2 AFTER source and pattern-evolution verification, including determining whether the Adapter and Controller candidates constitute distinct confirmed collaborations.

The team should also complete the blocked Rails tests using Ruby 3.4.2 and the locked dependencies before claiming end-to-end runtime verification.

Scenario 3 has not been started.

Scenario 2 — Multiple Booking Providers has now been implemented in:

scenario2_multiple_booking_providers

Do not start Scenario 3 yet.

The Scenario 1 verified state is frozen:

P01 Parsed Email to Trip Record Linking — Information Expert (GRASP)
P02 Inbound Travel Email Intake Coordination — Controller (GRASP)
P03 Trip Creation Coordination — Controller (GRASP)
P04 Trip Membership Management Coordination — Controller (GRASP)
P05 Bulk Itinerary Import Coordination — Controller (GRASP)
P06 OAuth Sign-in and Identity Linking Coordination — Controller (GRASP)
P07 Account Traveler Enrollment Coordination — Controller (GRASP)
P08 Outbound Mail Delivery Logging — Observer (GoF)
P09 Trip-Scoped Turbo Refresh Notifications — Observer (GoF)
S1-N01 Personalized Itinerary Generation Coordination — Controller (GRASP)

Scenario 1 counts:
Controller = 7
Information Expert = 1
Observer = 2
Total instances = 10
Distinct pattern types = 3

Now independently verify Scenario 2 using the actual Scenario 2 AFTER source ZIP and Scenario 1 → Scenario 2 patch.

Do not rely only on the implementation summary.

1. Verify the actual source changes.

Inspect at minimum:

- config/routes.rb

- app/controllers/booking_options_controller.rb
- app/services/booking_providers/registry.rb
- app/services/booking_providers/atlas_adapter.rb
- app/services/booking_providers/beacon_adapter.rb
- config/booking_provider_fixtures/atlas_demo.json
- config/booking_provider_fixtures/beacon_demo.json
- app/views/booking_options/index.html.erb
- app/views/trips/_trip_panel.html.erb
- app/models/trip_item.rb
- app/models/concerns/geocodable.rb
- app/jobs/geocode_trip_items_job.rb
- Scenario 1 generation/import source
- InboxItem / InboxItemsController / TripMailbox / ParseInboxItemJob
- tests
- SCENARIO2_CHANGE_RECORD.md
- source patch

Record reliable AFTER file and line ranges.

2. Verify the implementation claims.

Check that:

- Atlas and Beacon really use different raw formats
- both normalize to one common output shape
- Registry accepts only approved provider codes
- no arbitrary constantize/dynamic class loading is used
- BookingOptionsController re-fetches trusted provider data on POST
- client-submitted price/name/kind/status/reference data cannot control persistence
- selected offers save as TripItem status idea
- confirmation_ref remains nil

- no Expense is created
- no InboxItem is created
- geocoding uses existing instance-scoped deferral and existing job
- no migration was introduced
- Scenario 1 generation/import remains intact

If any implementation defect is found, stop and report it rather than silently correcting it.

3. Reconstruct the real Scenario 2 collaboration.

Trace:

trip owner
→ booking option route
→ BookingOptionsController#index
→ BookingProviders::Registry
→ selected provider implementation
→ provider-specific fixture
→ normalized offers
→ UI
→ create action
→ provider lookup again
→ trusted offer resolution
→ TripItem idea
→ GeocodeTripItemsJob
→ Turbo refresh/timeline/map

Distinguish application participants from Rails/framework mechanisms.

4. Evaluate the Adapter candidate carefully.

Candidate:

Adapter (GoF) — Multiple Booking Provider Normalization

Possible participants:
Client = BookingOptionsController
Target = common offers_for(destination_code:) / normalized offer contract
Adapters = AtlasAdapter and BeaconAdapter
Adaptees = the different provider-specific data formats

Determine whether this is truly GoF Adapter or merely ordinary data mapping.

Do not confirm it because the files are named Adapter.

Check:

- whether the application expects one stable interface
- whether each provider exposes a structurally different source format
- whether the provider classes translate those formats into the expected target representation
- whether the client is insulated from source-specific details

Final decision must be exactly one:

CONFIRMED
or
REJECTED

If confirmed:
- count the Atlas + Beacon implementations together as ONE Adapter collaboration
- identify all participant roles and source locations

5. Evaluate the new GRASP Controller candidate.

Candidate:

Multiple Booking Provider Option Coordination

Check whether BookingOptionsController coordinates a distinct system event/use case separate from:

- P05 Bulk Itinerary Import Coordination
- S1-N01 Personalized Itinerary Generation Coordination

Compare:

- system event
- responsibility
- coordinator
- collaborators
- inputs
- persistence behavior
- outcome

Final decision:

CONFIRMED NEW INSTANCE
or
REJECTED AS SEPARATE INSTANCE

Do not count a controller merely because it is a Rails controller.

6. Evaluate Strategy separately.

Do not double-count the same provider collaboration.

Determine whether AtlasAdapter and BeaconAdapter encapsulate genuinely interchangeable algorithms/behaviors, or whether they simply adapt different input schemas.

If Adapter better explains the implementation, reject Strategy with a concise reason.

7. Evaluate Factory.

Inspect BookingProviders::Registry.find.

Do not classify it as Factory Method or Abstract Factory unless the actual source structure supports the recognized pattern.

A simple whitelist/lookup is not enough by itself.

Give a final reject/confirm decision.

8. Evaluate impacts on all existing instances:

P01–P09 and S1-N01.

For each assign exactly one Scenario 2 evolution status:

preserved
modified
extended
replaced
removed

Do not call an instance modified merely because one of its classes is reused by Scenario 2.

Pay special attention to:

P01 — email-to-trip linking
P09 — Turbo refresh
S1-N01 — personalized generation
P05 — bulk import

9. If any new pattern instance is confirmed, assign:

newly_introduced

Do not modify frozen earlier-version rows.

10. Recalculate Scenario 2 AFTER pattern counts only after classification.

Show:

- Controller instances
- Information Expert instances
- Observer instances
- Adapter instances if confirmed
- any other confirmed pattern types
- total pattern types
- total current instances

Do not count rejected candidates.

11. Verify duplicate-handling behavior.

Inspect the provider/offer marker stored in TripItem.notes.

Determine:

- whether provider + offer ID is stable
- whether it prevents same-offer duplicates in normal sequential use
- whether it can collide across providers
- whether concurrency remains an acknowledged limitation

Treat this as implementation behavior, not a pattern.

12. Verify test evidence.

Separate:

- Ruby syntax checks
- standalone provider validation
- Rails integration tests
- live Anthropic
- live geocoding/Turbo
- live external provider calls

Do not combine them into a generic “tests passed” claim.

Preserve the exact Rails blocker if still present.

13. Verify regression preservation.

Check source-level preservation of:

- Scenario 1 personalized generation
- JSON import

- manual TripItem creation
- instance-scoped geocoding deferral
- email confirmation parsing/linking
- authentication and membership
- Expense behavior
- database schema

14. Prepare Scenario 2 evolution records.

For every current confirmed instance include:

- ID
- stable instance name
- pattern name/category
- Scenario 2 status
- AFTER participants
- AFTER source locations
- concise reason
- change description

Add any confirmed new Scenario 2 instance separately.

15. Do not generate the final cross-scenario patterns_evolution.csv yet.

Instead prepare append-ready Scenario 2 data for later final assembly.

16. Do not start cancellation-policy work.

At the end give me:

1. Source verification result.

2. Verified Scenario 2 workflow.
3. Adapter decision.
4. Controller decision.
5. Strategy decision.
6. Factory decision.
7. Status of P01–P09 and S1-N01.
8. Any newly introduced confirmed instances.
9. Scenario 2 AFTER counts.
10. Duplicate-handling verification.
11. Test evidence separated by what actually ran.
12. Regression verification.
13. Remaining limitations.
14. Append-ready Scenario 2 evolution record.
15. Whether Scenario 2 is sufficiently documented to move to Scenario 3.

Do not modify the source.
Do not start Scenario 3.

Worked for 4m 14s

DaPlanStan — Scenario 2 AFTER Verification

Source verification complete Rails tests blocked

I independently compared the actual Scenario 2 AFTER ZIP with the frozen Scenario 1 source, inspected the source patch, traced the provider collaboration, and re-executed the available standalone checks.

Final source-level finding: Scenario 2 introduces two confirmed pattern instances: one Adapter (GoF) and one Controller (GRASP). All ten existing collaborations remain preserved relative to Scenario 1.

Scenario 1 instances	Scenario 2 AFTER instances
10	12
3 distinct pattern types	4 distinct pattern types

1. Source verification

The comparison produced:

Check	Verified result
Scenario 1 files	412
Scenario 2 files	429
Identical existing files	409
Modified existing files	3
New files	17
Removed files	0
ZIP integrity	Passed
Patch dry-run	Passed
New database migrations	0

The three modified files are `config/routes.rb:42`, `app/views/trips/_trip_panel.html.erb:246–253`, and `README.md:155–201`.

The new controller, provider registry, two normalizers, two local catalogues, UI, test files and verification evidence are present.

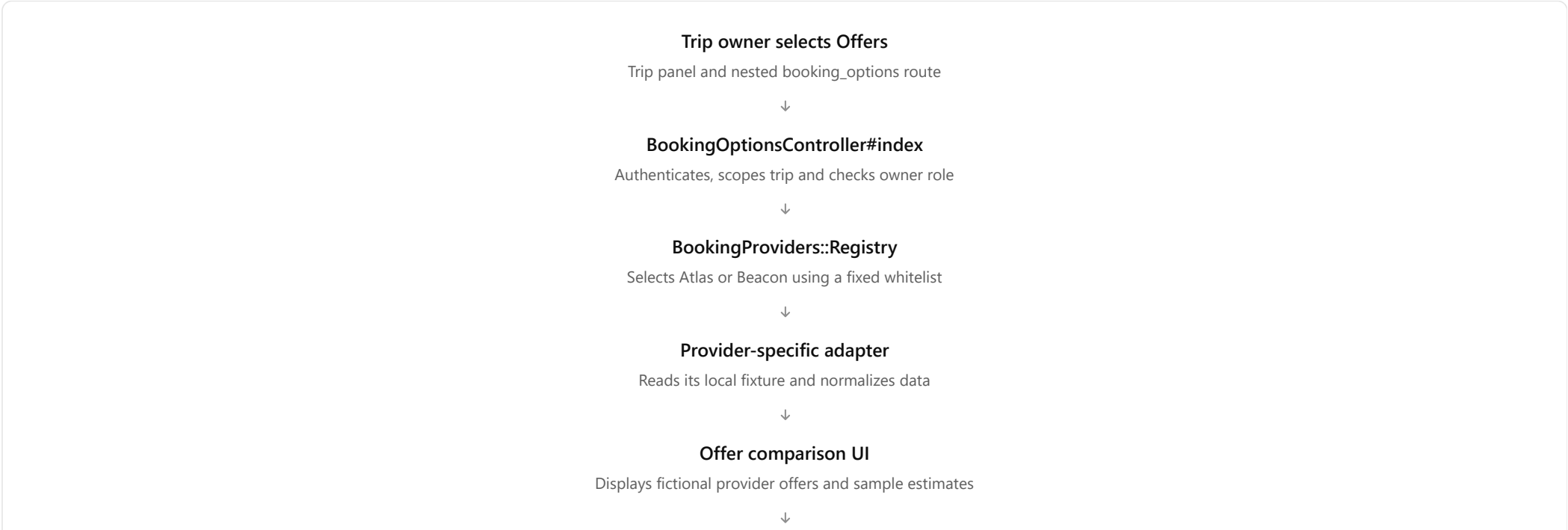
Verified implementation claims

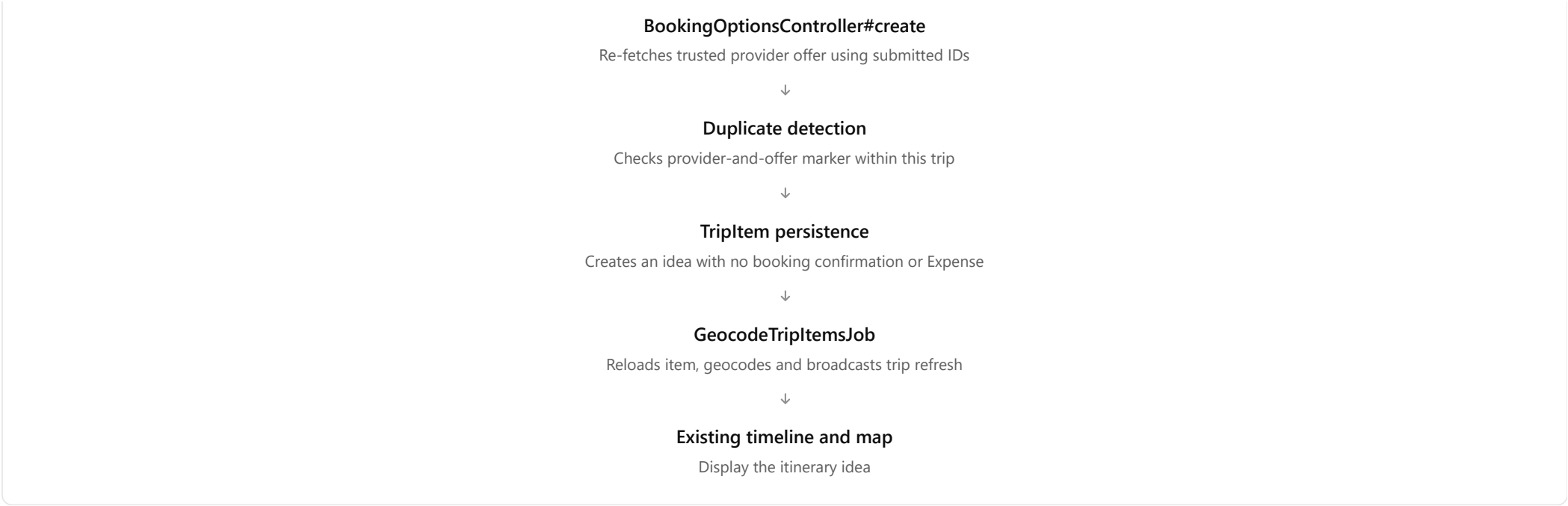
Claim	AFTER evidence	Result
Atlas and Beacon have different raw formats	Both fixture JSON files; adapter `:21–30 / :20–28`	Verified
Both use one normalized output contract	`registry.rb:14,38–49`	Verified
Strict provider whitelist	`registry.rb:11–30`	Verified
No arbitrary `constantize`	`registry.rb:24–30`	Verified

Trusted offer retrieved again on POST	`booking_options_controller.rb:18-26`	Verified
Browser cannot control persisted offer fields	Controller `:19-26,36-44`	Verified
TripItem saved as `idea`	Controller `:36-46`	Verified
Nil confirmation and inbox references	Controller `:43`	Verified
No Expense or InboxItem created by the new flow	Controller `:18-62`	Verified in source
Instance-scoped geocoding and job reuse	Controller `:45-55` ; `trip_item.rb:32-41`	Verified
Scenario 1 code retained	Byte-level comparison	Verified
Runtime behavior	Rails suite did not boot	Not verified

No source correction was made during this step.

2. Verified AFTER workflow





Application participants are `BookingOptionsController`, `BookingProviders::Registry`, `AtlasAdapter`, `BeaconAdapter`, `Trip`, `TripMember`, `TripItem` and `GeocodeTripItemsJob`.

Rails routing, Active Record, Active Job and Turbo are framework mechanisms. They are not additional independently counted pattern instances.

3. Adapter decision

CONFIRMED

S2-N01 — Adapter (GoF) — Multiple Booking Provider Normalization

Adapter role	Verified participant
Client	`BookingOptionsController#index` and `#create`
Target	Common `offers_for(destination_code:)` contract and normalized offer structure
Adapter 1	`BookingProviders::AtlasAdapter`

Adapter 2	`BookingProviders::BeaconAdapter`
Adaptees	Different Atlas and Beacon source-data formats
Selection and validation	`BookingProviders::Registry`

Atlas has nested destination-specific `offers` containing `title`, `category`, `location` and nested price information.

Beacon has a flat `results` collection containing `label`, `type`, `address`, `amount` and `currency`.

The adapters translate these incompatible data contracts into the same application-facing interface. `BookingOptionsController` never needs to inspect either raw schema.

The implementation has no separately declared Target interface or Adaptee Ruby class: the adapted inputs are distinct fixture formats. Its source-level Adapter classification is therefore based on actual data-source interface adaptation, not the names of the Ruby classes.

Count: One Adapter collaboration with two concrete adapters. Not one instance per provider.

4. Controller decision

CONFIRMED NEW INSTANCE

S2-N02 — Controller (GRASP) — Multiple Booking Provider Option Coordination

The new `BookingOptionsController#index` and `#create` coordinate a distinct system event: a trip owner requests provider-specific offers and chooses one to add as an itinerary idea.

Responsibility	P05	S1-N01	S2-N02
Main event	External JSON import	Personalized idea generation	Provider offer selection
Main input	Imported JSON	Preferences	Provider, destination, offer ID
Coordinator	TripItemsController	TripItemsController	BookingOptionsController
Key collaborator	Batch importer	Anthropic generator	Provider registry/adapters
Persistence	Batch ideas	Approved generated ideas	One selected provider idea

Outcome	Imported itinerary items	Personalized itinerary ideas	Provider-specific planning idea

S2-N02 is not a second count of P05 or S1-N01. It receives a different application event and coordinates a different responsibility and collaborator structure.

Count: One newly introduced GRASP Controller.

5. Strategy and Factory decisions

Candidate	Final decision	Reason
Strategy (GoF)	**REJECTED**	Atlas and Beacon primarily translate different data schemas; there is no separate family of interchangeable booking algorithms. Counting Strategy would duplicate the Adapter collaboration.
Factory Method (GoF)	**REJECTED**	`Registry.find` uses a fixed `case` statement and constructor calls. It lacks a polymorphic creator collaboration.
Abstract Factory (GoF)	**REJECTED**	No family of related objects is being created through interchangeable factory interfaces.

The registry is ordinary provider-selection infrastructure rather than an additional confirmed GoF pattern.

6. Status of all existing patterns

Each continuing instance is assessed relative to the Scenario 1 AFTER source, not retroactively against the original baseline.

ID	Stable pattern instance	Scenario 2 status
P01	Parsed Email to Trip Record Linking	`preserved`
P02	Inbound Travel Email Intake Coordination	`preserved`
P03	Trip Creation Coordination	`preserved`
P04	Trip Membership Management Coordination	`preserved`

P05	Bulk Itinerary Import Coordination	`preserved`
P06	OAuth Sign-in and Identity Linking Coordination	`preserved`
P07	Account Traveler Enrollment Coordination	`preserved`
P08	Outbound Mail Delivery Logging	`preserved`
P09	Trip-Scoped Turbo Refresh Notifications	`preserved`
S1-N01	Personalized Itinerary Generation Coordination	`preserved`
S2-N01	Multiple Booking Provider Normalization	`newly_introduced`
S2-N02	Multiple Booking Provider Option Coordination	`newly_introduced`

Why P05 is preserved now: P05 was modified during Scenario 1. Its Scenario 1 AFTER implementation is unchanged by Scenario 2. Therefore its Scenario 2 status is `preserved`, not `modified` again.

Why P09 is preserved: The new flow calls the existing geocoding job, but the Observer collaboration, trip refresh broadcast and subscriptions are unchanged.

Why P01 is preserved: The real email-confirmation linking methods are unchanged. The new demo offer workflow does not alter that collaboration.

Complete participants, source locations, intent/evidence and individual change descriptions for all 12 records are included in the downloadable JSON.

7. Scenario 2 AFTER counts

Confirmed pattern instances by source version

Pattern	Scenario 1	Scenario 2 AFTER
Controller (GRASP)	7	**8**
Information Expert (GRASP)	1	1
Observer (GoF)	2	2
Adapter (GoF)	0	**1**
Total current instances	**10**	**12**
Distinct pattern types	**3**	**4**

These totals exclude the rejected Strategy and Factory candidates.

8. Duplicate-handling verification

The marker constructed by `BookingOptionsController#offer_marker` at lines 74–76 has the form:

Plain text

```
[demo_provider_offer:atlas_demo:atlas-rome-walk-01]
```

The duplicate query at lines 28–34 searches for that marker among `idea` records belonging to the same trip.

Check	Result
Stable provider identity included	Yes
Stable offer ID included	Yes
Same-provider sequential duplicate prevention	Supported
Cross-provider marker collision	Avoided by including provider code
Case-by-case offer selection isolation	Supported
Concurrent duplicate protection	Not guaranteed
Protection if a user edits/removes the marker	Not guaranteed
Protection after an idea changes to confirmed	Not guaranteed by the idea-only query

This is valid application-level deduplication for the demo scope, but not a database uniqueness guarantee.

9. Actual verification results

I independently re-executed the standalone provider checks and Ruby syntax checks.

Standalone provider checks

10/10

67 assertions · 0 failures/errors/skips

Ruby syntax checks

9/9

All reported Syntax OK

Verification type	Actual status
Ruby syntax	Passed
Standalone provider validation	Passed
Rails integration tests	Blocked before test discovery
Live Anthropic	Not run
Live geocoding/Turbo	Not run

Live external booking-provider calls	None by design

The extracted archive's `bin/rails` initially lacked executable permission, producing shell exit code 126. Its ZIP metadata retains the executable permission, so I repeated the attempt using `ruby bin/rails test`.

That command exited with code 1:

```
Bundler::GemNotFound:
Could not find rails-8.1.3 ...
```

The environment uses Ruby 3.3.8 while the project specifies Ruby 3.4.2. Required Rails dependencies are missing.

Rails tests executed: 0. Rails pass, failure, error and skip counts are unavailable.

10. Regression preservation

The source comparison confirms the following remain byte-identical to Scenario 1:

Existing functionality	Source-level result
Personalized itinerary generation	Preserved
Generated preview and confirmation	Preserved
External JSON import	Preserved
Manual TripItem creation	Preserved
Instance-scoped geocoding deferral	Preserved
Geocoding job and Turbo refresh	Preserved
Genuine email parsing and record linking	Preserved

Authentication and membership	Preserved
Expense model and behavior	Preserved
Database schema	Unchanged

The new Rails regression tests were written but could not execute. Therefore these results represent **source preservation**, not completed runtime regression verification.

11. Downloadable verification records

Scenario 2 Independent Verification Report

Source audit, exact line ranges, pattern decisions, execution results and limitations

Download Verification Report ↗

Scenario 2 Append-Ready Evolution Data

12 records with stable IDs, participant roles, locations, evidence, status and change descriptions

Download Evolution Records JSON ↗

These files are separate from the source code. No final cross-scenario `patterns_evolution.csv` has been generated, and no previous-version rows have been changed.

12. Is Scenario 2 sufficiently documented to move to Scenario 3?

Yes for documentation and source-level pattern analysis. Not yet for full runtime verification.

The implementation, source diff, provider normalization, Controller collaboration, pattern classification, counts, duplicate handling and regression-preservation evidence are documented.

The remaining blocker is the Rails environment. Before final submission, the team should execute the complete test suite with the required Ruby and dependencies and record actual integration results.

Final Scenario 2 result: 12 confirmed instances across 4 distinct pattern types.

The next work can move to Scenario 3 design when appropriate, but no cancellation-policy work has been started here.

Pasted text(20261011-074054).txt
Document

Worked for 3m 10s

DaPlanStan — Scenario 3: Different Cancellation Policies

Design-only stage | 11 October 2026

Source reinspection complete

I re-inspected the actual `scenario2_multiple_booking_providers_AFTER.zip`, including the models, controllers, mailbox, Anthropic parser, provider implementations, routes, views, schema, migrations and existing tests.

The extracted source was verified against the ZIP contents. No source files were edited.

Recommended design: Implement three deterministic cancellation-policy calculators that produce hypothetical refund quotes for eligible email-confirmed itinerary items, without changing any booking, itinerary or financial records.

Decision	Recommendation
Starting version	`scenario2_multiple_booking_providers`
Proposed AFTER version	`scenario3_cancellation_policies`
Feature	Local cancellation-policy quote comparison
Policies	Flexible, Moderate, Strict
Architecture	Interchangeable policy objects with one evaluator
Real external cancellation	No
Persist cancellation status	No
Database migration	No
Confirmed pattern counts	Remain 12 instances and 4 types

1. Verified Scenario 3 BEFORE state

Existing cancellation-related behavior

Existing action	Actual behavior	Source evidence
Delete a TripItem	Deletes a local itinerary record	`app/controllers/trip_items_controller.rb:97–100`
Edit a TripItem	Updates itinerary details	`trip_items_controller.rb:89–95`
Confirm/unconfirm	Toggles `idea` and `confirmed`	`trip_items_controller.rb:102–106`
Delete an Expense	Deletes the expense record	`app/controllers/expenses_controller.rb:74–77`
Confirm an Expense	Toggles paid/confirmed timestamps	`expenses_controller.rb:79–85`
Confirm an extracted email	Links or creates TripItems and eligible Expenses	`app/models/inbox_item.rb:27–58,67–207`
Add a demo provider offer	Creates an idea without real reservation data	`app/controllers/booking_options_controller.rb:18–57`

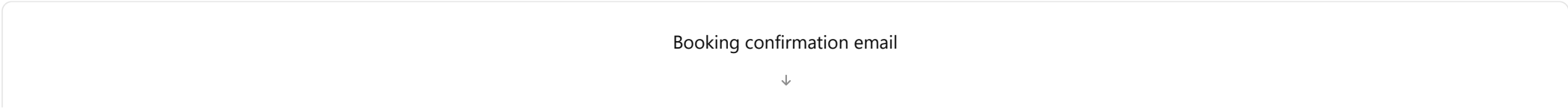
The current `TripItem` status enum contains only `idea` and `confirmed` (`app/models/trip_item.rb:21–24`).

The source search found no application-level cancellation-policy engine, refund calculator, cancellation deadline or booking-cancellation operation.

The Cancel control in `app/views/trip_items/_form.html.erb:149–161` merely exits the editing form.

Deleting a TripItem is not the same as cancelling a booking. In addition, `TripItem` has a `has_one :expense, dependent: :nullify` relationship (`trip_item.rb:7`), so deleting an item can leave its related expense as a separate record.

Genuine email booking workflow



TripMailbox#process

↓

InboxItem created and approved sender checked

↓

ParseInboxItemJob → Anthropic extraction

↓

Structured booking, provider, date, amount and reference

↓

InboxItemsController#update → confirm

↓

InboxItem#confirm_and_create_items!

↓

TripItem status confirmed; Expense created when eligible

Key evidence:

- `app/mailboxes/trip_mailbox.rb:2-31`
- `app/jobs/parse_inbox_item_job.rb:6-18,20-97,99-139`
- `app/controllers/inbox_items_controller.rb:16-35`
- `app/models/inbox_item.rb:79-99,101-185,191-207`

The existing parser can extract confirmation references, dates and prices, but it does **not** extract cancellation terms.

Furthermore, `InboxItem#fill_missing_links!` can match a previously existing item by name without necessarily changing its status. Therefore, merely having an `inbox_item_id` is not sufficient proof that an item represents a confirmed reservation.

Scenario 2 demo offers

`BookingOptionsController#create` explicitly assigns:

Ruby

```ruby
status: :idea,
starts_at: nil,
ends_at: nil,
confirmation_ref: nil,
inbox_item_id: nil
```

See `app/controllers/booking_options_controller.rb:36-46`.

These demo options are not bookings and must remain ineligible for cancellation-policy quotes.

2. Scope and architecture comparison

Approach	Fit and complexity	Database impact	Testability and pattern assessment
A. Hard-coded conditionals	Small but mixes refund rules with controller/model responsibilities	None	Easy initially; no separate Strategy collaboration
B. Separate policy objects	Good separation of distinct refund behaviors	None for quote-only	Highly testable; possible Strategy
C. JSON-only policy definitions	Compact data-driven calculation	None	Deterministic but may simply be table-driven logic
D. Real provider cancellation APIs	Poor fit with current demo and email-only integrations; high complexity	Likely substantial	Credentials and API access required; pattern not guaranteed
E. Small local policy abstraction	**Best fit** for distinct deterministic policy behavior	None	Independent tests; plausible Strategy collaboration

Selected: E — a small local policy abstraction.

Use three concrete policies, one evaluator and one Rails controller. No payment gateway, booking-client API, policy database table or persistent cancellation workflow is necessary.

This is not a claim that a provider will honor the calculated refund. The application must prominently label all results as **hypothetical demonstration estimates**.

3. Eligible TripItem rule

A TripItem may receive a cancellation-policy quote only when all the following conditions hold:

1. `item.confirmed?` is true.
2. `item.inbox_item_id` is present.
3. Its associated InboxItem has `review_status == confirmed`.
4. The item's **own** `confirmation_ref` is nonblank.
5. `starts_at` is present and interpretable.
6. `amount` is present and nonnegative.
7. `currency` is supported.

The evaluator must check these conditions independently of the UI.

This excludes manually created ideas, generated ideas and Scenario 2 provider-demo ideas, even if an ordinary user later toggles their status to `confirmed`.

The requirement for the item's own reference is intentional: `TripItemsController#show` can display a reference inherited from a sibling item (`:16–28`). That display fallback should not establish cancellation-quote eligibility for an item lacking its own booking reference.

Known limitation: Genuine email bookings without a confirmation reference, amount or start time will not be eligible for a quote in V1. They remain visible and otherwise functional.

4. Cancellation outcome: quote-only

✓ **Selected: Calculate and display only**

V1 will not create a `cancelled` status, change booking data, alter an Expense or send a cancellation request.

The existing `TripItem` record stays unchanged after an estimate.

This is appropriate because DaPlanStan does not know the real provider's contractual policy and has no external cancellation integration.

A persistent cancellation state could falsely imply that a reservation was cancelled. It would also require additional model semantics, UI changes, lifecycle rules and testing. Those are unnecessary for this coursework feature.

Proposed AFTER workflow

Trip owner opens eligible confirmed TripItem

↓

Selects Estimate cancellation (demo)

↓

CancellationQuotesController checks access and eligibility

↓

User chooses Flexible, Moderate or Strict

↓

CancellationPolicies::QuoteEvaluator selects the policy

↓

Concrete policy determines applicable fee percentage

↓

Evaluator calculates fee and hypothetical refund

↓

UI displays amounts, currency and explanation

↓

No booking or database record is changed

5. Exact cancellation-policy rules

Define the time difference as:

$$\Delta = \text{item.starts_at} - \text{evaluated_at}$$

Use elapsed seconds to determine each boundary. The calculation should receive an explicit evaluation timestamp, allowing reproducible unit tests without relying on wall-clock time.

All three policies are fictional application policies.

Time until start	Flexible	Moderate	Strict
At least 72 hours	0% fee	0% fee	100% fee
At least 24 but less than 72 hours	0% fee	50% fee	100% fee
More than 0 but less than 24 hours	10% fee	100% fee	100% fee
At or after start	100% fee	100% fee	100% fee

The table deliberately specifies inclusive boundaries:

- At exactly 72 hours, Moderate has no fee.
- At exactly 24 hours, Flexible has no fee and Moderate charges 50%.
- One second before start, Flexible charges 10%.
- At exactly the start timestamp and afterward, every policy charges 100%.

Example: original amount of €200

Illustrative policy quote calculator Demo only

Policy

- ○ Flexible
- ○ Moderate
- ○ Strict

Hours until itinerary item starts

48 hours

Original amount (EUR)

€200

Hypothetical fee

€100.00

Hypothetical refund

€100.00

This interactive example illustrates the proposed rules only. It is not connected to DaPlanStan and does not represent actual provider terms or perform cancellation.

The production implementation must use decimal-safe arithmetic, not JavaScript floating-point calculations.

Required monetary rules

Use `BigDecimal` for amounts and rates.

$$\text{fee} = \text{round}_2(\text{amount} \times \text{fee rate})$$

$$\text{refund} = \text{amount} - \text{fee}$$

Round the fee to two decimal places with a documented rounding mode such as `BigDecimal::ROUND_HALF_UP`. Derive the refund by subtracting the rounded fee so that:

$$\text{fee} + \text{refund} = \text{original amount}$$

For the first version, support EUR, GBP and USD, which fit the application's two-decimal amount representation. Other currencies, including JPY, should receive a clear unsupported-currency message rather than an inaccurately rounded quote.

Edge-case policy

Input	Required handling
Missing amount	Do not calculate; explain missing booking amount
Zero amount	Valid; zero fee and zero refund
Negative amount	Reject
Missing start time	Reject; do not guess from Trip dates
Missing currency	Reject
Unsupported currency	Reject clearly
Unknown policy	Reject before calculation
Item is an idea	Ineligible
Item has no genuine confirmed email link/reference	Ineligible
Cancellation at/after start	Apply after-start tier
Exactly at a deadline	Apply explicitly inclusive boundary
Evaluation time missing from direct service call	Reject; controller supplies `Time.current`
Malformed timestamp	Reject
Valid currency	Preserve it unchanged in the result

`refundable` in the result should mean `refund_amount > 0`. A zero-cost item can have a zero fee without a positive refundable amount.

6. Proposed classes and interfaces

The implementation does not need a large class hierarchy.

New class	Responsibility	Public method

`CancellationPolicies::Flexible`	Determine Flexible policy fee rate	`fee_rate(seconds_until_start:)`
`CancellationPolicies::Moderate`	Determine Moderate policy fee rate	`fee_rate(seconds_until_start:)`
`CancellationPolicies::Strict`	Determine Strict policy fee rate	`fee_rate(seconds_until_start:)`
`CancellationPolicies::QuoteEvaluator`	Validate the booking, select a policy, calculate monetary result	`call(trip_item:, policy_code:, evaluated_at:)`
`CancellationQuotesController`	Authorize and coordinate quote requests	`new`, `create`

The three policies return decimal fee rates between zero and one. They must be stateless and must not know about controllers, the database or external providers.

`QuoteEvaluator` acts as the common context. It uses a fixed policy-code-to-class map, performs eligibility and monetary validation, invokes the selected policy, and returns a consistent result.

A separate registry class or abstract superclass is unnecessary in V1.

Proposed output structure

Ruby

```ruby
{
  policy_code: "moderate",
  policy_name: "Moderate",
  original_amount: BigDecimal("200.00"),
  fee_amount: BigDecimal("100.00"),
  refund_amount: BigDecimal("100.00"),
  currency: "EUR",
  refundable: true,
  explanation: "The selected demo policy applies a 50% fee."
}
```

This is a proposed result contract, not an output currently produced by the application.

The evaluator should raise controlled, distinguishable errors for ineligible bookings, unknown policies, missing data and unsupported amounts/currencies. The controller will display a friendly message without exposing internal exceptions.

7. Database migration decision

No migration required

The current `db/structure.sql:91–104` shows:

Field	Existing type
`status`	String
`starts_at`	Datetime
`amount`	Decimal(10,2)
`currency`	String
`confirmation_ref`	String
`inbox_item_id`	Foreign key
`notes`	Text

Those fields are sufficient to calculate a hypothetical cancellation quote.

The `expenses` table (`db/structure.sql:105–125`) should remain unchanged.

No `cancellation_policy` , `cancelled_at` , `cancellation_fee` or `refund_amount` columns are necessary because no cancellation or quote is persisted.

The user selects the demonstration policy each time. The selection must not be treated as the real contractual policy for that booking.

8. Existing production files to modify

Existing file	Current responsibility	Proposed modification
`config/routes.rb:21-49`	Nested trip and item routes	Add nested singular `cancellation_quote` new/create routes
`app/views/trip_items/show.html.erb:127-257`	Itinerary item details and actions	Add owner-only **Estimate cancellation (demo)** link for qualifying items
`README.md`	Application and coursework instructions	Explain policies, quote-only limitation, tests and behavior

Suggested route within `resources :trip_items`:

‹/› Ruby

```ruby
resource :cancellation_quote, only: [ :new, :create ]
```

This uses Rails' existing nested resource conventions without changing JSON import, generation, item editing or provider routes.

Existing files deliberately left unchanged

`TripItem`, `TripItemsController`, `InboxItem`, `InboxItemsController`, `TripMailbox`, `ParseInboxItemJob`, `Expense`, `ExpensesController`, `BookingOptionsController`, `ItinerarySuggestionGenerator`, `Geocodable` and `GeocodeTripItemsJob` should not need source changes for this design.

The quote evaluator may read their data and relationships, but no existing business responsibility needs to be moved or rewritten.

9. New files to add

Proposed path	Purpose
`app/controllers/cancellation_quotes_controller.rb`	Owner authorization and quote request coordination

`app/services/cancellation_policies/flexible.rb`	Flexible fee-rate algorithm
`app/services/cancellation_policies/moderate.rb`	Moderate fee-rate algorithm
`app/services/cancellation_policies/strict.rb`	Strict fee-rate algorithm
`app/services/cancellation_policies/quote_evaluator.rb`	Eligibility, selection, calculation and standardized quote
`app/views/cancellation_quotes/new.html.erb`	Policy selector, quote result, error and back link
`test/services/cancellation_policies_test.rb`	Policy boundaries, arithmetic and validation
`test/controllers/cancellation_quotes_controller_test.rb`	Authorization, eligibility, rendering and read-only behavior

No new job, migration, payment integration or booking-provider client is required.

Authorization

Use the source's existing mechanisms:

```ruby
@trip = current_user.trips.find(params[:trip_id])
@item = @trip.trip_items.find(params[:trip_item_id])

head :forbidden unless current_member_for(@trip)&.owner?
```

For V1:

- Trip owners may request quotes.
- Planners, contributors and viewers cannot request them.
- Users who do not belong to the trip cannot access its items.
- Every GET/POST must enforce authorization server-side.
- The frontend link's visibility is not an authorization control.

The `new` action displays eligible booking details and a policy selector; `create` evaluates the selected policy and renders the result without writing to the database.

10. Acceptance criteria

ID	Acceptance criterion	Expected result
AC01	Eligible confirmed booking opened by owner	Quote interface available
AC02	Flexible selected	Correct Flexible calculation
AC03	Moderate selected	Correct Moderate calculation
AC04	Strict selected	Correct Strict calculation
AC05	Different policies receive same inputs	Same evaluation contract, different applicable outcomes
AC06	Unknown policy submitted	Controlled validation error
AC07	Exactly 72 hours before start	Moderate returns full refund
AC08	Exactly 24 hours before start	Flexible returns full refund; Moderate returns 50%
AC09	Start time reached or passed	After-start rule applies
AC10	Missing amount or start	Quote denied with reason
AC11	Negative amount or unsupported currency	Calculation rejected
AC12	Zero amount	Zero fee and refund
AC13	Scenario 1 generated idea selected	Ineligible
AC14	Scenario 2 demo offer selected	Ineligible
AC15	Genuine confirmed email-linked item with complete data	Quote produced
AC16	Non-owner attempts a quote	Access denied
AC17	Quote calculated	No TripItem, InboxItem or Expense changes

AC18	Quote calculated	No external cancellation API call
AC19	Existing generation, import and provider workflows used	Existing behavior preserved
AC20	Quote shown to user	Explicit hypothetical-only disclaimer

The UI should display:

> This is a hypothetical cancellation estimate based on a demonstration policy. No reservation has been cancelled and no actual refund is guaranteed. Check your booking provider's terms.

11. Test plan

Policy unit tests

Verify all three algorithms using fixed timestamps, especially exactly 72 hours, exactly 24 hours, one second before each deadline, at start and after start. Verify zero/negative amounts, decimal rounding, currency preservation and unknown policies.

Evaluator and controller tests

Verify policy selection, normalized quote fields, missing booking data, eligible inbox-linked bookings, owner authorization, rejection of unrelated users, and no persistence on GET or POST.

Regression tests

Retain Scenario 1 generation, preview, JSON import and manual creation coverage. Retain Scenario 2 provider-normalization, demo idea and duplicate-protection coverage. Check email-confirmation linking, expense creation and the existing geocoding/Turbo workflows.

Manual/runtime checks

With a compatible Rails environment, open an eligible confirmed item, select all three policies, inspect the quotes, and confirm that database records remain unchanged. Verify that demo ideas do not offer the quote action.

All calculation tests must pass `evaluated_at` explicitly. Do not introduce external API requests into policy tests.

The previously identified Rails environment blocker still needs resolution before full integration testing. No Scenario 3 tests have been executed during this design step.

12. Regression risks

Risk	Safeguard
Confirmed itinerary idea mistaken for a booking	Require confirmed InboxItem, item confirmation reference and `confirmed` status
Real policy implied without evidence	Label every policy and quote as hypothetical
Client manipulates booking amount	Evaluator reads amount from the server-loaded TripItem
Incorrect deadline handling	Compare timestamps in seconds; test exact boundaries
Money calculation inaccuracies	BigDecimal and explicit two-decimal rounding
Existing expense accidentally adjusted	Never write to Expense
Real booking marked cancelled	Quote-only; no cancelled status
Scenario 2 demo offer considered cancellable	Require confirmed email linkage, which demo offers lack
Scenario 1 changes accidentally	Do not edit generation/import source
Turbo/geocoding behavior disturbed	No changes to those participants

13. Tentative pattern candidates

These are candidates only. Confirmation requires inspecting the actual future implementation.

Candidate A — Strategy (GoF): Cancellation Policy Evaluation

Role	Proposed concrete participant
Context	`CancellationPolicies::QuoteEvaluator`
Strategy contract	`fee_rate(seconds_until_start:)`
ConcreteStrategy 1	`CancellationPolicies::Flexible`
ConcreteStrategy 2	`CancellationPolicies::Moderate`
ConcreteStrategy 3	`CancellationPolicies::Strict`
Client	`CancellationQuotesController#create`

Why it may qualify: Different algorithms for computing the fee are interchangeable under a stable contract. The evaluator selects one and delegates the variable behavior.

Why it might be rejected later: If implementation reduces to a single method with hard-coded conditionals or policy configuration values rather than actual interchangeable behavior objects, a separate Strategy classification may not be justified.

If eventually confirmed, all three concrete policies would form **one Strategy instance**, not three.

Candidate B — Controller (GRASP): Cancellation Quote Coordination

The potential coordinator is `CancellationQuotesController`, handling the distinct user event of requesting a hypothetical cancellation quote.

Its collaborators would be Trip, TripItem, TripMember, InboxItem and QuoteEvaluator.

This differs from P05 import, S1-N01 personalized generation and S2-N02 provider option coordination.

However, merely introducing a Rails controller does not confirm a new Controller pattern. The implemented responsibility, participants and delegation must be verified afterward.

Factory assessment

A fixed map from `"flexible"`, `"moderate"` and `"strict"` to classes is ordinary selection logic. Do not plan to count Factory Method or Abstract Factory without separate structural evidence.

14. Expected impact on existing 12 patterns

These are impact predictions, not evolution statuses.

ID	Existing collaboration	Expected impact
P01	Parsed Email to Trip Record Linking	Possibly affected — quote eligibility reads email-linked records
P02	Inbound Travel Email Intake Coordination	Likely unchanged
P03	Trip Creation Coordination	Likely unchanged
P04	Trip Membership Management Coordination	Likely unchanged
P05	Bulk Itinerary Import Coordination	Likely unchanged
P06	OAuth Sign-in and Identity Linking Coordination	Likely unchanged
P07	Account Traveler Enrollment Coordination	Likely unchanged
P08	Outbound Mail Delivery Logging	Likely unchanged
P09	Trip-Scoped Turbo Refresh Notifications	Likely unchanged
S1-N01	Personalized Itinerary Generation Coordination	Likely unchanged
S2-N01	Multiple Booking Provider Normalization	Likely unchanged
S2-N02	Multiple Booking Provider Option Coordination	Likely unchanged

The new quote workflow reads existing participant data, but no existing collaboration needs to be modified under the proposed scope.

The frozen current counts remain:

Type	Current count
Controller (GRASP)	8
Information Expert (GRASP)	1

Observer (GoF)	2
Adapter (GoF)	1
Total	**12**
Distinct types	**4**

No Strategy or additional Controller has been counted yet.

15. File-by-file coding sequence for the next prompt

1. **Prepare separate AFTER version**
   Copy the complete Scenario 2 source to scenario3_cancellation_policies; record the source ZIP checksum and preserve the previous snapshot.
2. **Write policy unit-test cases**
   Define fixed timestamps, fee-rate boundaries, expected monetary results and invalid-input behavior before implementation.
3. **Implement three policies**
   Create Flexible, Moderate and Strict with the common fee_rate(seconds_until_start:) contract.
4. **Implement the QuoteEvaluator**
   Validate eligibility and monetary data, select the approved policy, calculate amounts and produce the normalized result.
5. **Add controller tests**
   Cover owner/member authorization, eligible/ineligible records, policy selection, read-only operation and missing data.
6. **Implement CancellationQuotesController**
   Add new/create actions, authentication, trip-scoped item lookup, owner authorization and controlled error handling.
7. **Add nested routes**
   Update config/routes.rb with the singular cancellation_quote new/create resource without disturbing existing routes.
8. **Create the cancellation quote UI**
   Add app/views/cancellation_quotes/new.html.erb with policy selection, output and hypothetical-only notice.

Add TripItem entry point

9 Modify app/views/trip_items/show.html.erb with the owner-only Estimate cancellation (demo) link.

10 **Run verification**

Attempt Rails tests, record actual counts or exact environment blockers, check syntax and run any isolated deterministic policy tests.

11 **Check regressions**

Compare Scenario 2 and Scenario 3 source; verify generation, import, providers, inbox, Expenses, geocoding and schema remain unaffected.

12 **Package change evidence**

Write SCENARIO3_CHANGE_RECORD.md, source patch, test logs and the complete Scenario 3 AFTER ZIP. Defer final pattern classification.

Final Scenario 3 design decision

Implement a quote-only cancellation-policy demonstration using Flexible, Moderate and Strict calculation objects, with one evaluator and one controller.

The design deliberately avoids persistence and actual cancellation because the existing application does not have provider cancellation contracts or booking APIs. It uses real fields already present on email-linked TripItems, protects Scenario 1 and Scenario 2, and permits deterministic testing.

The source version for the next implementation step will be `scenario3_cancellation_policies` .

Scenario 3 design is complete. No source edits, migrations, pattern-status assignments or count changes were made.

Pasted text(20261011-074956).txt
Document

Worked for 5m 27s

DaPlanStan — Scenario 3 Implementation Complete

AFTER source packaged

Scenario 3 — Different Cancellation Policies has been implemented in a separate source version:

```
scenario3_cancellation_policies
```

The implementation starts from the completed Scenario 2 AFTER source, not the original DaPlanStan baseline.

It introduces three deterministic cancellation policies and a read-only quote interface. No external reservation is cancelled, no refund is issued and no booking or Expense record is modified by the new feature.

1. Download the deliverables

Scenario 3 Complete AFTER Source

Full Rails project · ZIP · Approximately 23 MB

Download Scenario 3 AFTER Source ↗

Scenario 2 → Scenario 3 Patch

Unified source patch · Approximately 49 KB · Application verified

Download Source Patch ↗

Scenario 3 Change Record

Source locations, policy rules, acceptance coverage, verification evidence and limitations

Download Change Record ↗

The ZIP passed integrity verification. The patch passed its dry-run check and reproduced the AFTER files byte-for-byte when applied to the Scenario 2 snapshot.

2. Exact source changes

3	15	0
Modified files	New files	Deleted files

Modified existing files

File	AFTER lines	Change
`config/routes.rb`	37	Added nested cancellation-quote resource
`app/views/trip_items/show.html.erb`	207–212	Added owner-only quote entry point

`README.md`	New Scenario 3 section	Documented feature, policy rules and verification

The route declaration is:

```ruby
resource :cancellation_quote, only: [ :new, :create ]
```

New production files

File	AFTER lines	Responsibility
`app/controllers/cancellation_quotes_controller.rb`	1–43	Authorization, eligibility and quote coordination
`app/services/cancellation_policies/flexible.rb`	1–13	Flexible cancellation calculation
`app/services/cancellation_policies/moderate.rb`	1–13	Moderate calculation
`app/services/cancellation_policies/strict.rb`	1–10	Strict calculation
`app/services/cancellation_policies/quote_evaluator.rb`	1–84	Eligibility, selection, money-safe evaluation and errors
`app/views/cancellation_quotes/new.html.erb`	1–35	Policy selection and hypothetical result interface

The remaining nine new files contain Rails tests, standalone validation checks, execution logs and the Scenario 3 change record.

3. Implemented cancellation policies

All policies implement:

```ruby
```

```
fee_rate(seconds_until_start:)
```

Their rules match the frozen design.

Time until booking starts	Flexible fee	Moderate fee	Strict fee
At least 72 hours	0%	0%	100%
24 to less than 72 hours	0%	50%	100%
More than 0 to less than 24 hours	10%	100%	100%
At or after start	100%	100%	100%

All three implementations are stateless and perform no database access or network calls.

Example: €200 booking, 48 hours before start

Flexible refund	Moderate refund	Strict refund
€200	**€100**	**€0**
€0 fee	€100 fee	€200 fee

These are application demonstration rules, not actual airline or hotel cancellation terms.

4. Eligibility implementation

`CancellationPolicies::QuoteEvaluator` independently checks that the item:

- Has `status = confirmed`.
- Has an associated InboxItem with confirmed review status.
- Contains its own confirmation reference.
- Has a valid start timestamp and nonnegative amount.
- Uses EUR, GBP or USD.

This means a Scenario 1 generated idea or Scenario 2 demo-provider idea cannot receive a quote merely because a user changes its status to confirmed.

The service also refuses to use another itinerary item's displayed fallback confirmation reference.

5. QuoteEvaluator behavior

The evaluator at `quote_evaluator.rb:32–55` selects one of the three explicitly permitted policies and calculates:

$$\text{Fee} = \text{round}_2(\text{Amount} \times \text{Rate})$$

$$\text{Refund} = \text{Amount} - \text{Fee}$$

It uses `BigDecimal` and `ROUND_HALF_UP` rather than floating-point money calculations.

The output includes the policy code and name, original amount, fee, refund, currency, refundable flag and explanation.

Unknown policies, negative amounts, missing required fields, unsupported currencies and invalid evaluation times produce controlled errors.

6. Final user workflow

Owner opens a confirmed email-linked TripItem

↓

Selects Estimate cancellation (demo)

↓

Controller verifies trip ownership and booking eligibility

↓

Owner selects Flexible, Moderate or Strict

↓

QuoteEvaluator calculates hypothetical fee and refund

↓

Application displays result and disclaimer

↓

TripItem, InboxItem and Expense remain unchanged

Both `new` and `create` enforce owner authorization in the controller. The POST action reads booking details from the server-loaded record, not browser-supplied amounts or booking fields.

The UI states explicitly that the result is hypothetical and that no reservation has been cancelled.

7. Read-only and persistence verification

Source inspection confirms that the new controller and evaluator contain no calls to save, update, destroy or create booking records.

They also do not invoke cancellation APIs, payment operations, email delivery or background jobs.

No migration or cancellation-status enum was added.

Important distinction: This is confirmed by source inspection and isolated calculation tests. Full database read-only behavior still requires the blocked Rails integration tests.

8. Tests written

Two new Rails test files were added:

File	Coverage
`test/services/cancellation_policies_test.rb`	Boundaries, all policies, money arithmetic, eligibility, invalid values and no mutation
`test/controllers/cancellation_quotes_controller_test.rb`	Owner access, unauthorized users, quote requests, tampering, read-only behavior and demo-item rejection

A separate `evidence/isolated_cancellation_policy_checks.rb` exercises the pure Ruby policy logic without Rails.

9. Actual test results

Standalone policy checks

10/10

45 assertions · 0 failures/errors

Ruby syntax checks

8/8

All Syntax OK

⚠ **Rails integration testing remains blocked.**

Test detail	Actual outcome
Available Ruby	3.3.8
Required Ruby	3.4.2
Focused Rails test command	Exit code 1
Full `ruby bin/rails test`	Exit code 1
Error	`Bundler::GemNotFound`
Missing dependencies	Rails 8.1.3 and other locked gems
Rails tests executed	0
Rails passes/failures/errors/skips	Not available

The full error logs are preserved in the ZIP under `evidence/`.

No live Anthropic, provider, geocoding or Turbo integration testing was performed.

10. Regression preservation

The file comparison confirms that the following source areas are unchanged from Scenario 2:

Existing functionality	Source-level status
Scenario 1 personalized generation	Preserved
Generated preview/confirmation	Preserved

Original JSON import	Preserved
Manual TripItem creation	Preserved
Instance-scoped geocoding	Preserved
Scenario 2 Atlas/Beacon adapters	Preserved
Provider browsing and duplicate detection	Preserved
Genuine email intake/linking	Preserved
Authentication and membership	Preserved
Existing Expense behavior	Preserved
Geocoding/Turbo mechanism	Preserved
Database schema/migrations	Unchanged

These are byte-level source-preservation results; runtime regression tests remain unexecuted.

11. Remaining limitations

The implementation intentionally cannot cancel an actual booking or guarantee a real refund. It does not retrieve provider-specific contractual terms.

Bookings without required start, amount, currency or confirmation data cannot receive estimates. No historical quote is persisted, and the controller uses the actual current time when generating a quote.

A compatible Rails environment is still required to verify rendering, authorization, database interactions and the complete application regression suite.

12. Pattern candidates for next verification

No new patterns have been confirmed during implementation.

Candidate	Collaboration to examine
Strategy (GoF)	QuoteEvaluator and the three interchangeable policy calculators

Controller (GRASP)	CancellationQuotesController coordinating a distinct quote-request event

A fixed policy lookup is not automatically a Factory.

The existing pattern counts remain unchanged pending source-level AFTER verification:

Pattern	Count
Controller (GRASP)	8
Information Expert (GRASP)	1
Observer (GoF)	2
Adapter (GoF)	1
Total instances	**12**
Distinct pattern types	**4**

Final handoff status

The complete Scenario 3 implementation, source patch, change record and available test evidence are packaged.

The source ZIP has **444 files**, including the unchanged Scenario 2 application and the fifteen additions. The input Scenario 2 archive remains unchanged.

Scenario 3 implementation is complete at the source level, but not fully Rails-runtime-verified. The next step is independent Scenario 3 AFTER verification and pattern-evolution classification.

No final pattern classifications, cross-scenario aggregation or cross-language work have been performed.

Scenario 3 — Different Cancellation Policies has now been implemented in:

scenario3_cancellation_policies

Do not modify the source.

The Scenario 2 verified state is frozen:

P01 Parsed Email to Trip Record Linking — Information Expert (GRASP)
P02 Inbound Travel Email Intake Coordination — Controller (GRASP)
P03 Trip Creation Coordination — Controller (GRASP)
P04 Trip Membership Management Coordination — Controller (GRASP)
P05 Bulk Itinerary Import Coordination — Controller (GRASP)
P06 OAuth Sign-in and Identity Linking Coordination — Controller (GRASP)
P07 Account Traveler Enrollment Coordination — Controller (GRASP)
P08 Outbound Mail Delivery Logging — Observer (GoF)
P09 Trip-Scoped Turbo Refresh Notifications — Observer (GoF)
S1-N01 Personalized Itinerary Generation Coordination — Controller (GRASP)
S2-N01 Multiple Booking Provider Normalization — Adapter (GoF)
S2-N02 Multiple Booking Provider Option Coordination — Controller (GRASP)

Current Scenario 2 counts:

Controller (GRASP) = 8
Information Expert (GRASP) = 1
Observer (GoF) = 2
Adapter (GoF) = 1
Total current instances = 12
Distinct pattern types = 4

Now independently verify Scenario 3 using the actual Scenario 3 AFTER ZIP and the Scenario 2 → Scenario 3 source patch.

Do not rely only on the implementation summary.

1. Verify the source changes.

Inspect at minimum:

- app/controllers/cancellation_quotes_controller.rb
- app/services/cancellation_policies/flexible.rb
- app/services/cancellation_policies/moderate.rb
- app/services/cancellation_policies/strict.rb
- app/services/cancellation_policies/quote_evaluator.rb
- app/views/cancellation_quotes/new.html.erb
- app/views/trip_items/show.html.erb
- config/routes.rb
- TripItem
- InboxItem
- Expense
- BookingOptionsController
- Scenario 1 generator/import source
- Scenario 2 provider services
- tests
- SCENARIO3_CHANGE_RECORD.md
- source patch

Record reliable AFTER file and line ranges.

2. Verify the implementation claims.

Check that:

- all three policy objects implement the same public fee-rate contract
- their boundary rules match the frozen design exactly
- QuoteEvaluator selects only approved policy codes
- no arbitrary constantize is used

- money handling uses BigDecimal
- fee rounding is explicit
- refund is derived from the rounded fee
- only EUR, GBP and USD are accepted
- zero amounts are valid
- negative amounts are rejected
- eligibility is checked server-side
- Scenario 1 generated ideas are ineligible
- Scenario 2 provider-demo ideas are ineligible
- no booking, InboxItem or Expense mutation occurs
- no cancellation API, payment operation or email cancellation is invoked
- no migration was introduced

If a defect is found, stop and report it rather than silently fixing it.

3. Reconstruct the actual Scenario 3 collaboration.

Trace:

eligible confirmed email-linked TripItem
→ cancellation quote route
→ CancellationQuotesController
→ authorization
→ QuoteEvaluator
→ policy selection
→ Flexible / Moderate / Strict
→ fee rate
→ BigDecimal fee/refund calculation
→ normalized quote
→ view
→ no persistence

Distinguish application participants from Rails/framework mechanisms.

4. Evaluate Strategy (GoF).

Candidate:

Cancellation Policy Evaluation

Possible roles:

Context:
CancellationPolicies::QuoteEvaluator

Strategy contract:
fee_rate(seconds_until_start:)

ConcreteStrategy:
CancellationPolicies::Flexible
CancellationPolicies::Moderate
CancellationPolicies::Strict

Client:
CancellationQuotesController

Determine whether the implementation genuinely represents interchangeable algorithms selected under one stable contract.

Do not confirm Strategy merely because there are three classes.

Final decision:

CONFIRMED
or
REJECTED

If confirmed:

- count all three policies together as ONE Strategy instance
- identify all participant roles
- record exact source locations
- explain why this is not just ordinary conditional code

5. Evaluate the new GRASP Controller candidate.

Candidate:

Cancellation Quote Coordination

Determine whether CancellationQuotesController coordinates a distinct system event separate from:

- P05 bulk import
- S1-N01 personalized generation
- S2-N02 booking-provider selection

Compare:

- event
- responsibility
- coordinator
- collaborators
- input

- outcome
- persistence behavior

Final decision:

CONFIRMED NEW INSTANCE
or
REJECTED AS SEPARATE INSTANCE

6. Evaluate Factory.

Inspect QuoteEvaluator's policy-code lookup.

Do not count Factory Method or Abstract Factory unless the actual structure genuinely supports them.

A fixed map or case expression is not automatically a Factory.

Give a final decision.

7. Check all existing pattern instances:

P01–P09
S1-N01
S2-N01
S2-N02

For each assign exactly one Scenario 3 status:

preserved
modified
extended

replaced
removed

Do not call something modified just because Scenario 3 reads data from its participants.

Pay special attention to P01 because cancellation eligibility reads email-linked records.

8. If Strategy and/or the new Controller are confirmed, add them with:

newly_introduced

Do not modify any earlier-version rows.

9. Recalculate Scenario 3 AFTER counts only after classification.

Show:

- Controller
- Information Expert
- Observer
- Adapter
- Strategy if confirmed
- total current instances
- total distinct pattern types

Do not count rejected candidates.

10. Verify cancellation-policy boundaries.

Check at minimum:

Flexible:
- >=72h = 0%
- exactly 24h = 0%
- below 24h = 10%
- at/after start = 100%

Moderate:
- >=72h = 0%
- exactly 24h = 50%
- below 24h = 100%
- at/after start = 100%

Strict:
- always 100%

Check exact 72h and 24h boundaries against actual implementation.

11. Verify eligibility.

Confirm an item requires:

- confirmed status
- linked InboxItem
- confirmed InboxItem review status
- own confirmation_ref
- starts_at
- amount
- supported currency

Confirm that a manually confirmed generated/demo idea is still rejected.

12. Verify read-only behavior.

Inspect the new implementation for:

save
save!
update
update!
destroy
create
create!
background jobs
mailer calls
external API calls

Confirm whether Scenario 3 itself writes any application record.

Do not confuse ordinary view rendering/session behavior with booking persistence.

13. Verify test evidence.

Separate:

- standalone cancellation-policy checks
- Ruby syntax checks
- Rails service/controller/integration tests
- live Anthropic
- live provider integration
- live geocoding/Turbo
- external cancellation calls

Do not produce a generic “all tests passed” statement.

If Rails is still blocked, preserve the exact blocker.

14. Verify regression preservation.

Check source-level preservation of:

- Scenario 1 generation
- generated preview/confirmation
- JSON import
- manual TripItem creation
- instance-scoped geocoding
- Scenario 2 provider adapters
- provider browsing and duplicate handling
- email parsing/linking
- Expense behavior
- authentication/membership
- geocoding/Turbo
- database schema

15. Prepare Scenario 3 evolution records.

For every current confirmed pattern instance include:

- ID
- stable instance name
- pattern name/category
- Scenario 3 status
- AFTER participants
- AFTER source locations

- concise reason
- change description

Add confirmed Scenario 3 instances separately.

16. Do not generate the final cross-scenario patterns_evolution.csv yet.

Prepare append-ready Scenario 3 data only.

17. Do not begin cross-language implementation yet.

At the end give me:

1. Source verification result.
2. Verified Scenario 3 workflow.
3. Strategy decision.
4. Controller decision.
5. Factory decision.
6. Status of P01–P09, S1-N01, S2-N01 and S2-N02.
7. Any newly introduced confirmed instances.
8. Scenario 3 AFTER pattern counts.
9. Policy-boundary verification.
10. Eligibility verification.
11. Read-only verification.
12. Actual test evidence separated by what really ran.
13. Regression preservation.
14. Remaining limitations.
15. Append-ready Scenario 3 evolution record.
16. Whether the three-scenario evolution phase is sufficiently documented for final aggregation.

Do not modify source.
Do not start cross-language work.

Worked for 4m 3s

DaPlanStan — Scenario 3 AFTER Verification

Source-level verification complete

I independently inspected the actual Scenario 3 AFTER ZIP, compared it with the Scenario 2 source snapshot, checked the source patch, and reran the available Ruby verification checks.

1. Final pattern-evolution decisions

The implementation supports **two newly introduced pattern instances**: one Strategy (GoF) and one Controller (GRASP).

Candidate	Verification decision
Cancellation Policy Evaluation — Strategy (GoF)	**CONFIRMED**
Cancellation Quote Coordination — Controller (GRASP)	**CONFIRMED NEW INSTANCE**
Factory Method / Abstract Factory	**REJECTED**

All 12 previously confirmed pattern collaborations are preserved relative to Scenario 2.

Scenario 2 instances
12
4 distinct pattern types

Scenario 3 AFTER instances
14
5 distinct pattern types

These are **source-supported pattern classifications**, not claims that the full Rails application has passed integration testing.

2. Source comparison

Verification	Actual result

Scenario 2 files	429
Scenario 3 files	444
Modified existing files	3
New files	15
Removed files	0
New database migrations	0
Original ZIP integrity	Passed
Scenario 3 ZIP integrity	Passed
Scenario 2 → 3 patch applicability	Passed

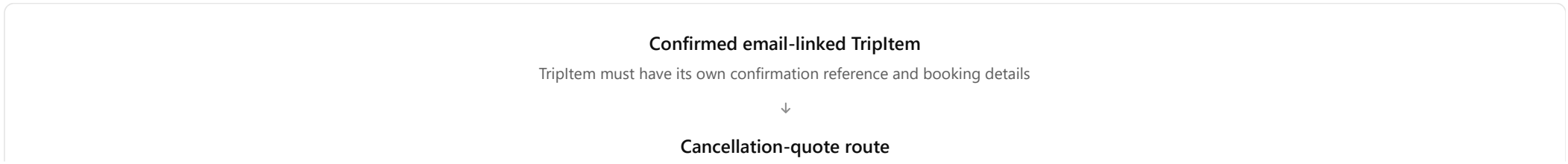
Only three existing files changed:

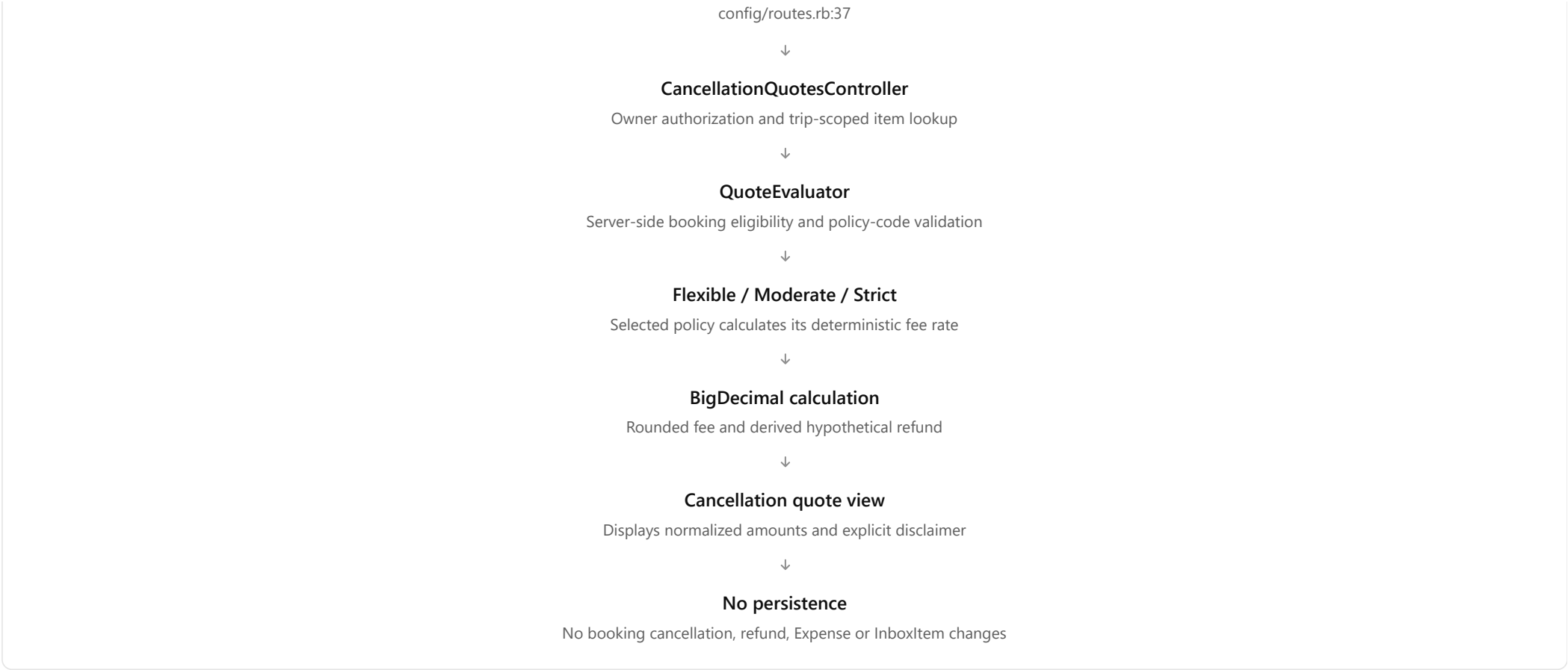
File	AFTER line range	Change
`config/routes.rb`	37	Nested singular cancellation-quote route
`app/views/trip_items/show.html.erb`	207–212	Owner-only, eligibility-checked quote link
`README.md`	New Scenario 3 section	Feature rules, limitations and verification instructions

All other application changes are contained in newly introduced cancellation-policy files.

The original Scenario 2 ZIP remains unchanged.

3. Verified Scenario 3 workflow





The application participants are `CancellationQuotesController`, `CancellationPolicies::QuoteEvaluator`, the three concrete policy objects, `TripItem`, `InboxItem`, `Trip`, and the existing membership context.

Rails routing, Action Controller callbacks, Active Record associations and Action View rendering are framework mechanisms. They are not additional pattern instances.

4. Strategy (GoF) — CONFIRMED

S3-N01 — newly_introduced

Instance name: Cancellation Policy Evaluation

Strategy role	Actual participant	AFTER source

Context	`CancellationPolicies::QuoteEvaluator#call`	`quote_evaluator.rb:32–56`
Strategy contract	`fee_rate(seconds_until_start:)`	All three policy files
ConcreteStrategy	`CancellationPolicies::Flexible`	`flexible.rb:3–12`
ConcreteStrategy	`CancellationPolicies::Moderate`	`moderate.rb:3–12`
ConcreteStrategy	`CancellationPolicies::Strict`	`strict.rb:3–9`
Client	`CancellationQuotesController#create`	`cancellation_quotes_controller.rb:14–22`
Policy selection	`QuoteEvaluator::POLICIES`	`quote_evaluator.rb:13–17,36–42`

This is a genuine interchangeable-algorithm collaboration, not merely three classes with different names.

Each policy implements the same method but supplies a different fee-rate algorithm. `QuoteEvaluator` delegates the variable calculation to the selected policy without embedding all three algorithms in its own conditional branches.

All three implementations participate in **one Strategy instance**, not three separate instances.

5. Controller (GRASP) — CONFIRMED NEW INSTANCE

S3-N02 — newly_introduced

Instance name: Cancellation Quote Coordination

`CancellationQuotesController` coordinates a distinct system event: an authenticated trip owner requests a hypothetical cancellation estimate.

It checks authorization, resolves the selected trip and item, verifies eligibility, invokes the evaluator and renders a result without saving anything.

Comparison	P05	S1-N01	S2-N02	S3-N02
User event	Import JSON	Generate itinerary	Choose provider offer	Request cancellation quote
Main input	JSON	Preferences	Provider/offer ID	Policy code
Specialized collaborator	Batch importer	Anthropic generator	Provider adapters	QuoteEvaluator

Outcome	Imported ideas	Personalized ideas	Provider idea	Hypothetical refund quote
Persistence	Yes	On confirmation	Yes	**None**

This is an independent coordination responsibility, not another count of an existing controller collaboration.

Factory decision: REJECTED. `QuoteEvaluator::POLICIES` is a fixed code-to-class map. It does not establish the polymorphic creator structure of Factory Method or the related-object-family structure of Abstract Factory.

6. Evolution status of all existing instances

Each existing pattern is evaluated against the **Scenario 2 AFTER version**, not against the original baseline.

ID	Stable instance name	Scenario 3 status
P01	Parsed Email to Trip Record Linking	`preserved`
P02	Inbound Travel Email Intake Coordination	`preserved`
P03	Trip Creation Coordination	`preserved`
P04	Trip Membership Management Coordination	`preserved`
P05	Bulk Itinerary Import Coordination	`preserved`
P06	OAuth Sign-in and Identity Linking Coordination	`preserved`
P07	Account Traveler Enrollment Coordination	`preserved`
P08	Outbound Mail Delivery Logging	`preserved`
P09	Trip-Scoped Turbo Refresh Notifications	`preserved`
S1-N01	Personalized Itinerary Generation Coordination	`preserved`
S2-N01	Multiple Booking Provider Normalization	`preserved`
S2-N02	Multiple Booking Provider Option Coordination	`preserved`

S3-N01	**Cancellation Policy Evaluation**	`newly_introduced`
S3-N02	**Cancellation Quote Coordination**	`newly_introduced`

Important preservation evidence

P01 remains preserved because Scenario 3 reads the existing `InboxItem` association and review status but does not alter the email-to-trip linking responsibility (`app/models/inbox_item.rb:19-207`).

P05 remains preserved because `TripItemsController#import` and `bulk_create_items` are unchanged (`trip_items_controller.rb:31-58,199-259`).

P09 remains preserved because neither `GeocodeTripItemsJob` nor the existing Turbo subscriptions were modified or invoked by cancellation quotes.

S1-N01 and both Scenario 2 collaborations are likewise unchanged at the source level.

AFTER participant/location summary

ID	Principal application participants	Key AFTER locations
P01	InboxItem, InboxItemsController, TripItem, Expense	`inbox_item.rb:19-207`
P02	TripMailbox, InboxItem, ParseInboxItemJob	`trip_mailbox.rb:2-60`
P03	TripsController, Trip, TripMember	`trips_controller.rb:35-48`
P04	TripMembersController, TripMember, Traveler	`trip_members_controller.rb:6-65`
P05	TripItemsController, TripItem, GeocodeTripItemsJob	`trip_items_controller.rb:31-58,199-259`
P06	SessionsController, UserIdentity, User	`sessions_controller.rb:4-81`
P07	TravelersController, Account, Traveler	`travelers_controller.rb:5-56`
P08	MailLogger, InvitationMailer, TripMailer	`mail_logger.rb:1-7`
P09	InboxItem, GeocodeTripItemsJob, Turbo subscriptions	`geocode_trip_items_job.rb:4-21`

S1-N01	TripItemsController, ItinerarySuggestionGenerator, TripItem	`itinerary_suggestion_generator.rb:42-153`
S2-N01	Registry, AtlasAdapter, BeaconAdapter	`booking_providers/registry.rb:11-66`
S2-N02	BookingOptionsController, Registry, TripItem	`booking_options_controller.rb:1-77`
S3-N01	QuoteEvaluator, Flexible, Moderate, Strict	`cancellation_policies/*.rb`
S3-N02	CancellationQuotesController, Trip, TripItem, InboxItem, QuoteEvaluator	`cancellation_quotes_controller.rb:1-43`

The append-ready file provides the full participant-role mappings, supporting paths, exact ranges, evidence, reasons and change descriptions for all 14 instances.

7. Scenario 3 AFTER counts

Confirmed instances across evolution versions

Pattern type	Scenario 2	Scenario 3 AFTER
Controller (GRASP)	8	**9**
Information Expert (GRASP)	1	1
Observer (GoF)	2	2
Adapter (GoF)	1	1
Strategy (GoF)	0	**1**
Total current instances	**12**	**14**
Distinct pattern types	**4**	**5**

The increase of two reflects one new Strategy collaboration and one new Controller collaboration. The three concrete policies count together as one Strategy instance.

8. Cancellation-policy boundary verification

I checked the actual implementations in:

- `flexible.rb:6–10`
- `moderate.rb:6–10`
- `strict.rb:6–7`
- `quote_evaluator.rb:40–44`

The resulting fee rates match the frozen specification.

Evaluation time	Flexible	Moderate	Strict
More than 72h before start	0%	0%	100%
Exactly 72h	0%	0%	100%
1 second below 72h	0%	50%	100%

Exactly 24h	0%	50%	100%
1 second below 24h	10%	100%	100%
1 second before start	10%	100%	100%
Exactly at start	100%	100%	100%
After start	100%	100%	100%

The evaluator uses `BigDecimal`, rounds the fee to two decimal places with `ROUND_HALF_UP`, and derives the refund by subtracting that rounded fee.

For example, on €0.05 with a Moderate 50% fee, the result is a €0.03 fee and €0.02 refund. This calculation is covered by the executed standalone checks.

9. Eligibility verification

The eligibility conditions are enforced in `CancellationPolicies::QuoteEvaluator#validate_booking!` (`quote_evaluator.rb:60–73`).

Condition	Source-level result
TripItem status must be confirmed	Verified
`inbox_item_id` must exist	Verified
Associated InboxItem must exist	Verified
InboxItem review status must be confirmed	Verified
Item must have its own confirmation reference	Verified
Start timestamp must be valid	Verified
Amount must be present	Verified
Zero amount allowed	Verified
Negative amount rejected	Verified
Only EUR, GBP and USD allowed	Verified

Explicit valid evaluation time required	Verified

The quote evaluator does not use a sibling item's confirmation-reference fallback.

Manually confirming a Scenario 1 generated idea or Scenario 2 demo offer does not, by itself, satisfy the required email linkage and reference conditions.

10. Read-only verification

The new controller and policy services were searched for model-persistence operations, background-job calls, mail delivery and external cancellation requests.

No application-record write or external cancellation operation was found in the new Scenario 3 quote workflow.

`CancellationQuotesController#create` is an HTTP POST action name; it does not call Active Record `create` or `create!`.

The application loads the existing item and calculates an in-memory result. Its view displays a hypothetical estimate and makes no claim that a cancellation was submitted.

This is verified from source inspection. Database read-only behavior in a running Rails application remains unverified until integration tests can execute.

11. Actual test evidence

I independently reran the checks against the extracted Scenario 3 source.

Standalone policy tests

10/10

45 assertions · No failures/errors/skips

Ruby syntax checks

8/8

All Syntax OK

Verification category	Actual outcome
Standalone cancellation-policy checks	Passed, exit 0
Ruby syntax checks	Passed, exit 0
Rails service/controller tests	Blocked, 0 executed
Full Rails integration/regression suite	Previously blocked by the same environment issue

Live Anthropic	Not run
Live demo-provider application flow	Not run
Live geocoding/Turbo	Not run
External cancellation requests	None by design

I attempted:

```bash
ruby bin/rails test test/services/cancellation_policies_test.rb test/controllers/cancellation_quotes_controller_test.rb
```

The command exited with code 1 before test discovery:

```
Bundler::GemNotFound:
Could not find rails-8.1.3 ...
```

The available Ruby version is 3.3.8; the repository specifies 3.4.2, and required locked gems are unavailable.

No Rails test passes are claimed.

12. Regression preservation and remaining limitations

The archive comparison confirms byte-identical source for Scenario 1's generator/import workflows, Scenario 2's provider services and controller, inbox ingestion, authentication, membership, Expenses, instance-scoped geocoding, background jobs and Turbo subscriptions.

The database schema remains unchanged. Only the intended cancellation quote route and UI entry point were added to existing application files.

No source-level defect was identified against the approved quote-only specification that would require stopping the verification.

Remaining limitations are:

- Full Rails rendering, authorization and persistence assertions are not runtime-verified.
- The policies are fictional estimates, not actual provider contractual terms.
- Bookings without sufficient reference, amount, currency or start-time data cannot receive quotes.
- Quotes are not persisted as historical records.
- Real-world cancellation and refund operations are deliberately unsupported.

13. Append-ready Scenario 3 records

Independent Scenario 3 Verification Report

Detailed source audit, pattern decisions, all 14 participant mappings, test evidence and limitations

Download Verification Report ↗

Scenario 3 Append-Ready Evolution Data

14 records · Previous IDs preserved · Two new confirmed collaborations

Download Scenario 3 Evolution JSON ↗

The JSON includes `source_version`, pattern ID, stable instance name, pattern/category, participant roles, source locations, evidence, status and change description.

It is deliberately separate from the final cross-scenario `patterns_evolution.csv`. No earlier-version rows were changed.

14. Is the three-scenario evolution phase ready for final aggregation?

Yes — for source-level documentation and pattern-evolution aggregation.

- ✓ Scenario 1 implementation and source verification documented
- ✓ Scenario 2 implementation and source verification documented
- ✓ Scenario 3 implementation and source verification documented
- ✓ Stable IDs, participants, evolution statuses and version counts established

Full Rails integration and regression tests still require a compatible environment

The three-scenario source-evolution result is **14 confirmed pattern instances across 5 distinct pattern types**, compared with the original nine instances across three types.

The next stage can assemble the final evolution dataset from all four frozen source versions while preserving the original baseline records and the documented runtime-verification limitations.

No final CSV has been generated, and no cross-language implementation has been started.

The three software-evolution scenarios are now complete and source-verified.

Do not modify any application source code.

Use only the frozen verified records from:

Original baseline
Scenario 1 — scenario1_personalized_itinerary
Scenario 2 — scenario2_multiple_booking_providers
Scenario 3 — scenario3_cancellation_policies

The original baseline must remain unchanged.

FINAL VERIFIED PATTERN STATE BY VERSION

Original baseline:
P01 Information Expert (GRASP) — Parsed Email to Trip Record Linking
P02 Controller (GRASP) — Inbound Travel Email Intake Coordination
P03 Controller (GRASP) — Trip Creation Coordination
P04 Controller (GRASP) — Trip Membership Management Coordination
P05 Controller (GRASP) — Bulk Itinerary Import Coordination
P06 Controller (GRASP) — OAuth Sign-in and Identity Linking Coordination

P07 Controller (GRASP) — Account Traveler Enrollment Coordination
P08 Observer (GoF) — Outbound Mail Delivery Logging
P09 Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

Baseline counts:
Controller = 6
Information Expert = 1
Observer = 2
Total instances = 9
Distinct types = 3

Scenario 1:
P01 preserved
P02 preserved
P03 preserved
P04 preserved
P05 modified
P06 preserved
P07 preserved
P08 preserved
P09 preserved
S1-N01 Controller (GRASP) — Personalized Itinerary Generation Coordination — newly_introduced

Scenario 1 counts:
Controller = 7
Information Expert = 1
Observer = 2
Total = 10
Types = 3

Scenario 2:

P01–P09 preserved relative to Scenario 1
S1-N01 preserved
S2-N01 Adapter (GoF) — Multiple Booking Provider Normalization — newly_introduced
S2-N02 Controller (GRASP) — Multiple Booking Provider Option Coordination — newly_introduced

Scenario 2 counts:
Controller = 8
Information Expert = 1
Observer = 2
Adapter = 1
Total = 12
Types = 4

Scenario 3:
P01–P09 preserved relative to Scenario 2
S1-N01 preserved
S2-N01 preserved
S2-N02 preserved
S3-N01 Strategy (GoF) — Cancellation Policy Evaluation — newly_introduced
S3-N02 Controller (GRASP) — Cancellation Quote Coordination — newly_introduced

Scenario 3 counts:
Controller = 9
Information Expert = 1
Observer = 2
Adapter = 1
Strategy = 1
Total = 14
Types = 5

Runtime limitation:

Full Rails integration/regression testing is still blocked because the available environment uses Ruby 3.3.8 while the repository specifies Ruby 3.4.2 and the locked Rails 8.1.3 dependencies are unavailable.
Do not convert this into a passed-test claim.

Now perform FINAL EVOLUTION AGGREGATION ONLY.

1. Read the frozen baseline pattern records plus the append-ready Scenario 1, Scenario 2 and Scenario 3 evolution records.

Do not infer missing data from memory if the actual files contain more precise participants or source locations.

2. Build the faculty-required final:

patterns_evolution.csv

Use EXACT columns:

source_version,
pattern_name,
pattern_type,
instance_name,
participant_roles,
source_locations,
intent_and_evidence,
status_after_change,
change_description

Do not add extra columns.

3. Preserve every original baseline row.

For the original source version use status:

baseline

Do not rewrite the original pattern classifications.

4. Add rows for each successive source version.

Each source version must contain the complete set of pattern collaborations existing in that version, not only the newly changed ones.

Use stable instance names across versions.

For a continuing instance use the appropriate status for that transition:

preserved
modified
extended
replaced
removed

For a newly introduced collaboration use:

newly_introduced

No currently verified instance has been removed or replaced unless the frozen records explicitly say so.

5. Use the verified source-version names consistently:

baseline

scenario1_personalized_itinerary

scenario2_multiple_booking_providers

scenario3_cancellation_policies

If the existing baseline records use a slightly different exact source_version label, preserve that existing label consistently instead of silently creating two baseline names.

6. Ensure the version row counts are correct.

Expected current-instance counts:

baseline = 9
Scenario 1 = 10
Scenario 2 = 12
Scenario 3 = 14

Check that the CSV contains the correct number of records for each version.

7. Build the final:

pattern_counts.csv

Use EXACT columns:

source_version,summary,pattern_name,count

For each source version provide:

per_pattern rows

total_pattern_types row
total_pattern_instances row

Expected counts:

BASELINE
Controller (GRASP) = 6
Information Expert (GRASP) = 1
Observer (GoF) = 2
total_pattern_types = 3
total_pattern_instances = 9

SCENARIO 1
Controller (GRASP) = 7
Information Expert (GRASP) = 1
Observer (GoF) = 2
total_pattern_types = 3
total_pattern_instances = 10

SCENARIO 2
Controller (GRASP) = 8
Information Expert (GRASP) = 1
Observer (GoF) = 2
Adapter (GoF) = 1
total_pattern_types = 4
total_pattern_instances = 12

SCENARIO 3
Controller (GRASP) = 9
Information Expert (GRASP) = 1
Observer (GoF) = 2

Adapter (GoF) = 1
Strategy (GoF) = 1
total_pattern_types = 5
total_pattern_instances = 14

Validate these counts from the evolution records rather than merely copying the expected numbers.

8. Check the original patterns.csv.

Do not overwrite or retroactively change the frozen baseline patterns.csv.

If a final multi-version patterns.csv is required by the faculty instructions, prepare it separately using the verified records while preserving baseline data.

Explain exactly what file should be retained as the frozen baseline and what file is the evolution dataset.

9. Produce a concise BEFORE/AFTER evolution summary for the final report.

Cover:

Scenario 1:
personalized itinerary generation
P05 modified
new Controller S1-N01

Scenario 2:
multiple booking providers
new Adapter S2-N01
new Controller S2-N02

Scenario 3:

different cancellation policies
new Strategy S3-N01
new Controller S3-N02

Explain the architectural progression without claiming that more patterns automatically means better design.

10. Produce one evolution-status table showing every stable pattern ID across:

baseline
Scenario 1
Scenario 2
Scenario 3

Use — where an instance did not yet exist.

11. Produce the final source-version count comparison table.

Include:

pattern type counts
total instances
distinct pattern types

for all four versions.

12. Consolidate the three refactoring records.

Scenario 1:
global TripItem callback manipulation replaced by instance-scoped geocoding deferral.

Scenario 2:

provider-specific data differences isolated behind provider normalizers/adapters.

Scenario 3:
cancellation calculations isolated into deterministic interchangeable policies rather than controller conditionals.

For each state:

- original problem/design pressure
- implemented change
- benefit
- trade-off
- runtime-verification limitation

Do not invent a formal refactoring name where one was not actually established.

13. Consolidate test evidence.

Clearly separate:

- source comparison/integrity checks
- Ruby syntax checks
- standalone Scenario 1 checks
- standalone Scenario 2 provider checks
- standalone Scenario 3 cancellation checks
- Rails integration tests
- live external integrations

State that Rails integration tests remain unexecuted because of the environment blocker.

Do not combine standalone checks into “application tests passed.”

14. Consolidate remaining limitations.

Include at minimum:

- Ruby/Rails runtime verification outstanding
- live Anthropic generation not verified
- live geocoding/Turbo not verified
- Scenario 2 providers are fictional/local demonstrations
- Scenario 3 policies are hypothetical demonstration policies
- no real provider booking or cancellation is performed

15. Prepare the evolution/refactoring report section for the final team report.

It should contain:

- methodology
- original baseline
- Scenario 1
- Scenario 2
- Scenario 3
- pattern evolution
- before/after counts
- refactoring analysis
- verification/testing evidence
- limitations

Use concise academic prose suitable for the faculty case-study report.

16. Prepare the final evolution handoff for the teammate doing cross-language analysis.

State the final confirmed pattern types available after Scenario 3:

Controller (GRASP)
Information Expert (GRASP)
Observer (GoF)
Adapter (GoF)
Strategy (GoF)

Do not choose the cross-language representative instances yet unless the faculty requirement explicitly requires all pattern types rather than two selected patterns.

Flag the exact faculty requirement if there is any mismatch with the earlier team division assumption.

17. Check filenames and faculty terminology carefully.

The faculty instructions use:

patterns_evolution.csv

Do not accidentally use misspellings such as:

pattern_evalution.csv

unless reproducing a folder example while explicitly noting the typo.

18. Create downloadable artifacts:

- patterns_evolution.csv
- pattern_counts.csv
- final evolution/refactoring report section in Markdown
- evolution status matrix CSV or Markdown
- cross-language handoff Markdown

- one ZIP containing these final evolution artifacts

19. Do not modify application source.
20. Do not start cross-language code yet.

At the end give me:

1. Final evolution dataset validation.
2. Final counts validation.
3. Evolution-status matrix.
4. Scenario-by-scenario summary.
5. Refactoring summary.
6. Test/verification summary.
7. Remaining limitations.
8. Cross-language handoff.
9. Download links for every generated artifact.
10. Any issue that still requires manual/human action before final submission.

Worked for 3m 51s

DaPlanStan — Final Software Evolution Aggregation

Aggregation complete and validated

The final evolution datasets, refactoring report, status matrix and cross-language handoff have been created from the frozen baseline and the three verified Scenario AFTER records.

No application source files or frozen baseline records were modified.

1. Download final evolution deliverables

Complete Final Evolution Package

ZIP containing all seven final artifacts and validation evidence

Download All Evolution Artifacts ↗

Individual artifact	Contents
patterns_evolution.csv ↗	Faculty-required nine-column dataset, 45 versioned records
pattern_counts.csv ↗	Four-version pattern counts and totals
Evolution and Refactoring Report ↗	Academic report section ready for team-report integration
Evolution Status Matrix ↗	All 14 stable IDs across four versions
Cross-Language Handoff ↗	Final confirmed types, evidence requirements and teammate instructions
Multi-Version Patterns Candidate ↗	Optional seven-column dataset for faculty `patterns.csv` assembly
Aggregation Validation Record ↗	File checks, row counts, source-path audit and submission checklist

2. Final dataset validation

Check	Result
Frozen baseline rows retained	9 of 9
Scenario 1 rows	10
Scenario 2 rows	12
Scenario 3 rows	14
Total evolution records	**45**
`patterns_evolution.csv` columns	Exactly 9
`pattern_counts.csv` columns	Exactly 4
Count summary rows	23
Distinct collaboration IDs	14

Removed/replaced instances	0
Source path and line-bound checks	Passed for archived application paths
ZIP integrity	Passed

The baseline label was confirmed as `baseline`, and the original seven fields of all nine baseline records were preserved exactly. Only the required evolution fields were added, with `status_after_change = baseline`.

The source-location fields retain the verified record text. P09's external Turbo Rails dependency references remain labeled as external rather than being incorrectly treated as files inside the application ZIP.

3. Final before/after counts

Confirmed pattern instances across versions

Pattern type	Baseline	S1	S2	S3
Controller (GRASP)	6	7	8	9
Information Expert (GRASP)	1	1	1	1
Observer (GoF)	2	2	2	2
Adapter (GoF)	0	0	1	1
Strategy (GoF)	0	0	0	1
Total instances	**9**	**10**	**12**	**14**
Distinct types	**3**	**3**	**4**	**5**

The final inventory contains **14 instances across 5 pattern types**. This represents five newly introduced collaborations across the three scenarios, with no confirmed removals or replacements.

4. Complete evolution-status matrix

Statuses describe changes relative to the immediately preceding version.

Pattern ID	Baseline	S1	S2	S3
P01	baseline	preserved	preserved	preserved
P02	baseline	preserved	preserved	preserved
P03	baseline	preserved	preserved	preserved
P04	baseline	preserved	preserved	preserved
P05	baseline	modified	preserved	preserved
P06	baseline	preserved	preserved	preserved
P07	baseline	preserved	preserved	preserved
P08	baseline	preserved	preserved	preserved
P09	baseline	preserved	preserved	preserved
S1-N01	—	newly_introduced	preserved	preserved

S2-N01	—	—	newly_introduced	preserved
S2-N02	—	—	newly_introduced	preserved
S3-N01	—	—	—	newly_introduced
S3-N02	—	—	—	newly_introduced

The CSV matrix also contains every stable instance's full name, pattern classification and category.

5. Scenario-by-scenario evolution summary

Scenario	Architectural evolution	Confirmed pattern effects
Scenario 1 — Personalized itinerary generation	Introduced an Anthropic-backed preference-to-itinerary workflow with preview and existing TripItem persistence	P05 modified; S1-N01 Controller newly introduced
Scenario 2 — Multiple booking providers	Added two local provider implementations that normalize different response formats	S2-N01 Adapter and S2-N02 Controller newly introduced
Scenario 3 — Different cancellation policies	Introduced interchangeable deterministic fee calculators with a read-only quote workflow	S3-N01 Strategy and S3-N02 Controller newly introduced

The architecture progresses from existing trip and email coordination toward more specialized generation, provider normalization and cancellation-policy evaluation.

This demonstrates separation of responsibilities, but an increase in pattern counts is **not, by itself, proof of improved design quality**.

6. Consolidated refactoring analysis

Scenario	Original pressure	Implemented change	Benefit and trade-off
S1	Global `TripItem.skip_callback / set_callback` affected model callback configuration	Instance-scoped `defer_geocoding` mechanism	Avoids globally suppressing callbacks; introduces deferred-job coordination and still requires runtime concurrency verification

S2	Incompatible provider response formats	Atlas and Beacon normalization behind a common interface	Keeps provider-specific schemas out of controller code; remains limited to fictional local catalogues
S3	Different cancellation fee rules could concentrate conditional logic in the controller	Stateless interchangeable Flexible, Moderate and Strict policy objects	Supports isolated, deterministic calculations; quotes remain hypothetical rather than contractual

The report documents the actual changes in plain software-engineering terms without assigning unsupported formal refactoring labels.

7. Consolidated test and verification evidence

Verification tier	Scenario 1	Scenario 2	Scenario 3
Ruby syntax	9/9 checks passed	9/9 checks passed	8/8 checks passed
Standalone checks	6 tests, 20 assertions	10 tests, 67 assertions	10 tests, 45 assertions
Standalone failures/errors	0	0	0
Rails integration tests	Blocked	Blocked	Blocked
Live Anthropic	Not verified	Not retested	Not retested
Live geocoding/Turbo	Not verified	Not verified	Not verified
Live provider booking/cancellation	Not performed	Not performed by design	Not performed by design

The Rails blocker remains:

Plain text

```
Available Ruby: 3.3.8
Required Ruby: 3.4.2
Required locked Rails: 8.1.3
```

```
Bundler::GemNotFound:
Could not find rails-8.1.3 ...
```

Zero Rails integration tests executed. The source comparisons and standalone tests are valuable evidence, but they cannot be reported as completed Rails application tests.

8. Cross-language handoff

The final source-confirmed pattern types are:

Pattern type	Current instances
Controller (GRASP)	9
Information Expert (GRASP)	1
Observer (GoF)	2
Adapter (GoF)	1
Strategy (GoF)	1

⚠ **Important faculty requirement**

The uploaded faculty instructions require one representative problem **for each detected pattern type**, implemented in Java, Python, JavaScript and C++.

That implies **5 representative pattern problems × 4 languages = 20 implementations**.

If the team's earlier division assumed only two selected patterns, that would not satisfy the provided written requirement unless faculty explicitly approves a different scope.

The handoff does not select representative instances or write any cross-language code. It includes the required CSV fields, measurement rules, shared-fixture expectations, actual test reporting, and the specified one-warmup/five-measured-run timing procedure.

9. Remaining manual actions before submission

- **Run Rails tests** using Ruby 3.4.2 and the locked dependencies, preserving actual execution logs and counts.

- **Review the final CSV evidence** with a human teammate, especially participant-role mappings and the five newly introduced collaborations.
- **Assemble the faculty team folder.** Preserve the original frozen `DaPlanStan_Part2/patterns.csv`. Use `patterns_evolution.csv` for the complete version/status history. The separate `patterns_multiversion_candidate.csv` can be used to prepare a team-root seven-column `patterns.csv` if required, without overwriting the baseline archive.
- **Confirm cross-language scope** against the written five-pattern-type requirement before dividing work.
- **Finalize submission evidence**, including team contributions, source provenance, source-relative paths and any historical LLM prompt/output archive gaps noted in the baseline package.

Final outcome

- ✓ Evolution and count CSVs generated and validated
- ✓ Original nine-pattern baseline preserved
- ✓ All 45 versioned records included
- ✓ Academic report section and cross-language handoff prepared
- ✓ Final ZIP integrity verified
- ⓘ Rails runtime verification and human approval still outstanding

The final software-evolution aggregation is complete and ready for team-report integration. The resulting source-level inventory is 14 confirmed instances across 5 distinct pattern types.

No application source was modified, and no cross-language implementation was started.

